软件工程与方法

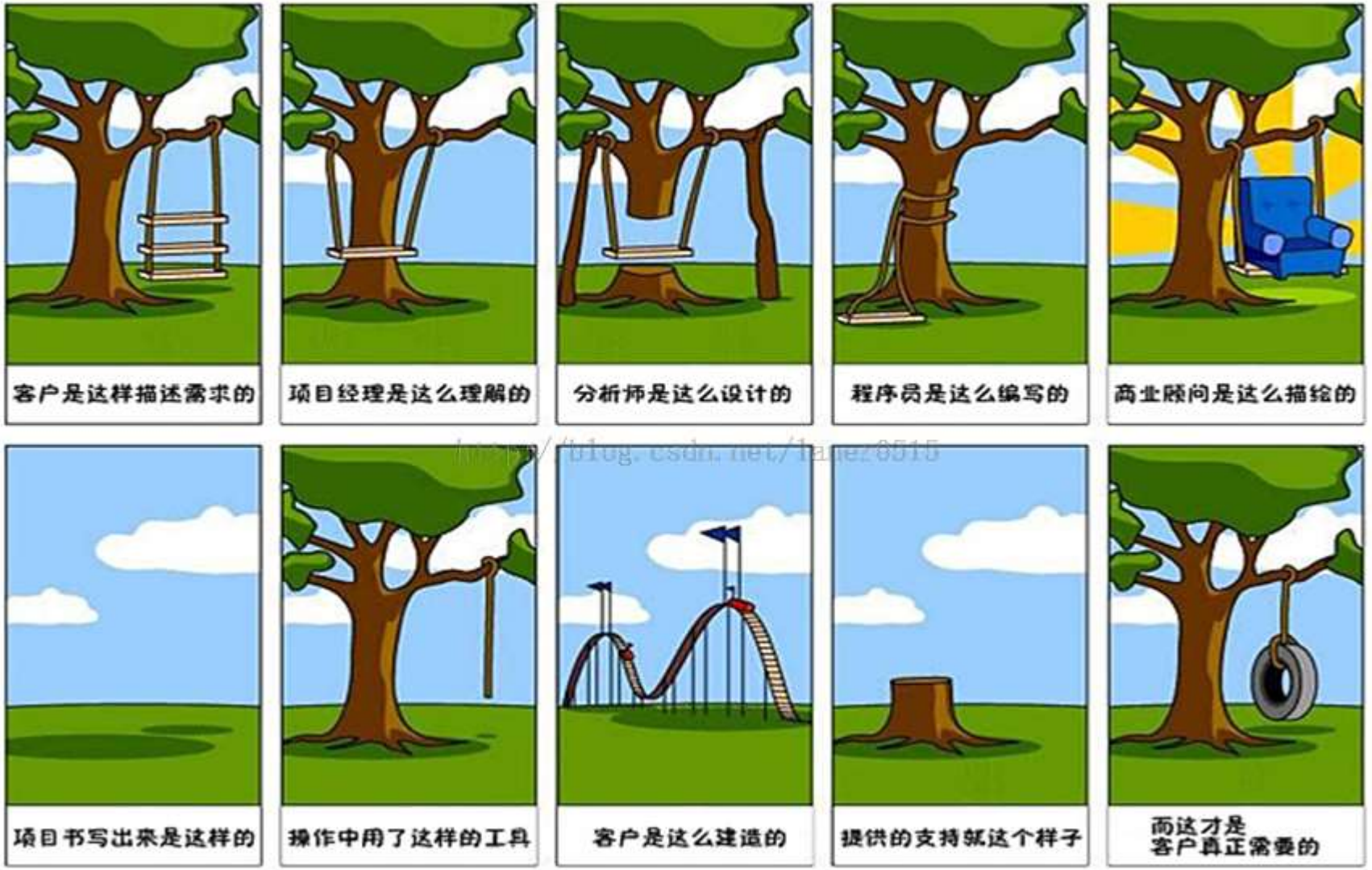
第4讲 软件开发

本章内容

- 系统工程
- 需求分析
- 软件设计
- 软件实现



软件开发面临的严重问题



软件开发的要素（essence）



规划解决方案
（建模和软件设计）

Plan a solution
(modeling and software design)

执行规划
（代码生成）

Carry out the plan
(code generation)

理解问题
（通信和分析）

Understand the problem
(communication and analysis)

为了准确性检查结果
（测试和质量保证）

Examine the result for accuracy
(testing and quality assurance)

软件开发

1 系统工程

主要内容

① 计算机系统工程
② 业务流程工程
③ 产品工程

1.1 基于计算机的系统

- 系统工程（system engineering）关注于将客户的需求、期望以及提出的限制条件转化到产品的解决方案中，并且在产品的整个生命周期中对解决方案给予始终的支持。
- 系统工程涵盖了一个系统所有的开发工作，其中不一定包括软件。
- 计算机系统工程
  - 通常把基于计算机的系统定义为某些元素的一个集合，并把这些元素组织起来以实现某种方法、过程或借助处理信息进行控制。
  - 一个问题求解活动，目的是揭示、分析所期望的功能，并把它们分配到系统元素中去。
  - 它一定包括计算机，并且包括软件。

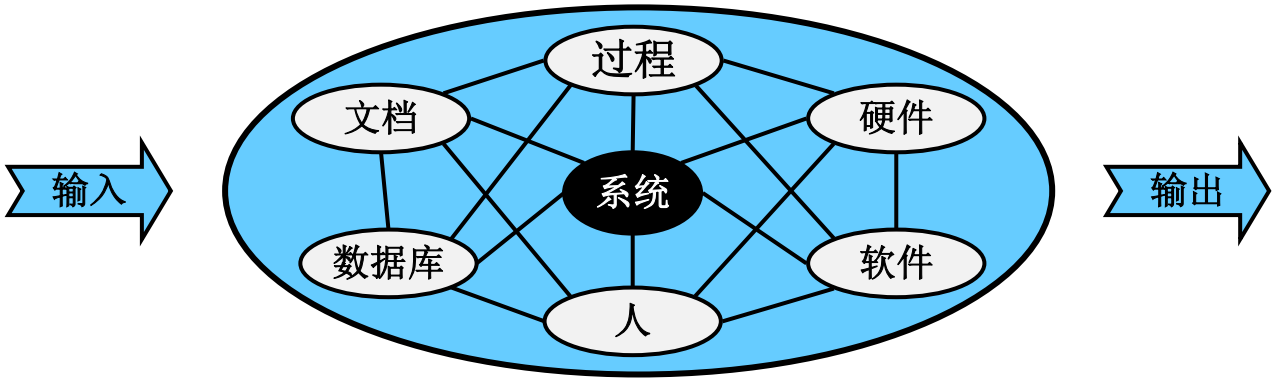
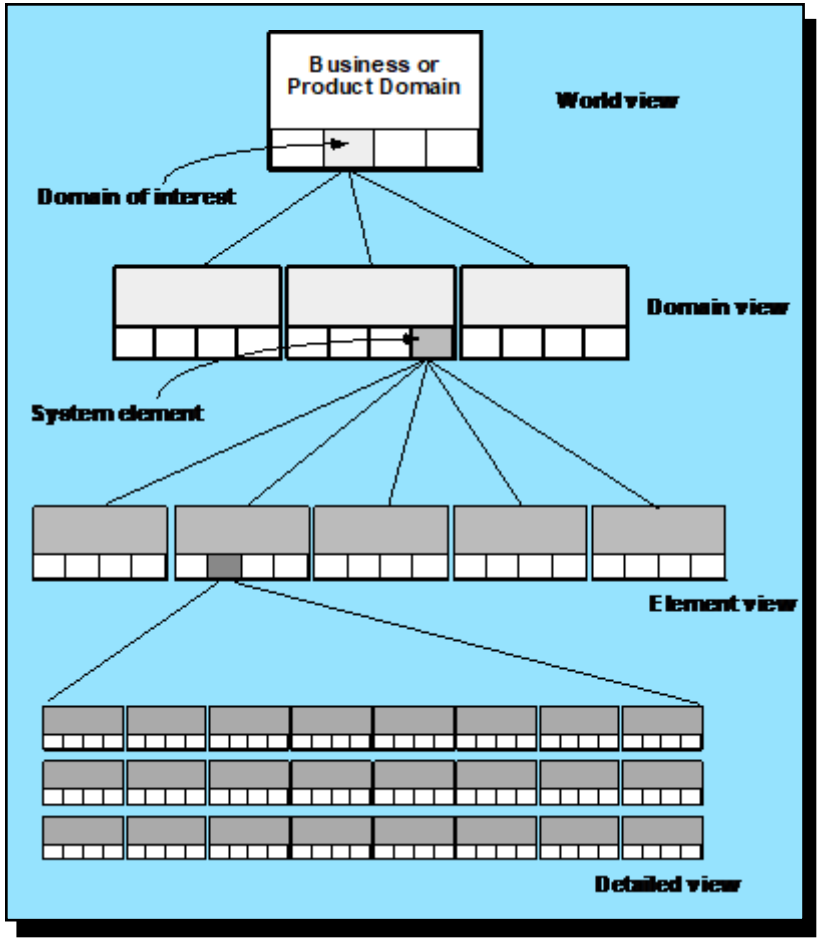


图 基于计算机系统的系统元素

The system engineering of hierarchy



- 全局视图

WV（*World View*）的形式化表示：

$$WV = \{ D_1, D_2, D_3, \ldots, D_n \}$$

$$D_j = \{ E_1, E_2, E_3, \ldots, E_m \}$$

$$E_k = \{ C_1, C_2, C_3, \ldots, C_k \}$$

其中：

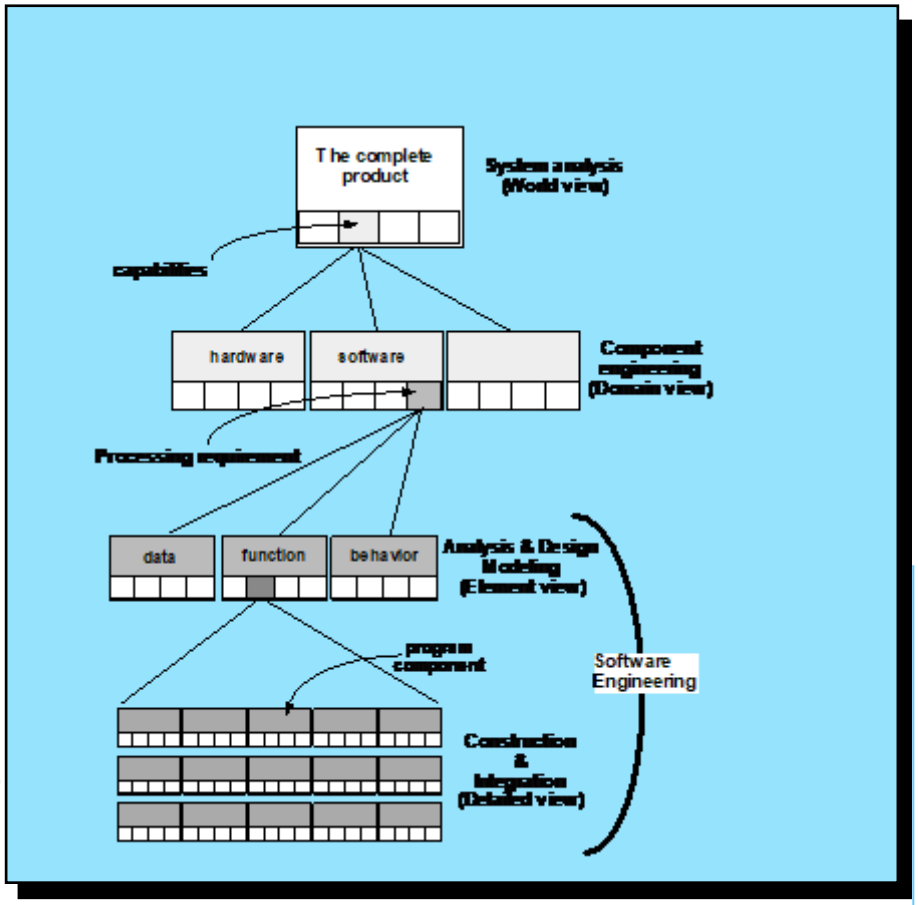
- D_i 表示域（*domain*）
- E_j 表示元素（*element*）
- C_k 表示组件（*component*）

1.2 业务流程工程

- 业务流程工程（BPE，Business Process Engineering）的目标是定义一个体系结构（architecture），使得业务能够高效地使用信息。
- 需要分析和设计的体系结构：
  - 数据体系结构（data architecture）
  - 应用体系结构（application architecture）
  - 技术基础设施（technology infrastructure）
- 业务流程工程的主要任务
  - 使用集成的过程、方法和工具识别系统如何能最好地满足企业的战略目标。
  - 首先是关注企业，其次才是商业领域。
  - 建立企业模型、数据模型和流程模型。
  - 建立一个框架，以便获得更好地信息管理分配和控制。

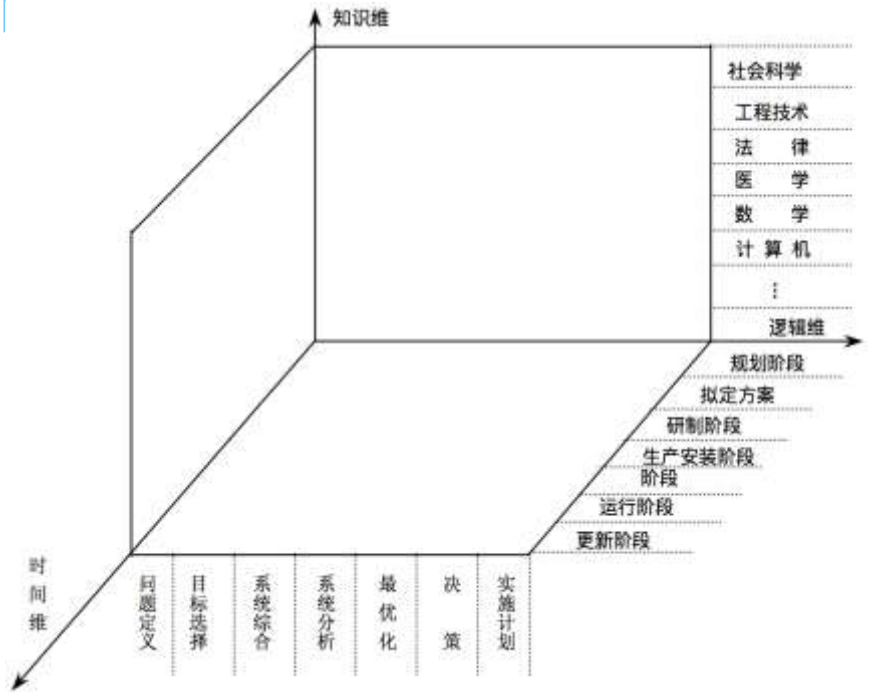
1.3 产品工程

- 产品工程（product engineering）的目标是将客户的愿望转化为可工作的产品，是创新、设计、开发、测试和部署软件产品的过程。
- 为了达到此目标，产品工程必须导出体系结构和基础设施。
- 体系结构包括4种可区分的系统构件：软件、硬件、数据和人。
- 产品工程的各个阶段：
  - Product Ideation（产品构思）
  - Product Architecture（产品架构）
  - Product Design（产品设计）
  - Product Testing（产品测试）
  - Product Migration and Porting（产品迁移和移植）
  - Technical Support（技术支持）
  - Sustaining Engineering（支持工程）
  - Professional Services（专业服务）



图：产品工程层次结构

A. D. Hall提出的系统工程方法的三维模型



系统工程的三维模型

描述了复杂系统建设方法的一个总体结构框架，其中：

- 时间维表示系统建设在时间上的阶段划分
- 逻辑维表示处理问题的逻辑步骤
- 知识维表示系统建设所 涉及的知识领域

软件开发

2 系统分析

主要内容

① 系统分析
② 可行性研究
③ 成本-效益分析
④ 技术分析
⑤ 系统结构的模块化
⑥ 系统定义

2.1 系统分析的目标

① 识别用户要求；
② 评价系统的可行性；
③ 进行经济分析和技术分析；
④ 把功能分配给硬件、软件、人、数据库和其它系统元素；
⑤ 建立成本和进度限制；
⑥ 生成系统规格说明，形成所有后续工程的基础。

系统分析所要回答的问题：

所期望的功能和性能是什么？

可靠性和质量问题是哪些？

总的系统目标是什么？

成本与进度限制如何？

制造的需求是什么？

市场与竞争情况怎样？

有效的技术有哪些？

将来可能有哪些扩充？



2.2 可行性研究

- 对于所有新的系统，需求都应该从可行性研究开始。
- 与可行性密切相关的因素：资源、时间、风险



- 可行性研究焦点问题：
  1. 系统是否符合机构的总体目标？
  2. 系统是否可能在现有技术条件、预算和时间限制内完成？
  3. 系统能否与已经存在的其它系统集成？
  4. 组织是否能够承受开发所面临的风险？

可行性研究报告的主要内容

1. 项目背景：问题描述；实现环境；限制条件
2. 管理概要：可行性研究方法；评价尺度；说明；影响
3. 对现有系统的分析：同类系统分析；待改造系统的分析
4. 候选方案：候选系统的配置；选择最终方案的准则
5. 系统描述：简略的范围描述；分配元素的可行性
6. 经济可行性（成本-效益分析）：经费概算；预期的经济效益
7. 技术可行性（技术风险评价）：技术实力；已有工作基础；设备条件
8. 法律可行性：系统开发可能导致的侵权、违法和责任
9. 用户使用可行性：用户单位的行政管理、工作制度；使用人员的素质
10. 其它与项目有关的问题：其它方案介绍；未来可能的变化

2.3 成本-效益分析

- 成本是管理者最关心的问题，它是项目成败的关键。
- 追求效益是项目的目标。
- 效益分类：

经济效益 vs 社会效益

有形的 vs 无行的

短期的 vs 长期的



价格构成

- 销售价格
  - 生产成本
  - 质量成本
    - 失败成本
      - 内部失败成本：报废、返修
      - 外部失败成本：退货、索赔、企业声誉败坏
    - 鉴定成本：检验、试验、审核
    - 预防成本：员工培训、质量检测、设备质量管理和质量保证开销
  - 利润

价格/质量成本的构成



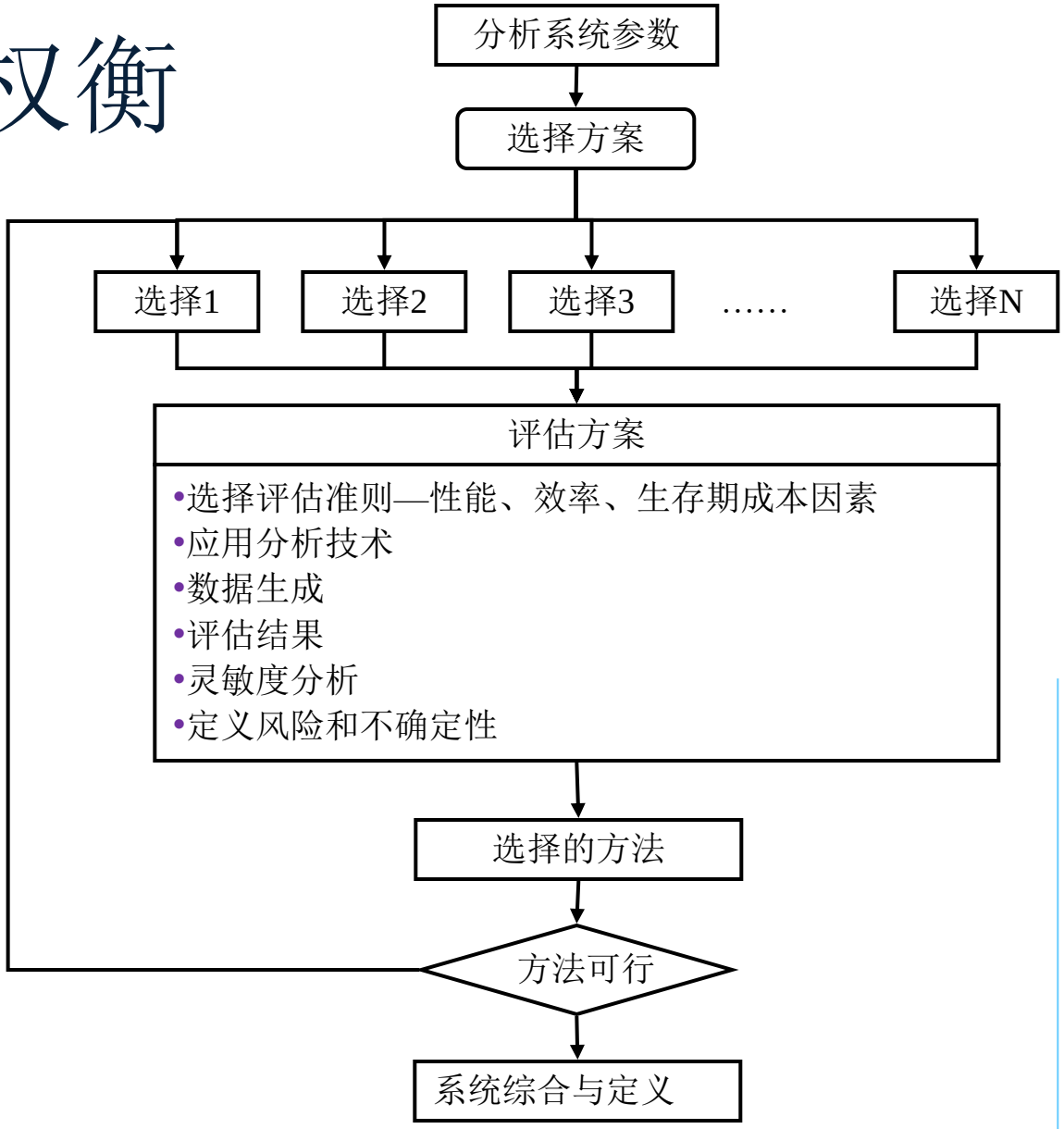
2.4 技术分析

- 技术分析主要解决的问题有：
  - 为完成系统的功能和性能需要什么技术？
  - 需要哪些新材料、方法、算法或者过程？
  - 有什么开发风险？
  - 这些技术问题对成本的影响如何？
- 用于技术分析的方法：
  - 数学模型和优化技术
  - 概率和统计
  - 排队论
  - 控制论



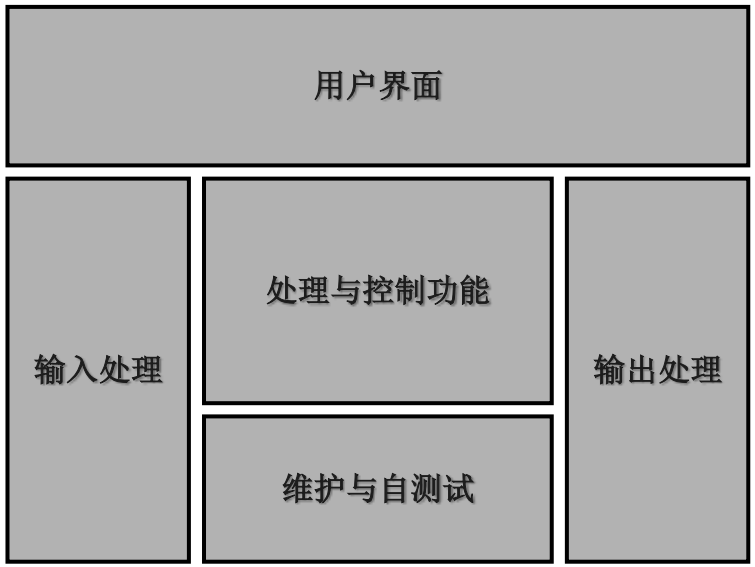
2.5 分配与权衡

- 一旦解决了与分析任务相关的问题，就应开始考虑候选的解决方案。
- 每一个系统功能，与其必要的一些性能和接口特性一起，分配给一个或多个系统元素。



2.6 系统结构的模块化

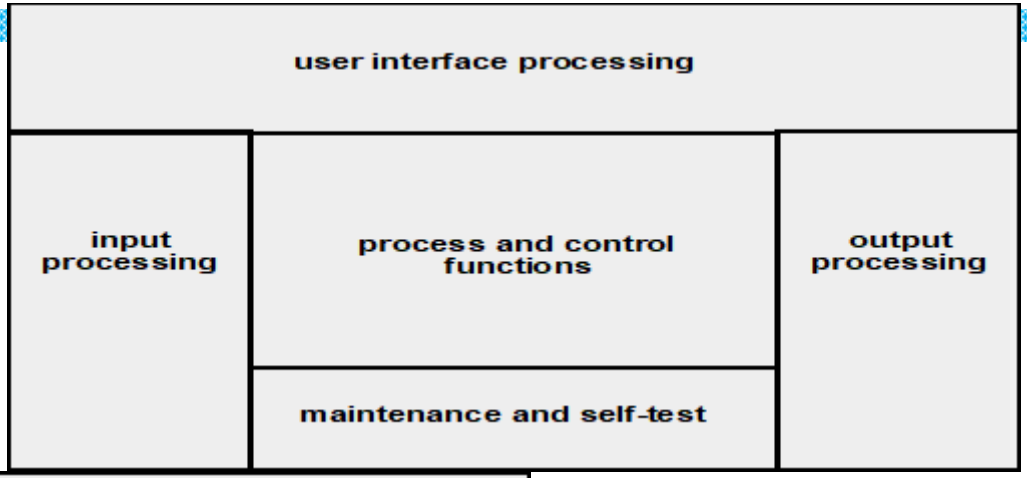
- 一旦把基于计算机系统的功能分配到各个元素，就能够据此建立一个模型，用以表达系统元素之间的相互关系，并为需求分析和设计奠定基础。
- 任意一个基于计算机的系统都能够模型化为使用输入-处理-输出（IPO）结构的信息变换。
- 结构图（Architecture Diagrams）模板的5个处理区域：
  - 用户界面
  - 输入
  - 系统功能与控制
  - 输出
  - 维护与自测试

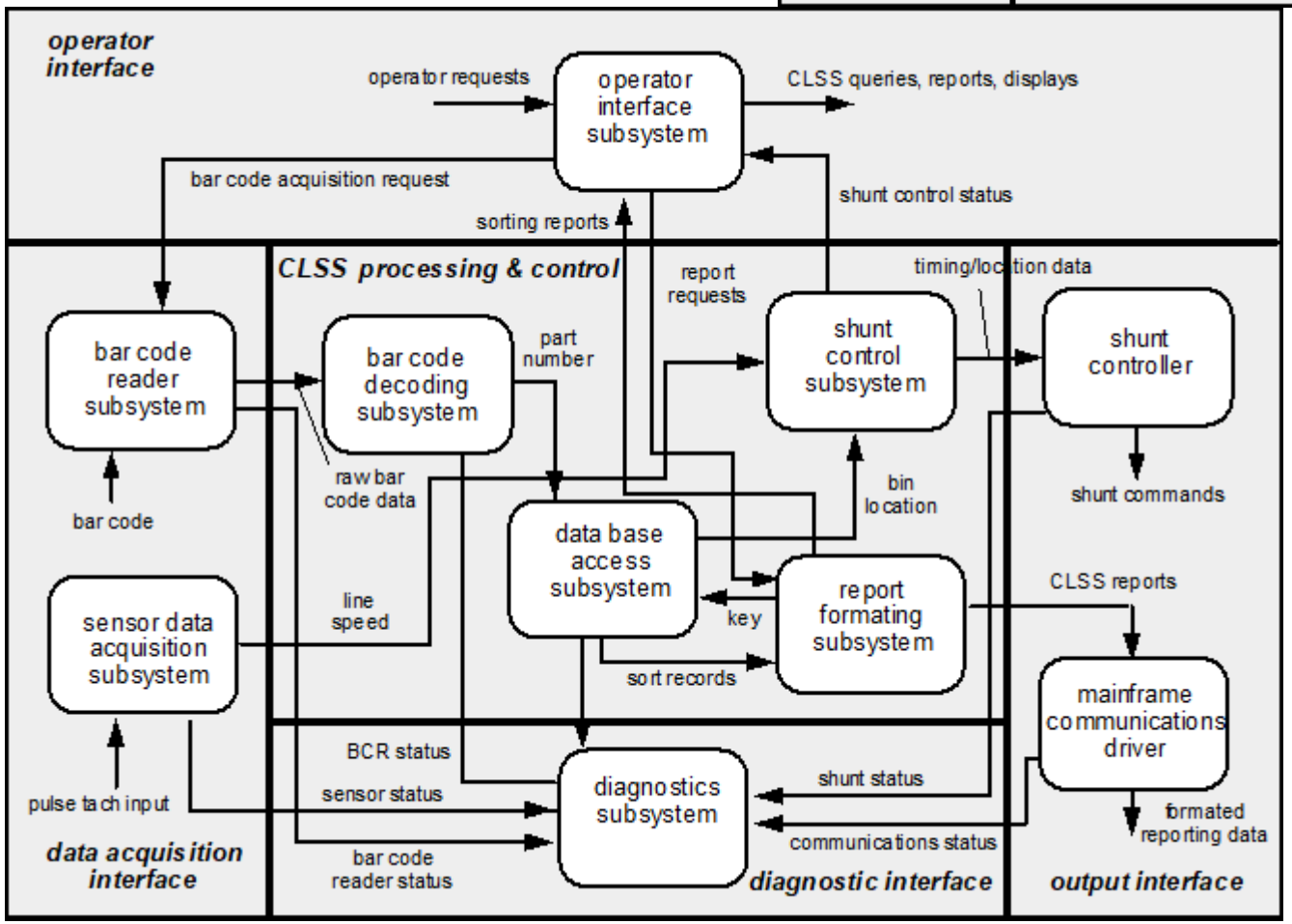


结构模板（Hatley-Pirbhai Modeling）

产品体系结构模板

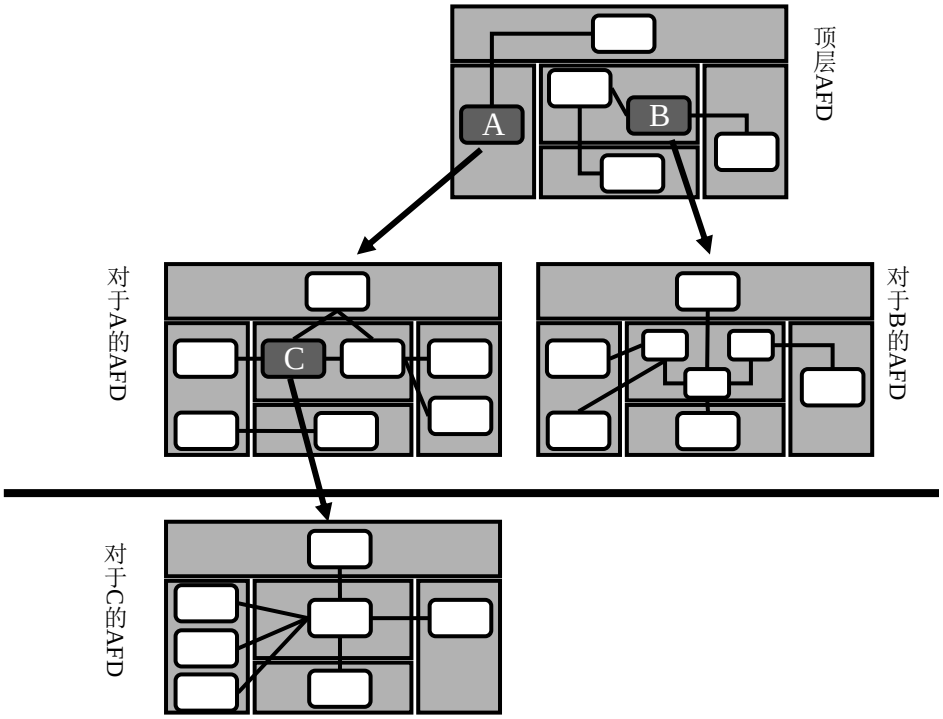
ACD(Architecture Context Diagram)



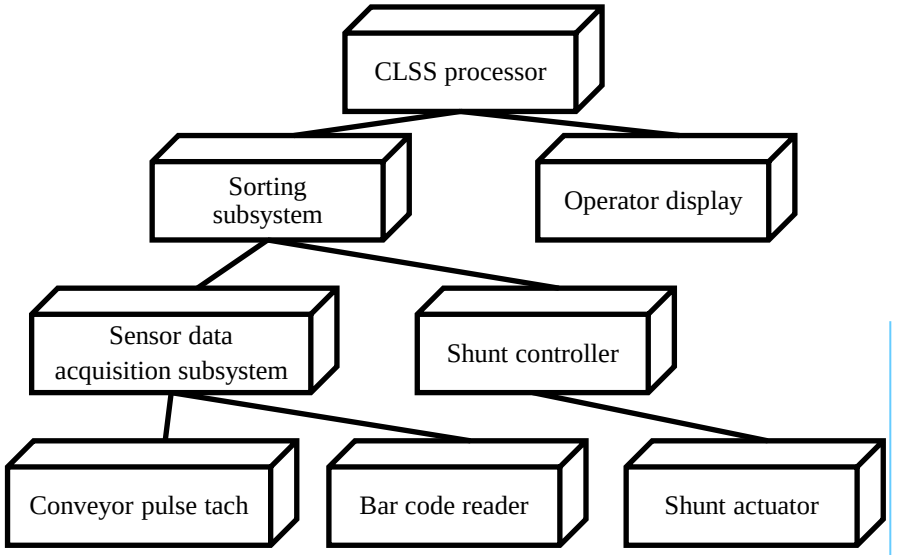


模型图

- ACD(Architecture Context Diagram)
- AFD (Architecture Flow Diagram)
- AID(Architecture Interconnect Diagram)
- ADS(Architecture Diagram Specification)



AFD的层次结构图



CLSS的UML布置图

2.7 系统定义

- 系统规格说明书作为硬件工程、软件工程、数据库工程、人机工程的基础而使用的一个文档。
- 系统规格说明书描述了系统的功能和性能，以及管理该系统开发的一些限制条件。
- 系统规格说明书界定每个被分配的系统元素，给软件工程师指明了软件在整个系统和用结构图描述的各种子系统环境中的作用。它还描述系统的输入/输出信息。

系统规格说明书的主要内容：

1. 引言
   - 文档的范围和目的
   - 概述：目标；限制条件
2. 功能和数据描述
   - 系统结构：ACD图；ACD描述
3. 子系统描述
   - 对于子系统n的ADS；AFD；系统模块描述；性能问题；设计限制条件；系统部件的分配
   - 结构词典
   - AID及其描述
4. 系统模型化和模拟结果
   - 用于模拟的系统模型
   - 模拟结果
   - 特殊的性能问题
5. 项目问题
   - 项目开发成本
   - 项目进度安排
6. 附录

软件开发

3 需求工程

主要内容

① 需求工程过程
② 需求采集
③ 需求分析方法
④ 需求评审
⑤ 需求管理

老婆给码公老公打电话："下班顺路买一斤包子，如果看到卖西瓜的，买一个。"
当晚，码公老公手捧一个包子进了家门....
老婆怒道："你怎么就买了一个包子？！"
老公答曰："因为看到卖西瓜的。"

需求案例：F-22飞机软件故障



- 现象：
  - 2004年12月21日，一架F-22战斗机在内利斯空军基地进行战术训练的过程中，飞机坠毁。
  - 2007年2月11日，12架F-22战斗机从夏威夷飞往日本，但在穿越国际日期变更线的时候，飞机失去了所有的导航和通信资料，被迫目视返航。
- 原因：分别是飞行控制软件故障、通信导航软件没有考虑日期变更线，导致软件瘫痪。

需求案例：血淋淋的 BUG



两起空难

- 2018年10月29日，一架波音737 Max 8客机撞向印度尼西亚的爪哇海，造成189名乘客和机组人员死亡。调查人员称该飞机的飞行控制软件出现“故障”。
- 2019年3月10日，埃塞俄比亚航空公司一架波音737 MAX 8客机在飞往肯尼亚首都内罗毕途中坠毁。飞机上载有157乘客和机组人员。怀疑埃航这架飞机发生了同样的“软件故障”。

迟迟修正不了的 BUG

MCAS，自动纠正失速系统，是波音737 MAX的一种操纵辅助系统。它有几个特点：

① 发现失速后，程序只相信主传感器，不与备份传感器核实。（同样的情况空客的飞机则会交给飞行员处理。）

② 一旦相信，不通知飞行员，直接操纵机翼。

③ 飞行员手动操作后，仍旧会每五秒自动执行，让飞行员不得不与飞机较劲。

④ 程序开关非常隐蔽。


THE WALL STREET JOURNAL.

Home World U.S. Politics Economy Business Tech Markets Opinion Life & Arts Real Estate WSJ. Magazine

BUSINESS

Boeing to Make Key Change in 737 MAX Cockpit Software

MAX software change marks shift for Boeing design of anti-stall system

从技术角度看，这是个很容易修正的错误。延误的原因，其中之一是联邦官员和波音公司安全专家对于需要进行多大改动，有意见分歧：

- 波音公司认为，更新这个BUG很简单，减少下压幅度就行。
- 联邦官员认为，189条人命是大事，必须大改，确保单个传感器错误信号不会触发MCAS系统。
- 加上波音公司和美国政府的低效，这个性命攸关的 BUG，硬生生被拖到四个月后。

- 波音 737 MAX 的交付
  - 波音要求每个需求都能追溯到相应的代码行。反之亦然。也就是说每行代码发生变化，必须能追溯到软件的设计文档。
- 软件外包和成本压缩
  - 裁掉了90%熟练员工，用“外包”来代替他们。
  - 2015年，波音公司给印度的外包业务翻了一番，达到了5亿美元。
- 使用Ada语言达到99%
  - 属于80年代的编程语言。
  - 缺少合适数量精通Ada的程序员。

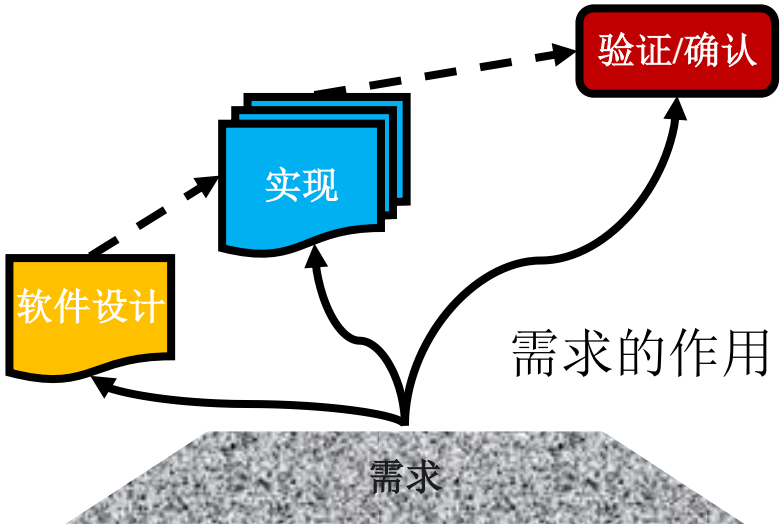
客户需求分析的重要性



根据美国工业协会统计，产品失败的原因核心有6个，其中不适当的市场分析排名第一，独占产品失败原因的32%，客户需求分析又是市场分析中非常重要的内容，可以这样说，能否有效洞察客户需求是产品成败的关键要素。

3.1 什么是需求工程

- 需求工程是一个涵盖创建和维护系统需求文档所必需的一切活动的过程。
- 需求工程：
  - 需求开发：指需求的获取、分析、规格说明和验证。典型的需求开发结果是需求规格说明书及相关的分析模型。
  - 需求管理：在软件开发过程中对需求开发结果的控制、跟踪和管理。



需求工程的任务

- **引导：** 决定客户需要什么
- **分析和协商：** 理解各种客户需求之间的关系，整理这些关系成为一个高效的解决方案。
- **需求规格书：** 建立一个明确的需求模型。
- **系统模型：** 建立需求表示，这个表示能用于评估需求的正确性、完整性和一致性。
- **确认：** 检查模型。
- **管理：** 识别、控制和追踪需求和需求变更。

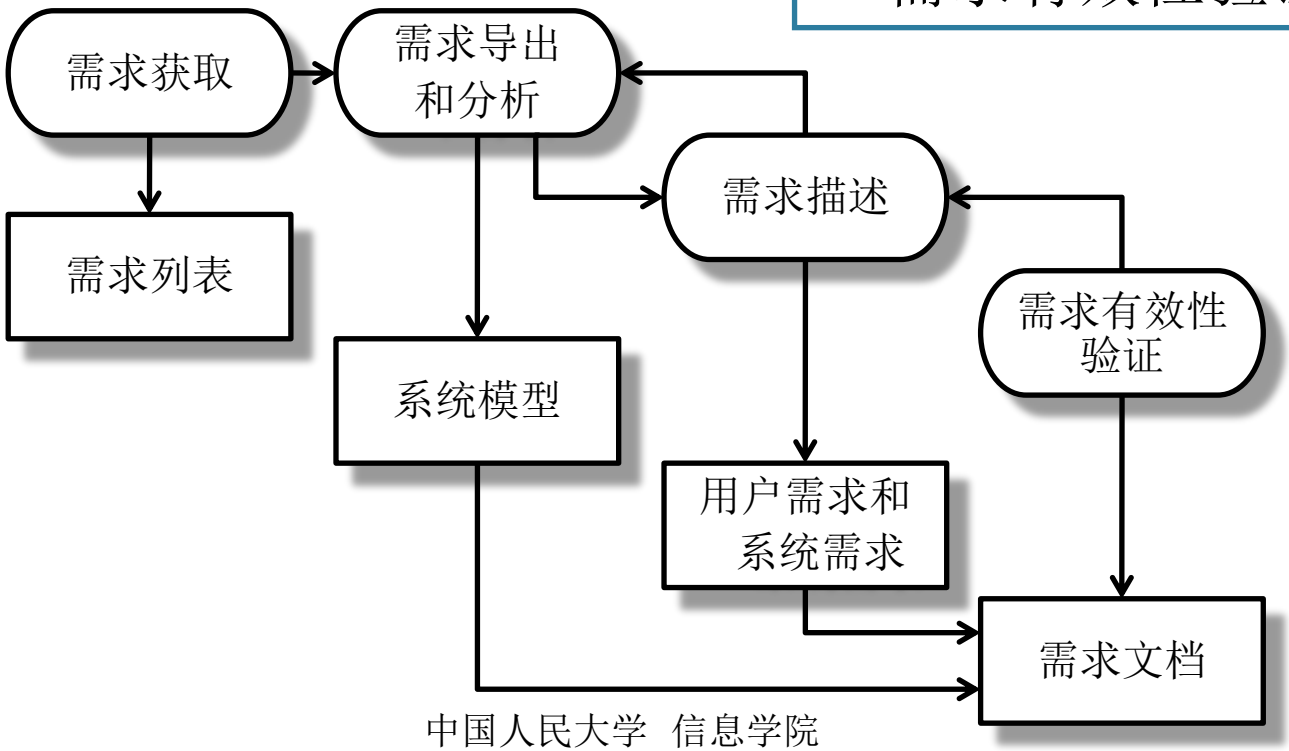
需求工程的主要活动

需求管理的主要活动包括：

- 需求基线的建立
- 需求变更控制
- 需求跟踪

需求开发的主要活动：

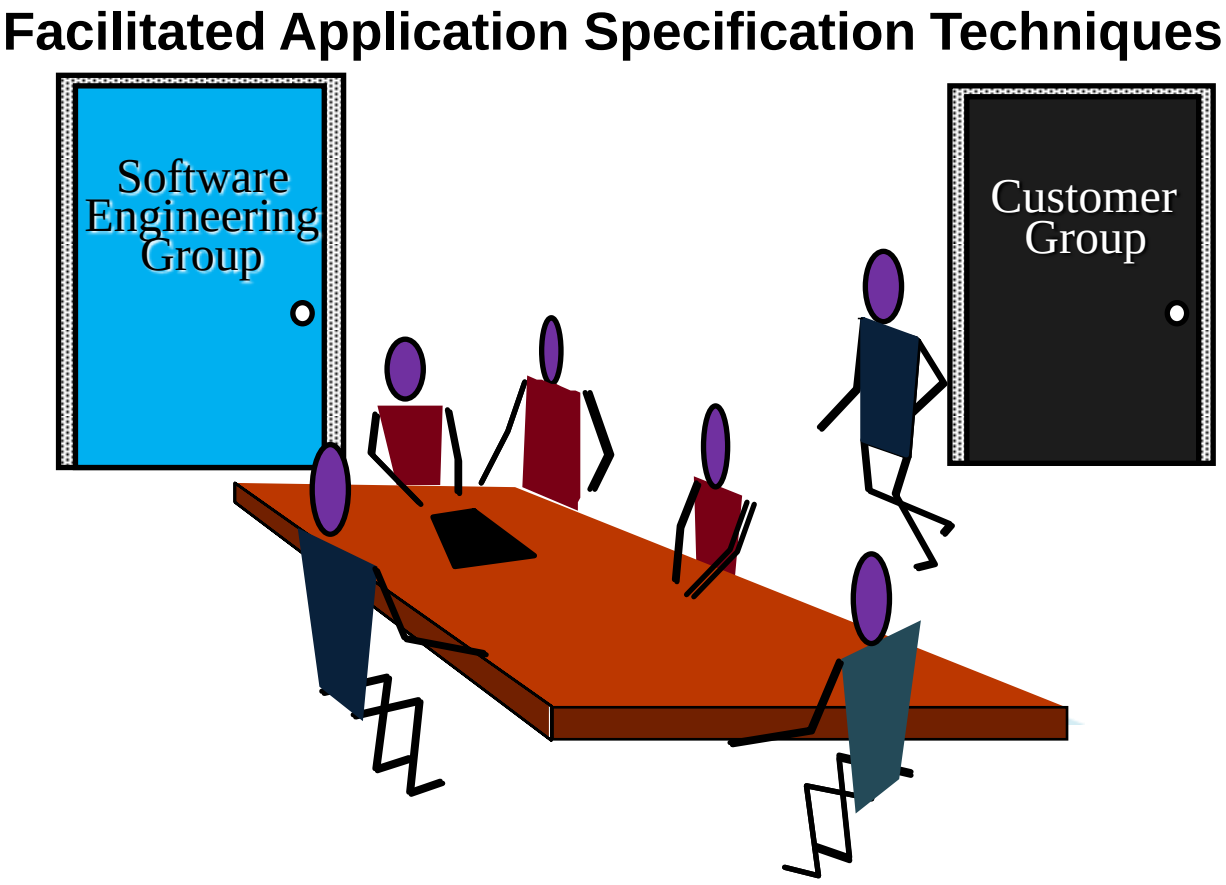
- 需求获取
- 需求导出和分析
- 需求描述和文档编写
- 需求有效性验证



3.2 需求采集

系统分析员、又称系统工程师，他的主要责任是：

- 引导用户，启发用户，挖掘需求。
- 将需求以用户和开发人员都易于理解的方式表达出来。



FAST: basic guidelines

- **A meeting** is conducted at a neutral site and attended by both software engineering analysts and customers.
- **Rules** for preparation and participation are established.
- **An agenda** is suggested that is formal enough to cover all important point but informal enough to encourage the free flow of ideas.
- **A facilitator** (can be a customer, a developer or an outsider) is assigned to control the meeting.
- **A definition** mechanism (can be work sheets flip charts or wall stickers or chat room) is used where how meeting will be conducted.
- **The goal** of this meeting is to identify the problem, propose elements of the solution, negotiate different approaches and specify a preliminary set of solution requirement in an atmosphere for the achievement of the goal.

FAST: the steps activities

- In the beginning a formal meeting is conducted between developer and customer, Q&A session is started to have the general idea about the problem, desired solution, customer and benefits.
- Then, customer and developer both write a separate one or two page. Product request document. This is just like a small report with brief description.
- Next, step is to set a meeting place, time and date for FAST. Parallel a facilitator and attendees of the FAST (both the development and customer user organization sides) are selected.
- After that the product request reports are distributed to all the attendees before the meeting starts.
- Additionally before the meeting each attendee is asked to make a list of three objects or instances:
  - Objects that will be used by the system to perform its functions (Objects of input).
  - Objects that will be produced by the system (objects of output) and,
  - Objects that will be part of the environment that surrounds the system.

Guidelines & Principles

FAST Guidelines

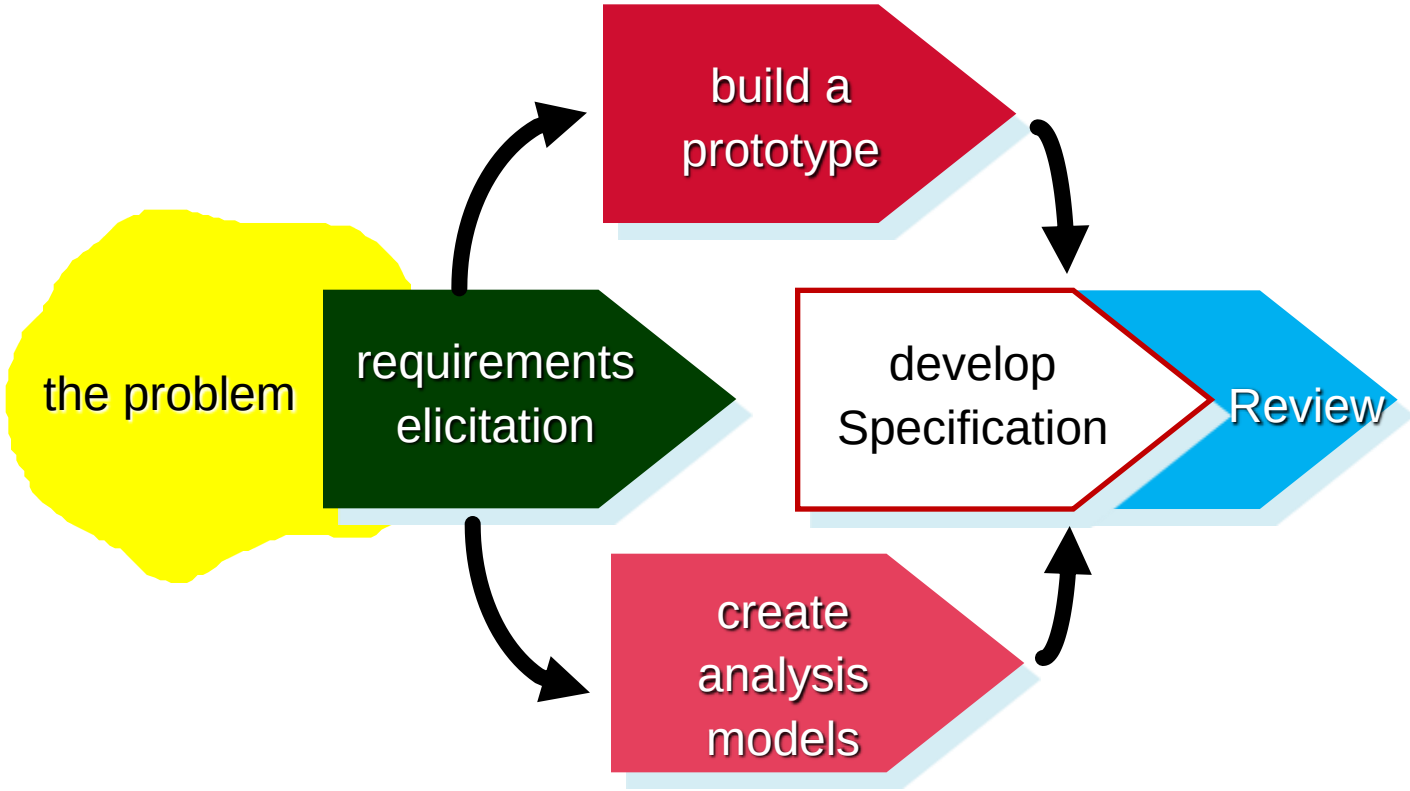
- participants must attend entire meeting
- all participants are equal
- preparation is as important as meeting
- all pre-meeting documents are to be viewed as “proposed”
- off-site meeting location is preferred
- set an agenda and maintain it
- don’t get mired in technical detail

J. Wood & D. Silver

Davis’ Principles

- Understand the problem before you begin to create the analysis model.
- Develop prototypes that enable a user to understand how human-machine interaction will occur.
- Record the origin of and the reason for every requirement.
- Use multiple views of requirements.
- Prioritize requirements.
- Work to eliminate ambiguity.

需求分析过程


build a prototype
the problem
requirements elicitation
develop Specification
Review
create analysis models

开发原型（Prototype）

- 在开发的早期，用户和开发者往往对系统的认识模糊，很难完全准确地表达系统的全面要求。
- 在软件开发过程中，原型是软件的一个早期可运行的版本，它反映最终系统的部分重要特性。

- 原型主要有3种不同的作用类型：

① 探索型
- 目的是要弄清对目标系统的要求，确定所希望的特性，并探讨多种方案的可行性。
- 它主要针对开发目标模糊，用户和开发者对项目都缺乏经验的情况。

② 实验型
- 用于大规模开发和实现之前，考核方案是否合适，规格说明是否可靠。

③ 进化型
- 目的不在于改进规格说明，而是将系统建造得易于变化，在改进原型的过程中，逐步将原型进化成最终系统。

- 运用原型的2种不同策略：

① 废弃策略
- 先构造一个功能简单而且质量要求不高的模型系统。
- 系统构造完成后，原来的模型系统被废弃。
- 探索型和实验原型属于这种策略。

② 追加策略
- 先构造一个功能简单而且质量要求不高的模型系统，作为最终系统的核心。
- 它对应于进化原型。

原型开发法的特点分析

- 原型开发的目的是确定系统是否满足用户需要，或者说要确定系统做什么。
- 软件原型化开发方法通常有3个基本步骤：
  ① 获得基本需求；
  ② 开发工作模型；
  ③ 模型验证
- 原型开发技术的特点：
  ① 加快需求的确定；
  ② 简化项目管理；
  ③ 加强用户参与与决策。

原型法优点：

1）增进用户与开发人员之间的沟通

2）用户在系统开发过程中起主导作用

3）辨认动态的用户需求

4）启迪衍生式的用户需求

5）缩短开发周期，降低开发风险

原型法缺点：

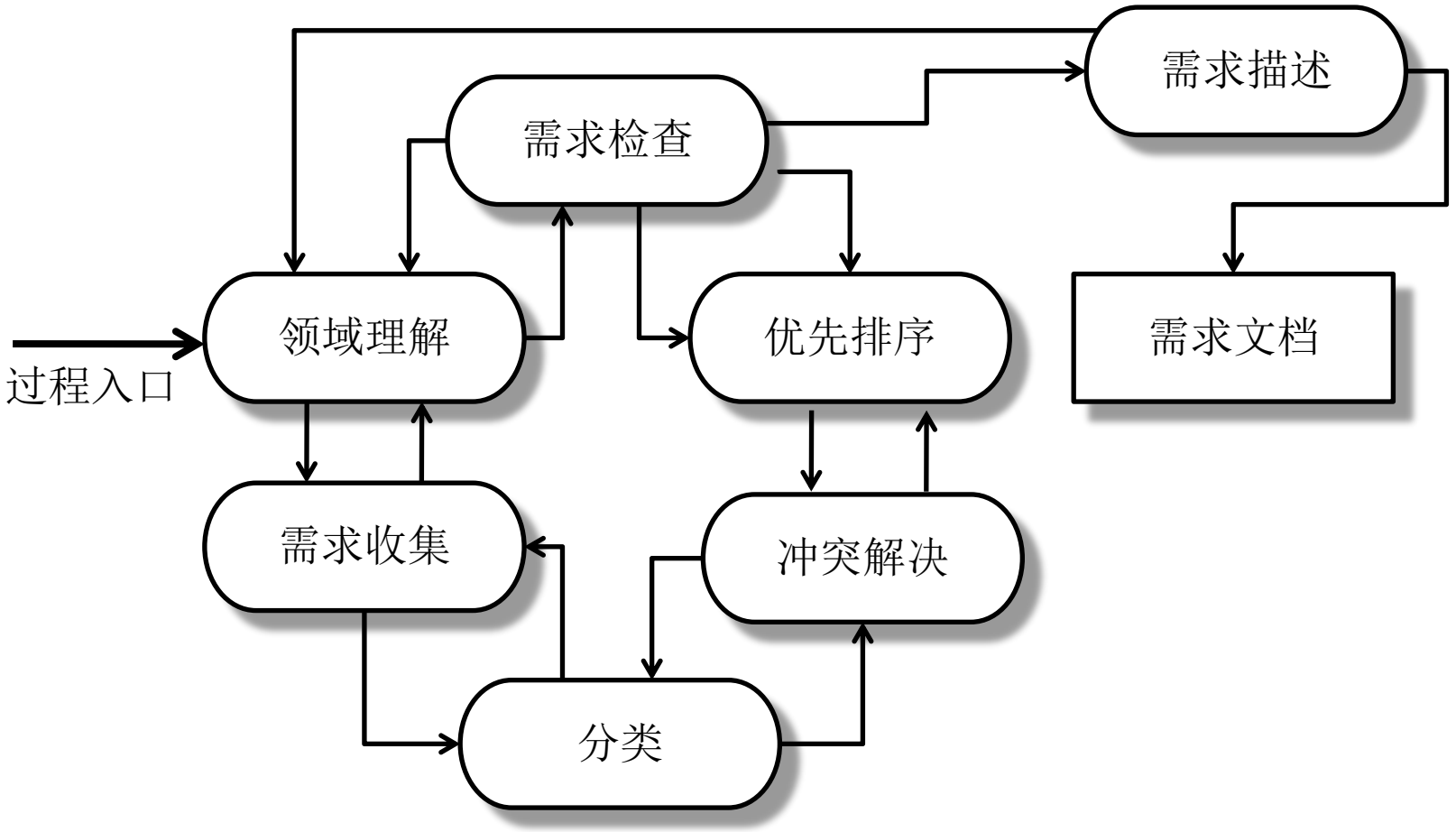
1）原型开发时难以管理，无法备全全部资料，导致存在质量隐患。

2）难以确定原型，原型目标偏离太多，则会导致系统失效。

3）比较费劲的是要把所有子系统相互融合的形式放到整个信息系统中。

4）由于多次反复，导致用户失去开始时的耐心，在不耐烦的情况下选择了不合格的原型当做最终系统。

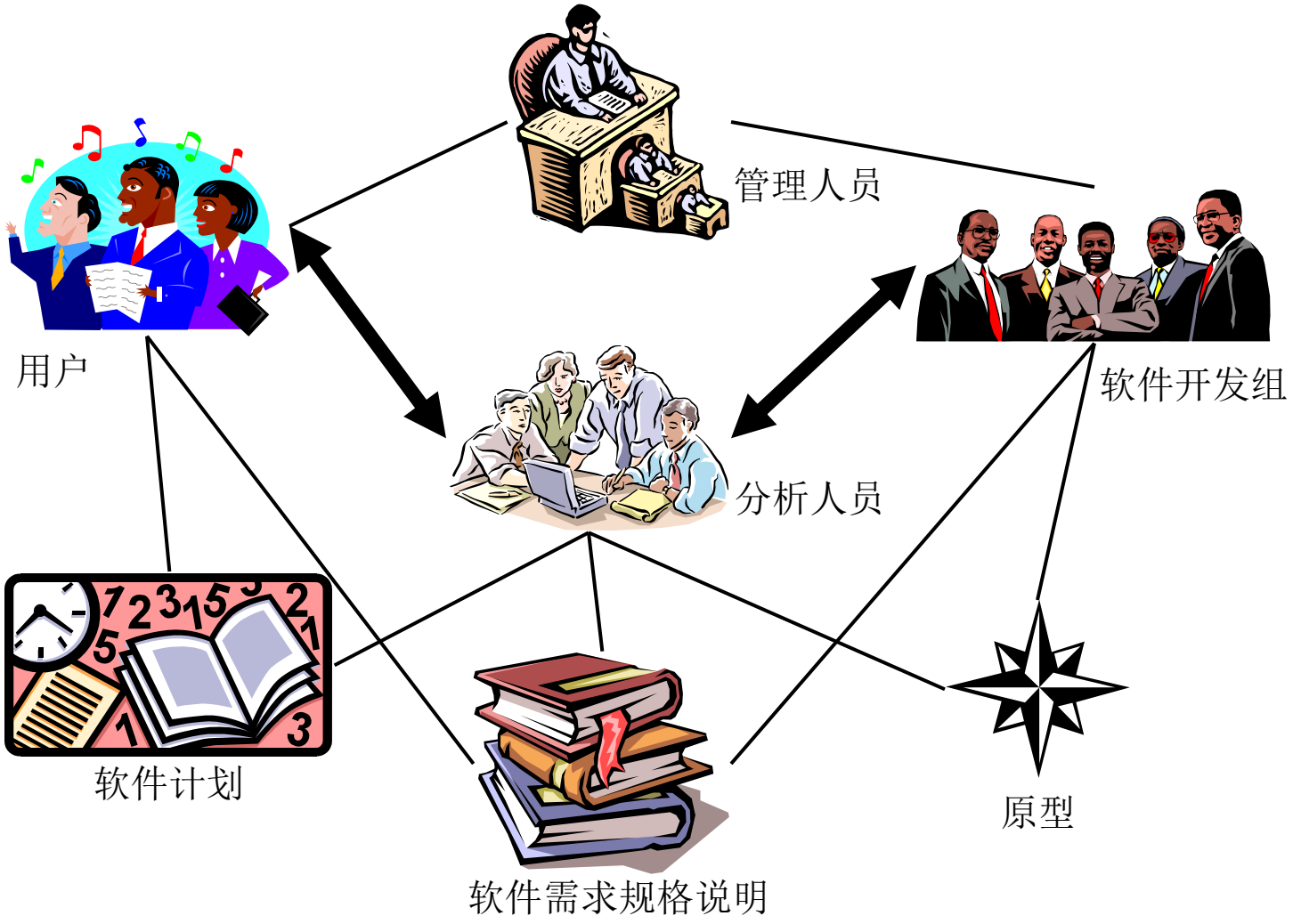
需求导出和分析过程



问题识别

- 系统分析人员要研究可行性报告和软件项目实施计划。
- 从系统角度来理解软件并评审用于产生计划估算的软件范围是否恰当。
- 确定对目标系统的综合需求，并提出这些需求实现条件，以及需求应达到的标准。
- 问题识别的另一项工作是建立分析所需要的通信途径，以保证能顺利地对问题进行分析。
- 分析员必须与用户、软件开发机构的管理部门、软件开发组成员建立联系。
- 项目负责人在此过程中起协调人的作用。

软件需求分析的通信途径



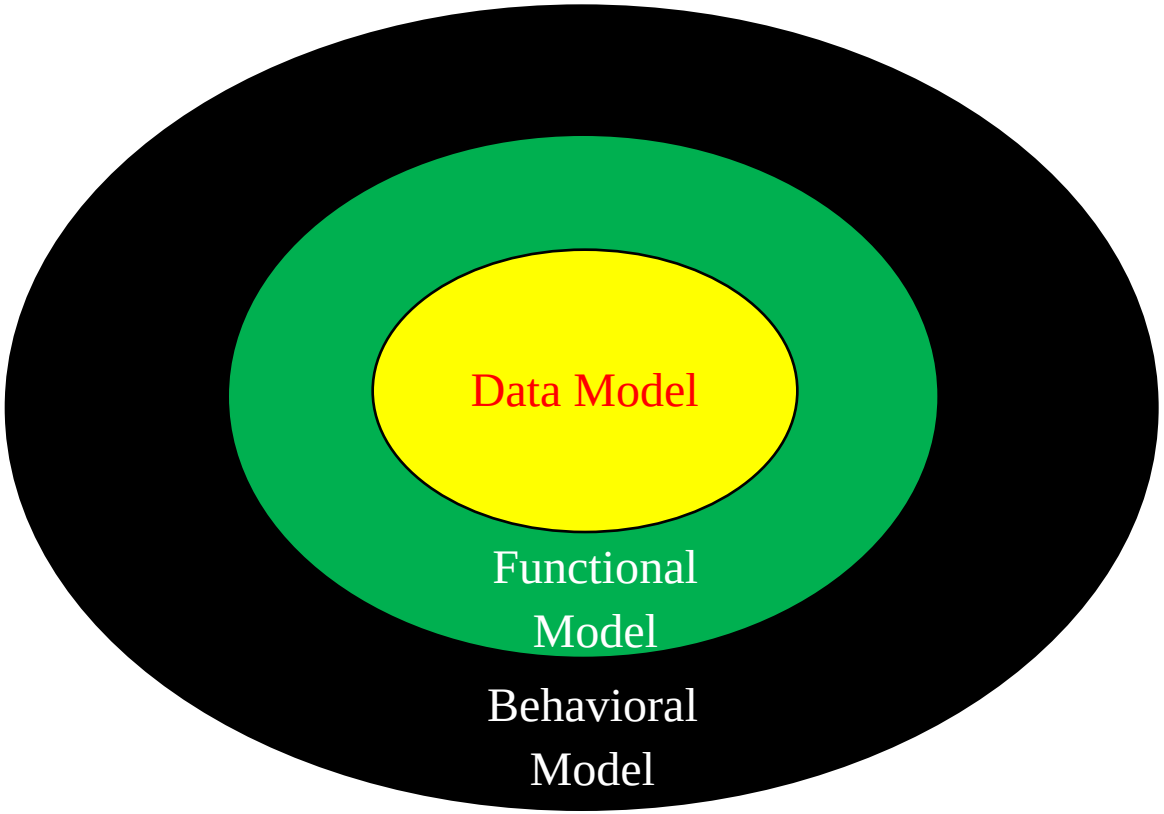
例：12306订票系统的需求项



- **终端用户方面的:**
  - 访问用户数量、总体注册用户数量、平时访问用户的峰值、平时访问用户的谷值、大假期访问用户的峰值、大假期访问用户 的谷值、小假期访问用户的峰值、小假期访问用户的谷值; 用户的地域分布性、用户可能介入的设备、用户接入的网络状况统计分析；
- **安全方面：**
  - 内部安全：主要关注资金以及交易的安全，特别是防止内部人员尤其是系统管理员；
  - 外部安全：主要关注如何确保拒绝外部恶意入侵与攻击成为一个核心，特别是类似DDOS之类的攻击；
- **后台服务方面的:**
  - 关键购票流程业务分析: 购票的基本流程分析、一次购票的TPS数量分析、一次购票的用户流量分析、一次购票的用户静态流量分析、一次购票的用户动态流量分析；
  - 这其中又分为初次购票与再次购票两种情况，流程稍微有些不同; 系统提供的其他服务统计分析; 节假日中长假只有春节与国庆,小假有元旦、清明、五一、端午、中秋等；

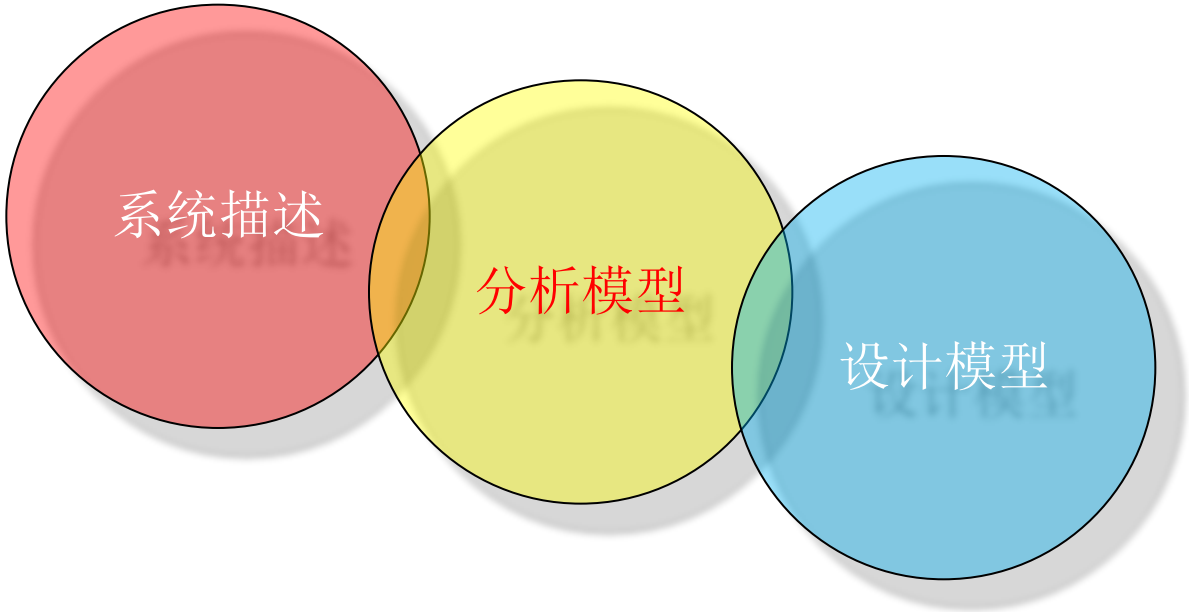


分析模型



分析模型是一座桥梁

- 系统模型最重要的特点是抛弃细节。
- 系统模型是系统的抽象而不是系统的替代表示。
- 不同的系统模型基于的是不同的抽象方法。



系统建模

① **假设**，能减少可能的排列和变化数量，这样使得模型以合理的方式反映问题。

② **简化**，使得能够以适当的方式创建模型。

③ **限制**，有助于定义边界。

④ **约束**，将指导用什么方式来建立模型，以及用什么方法来实现模型。

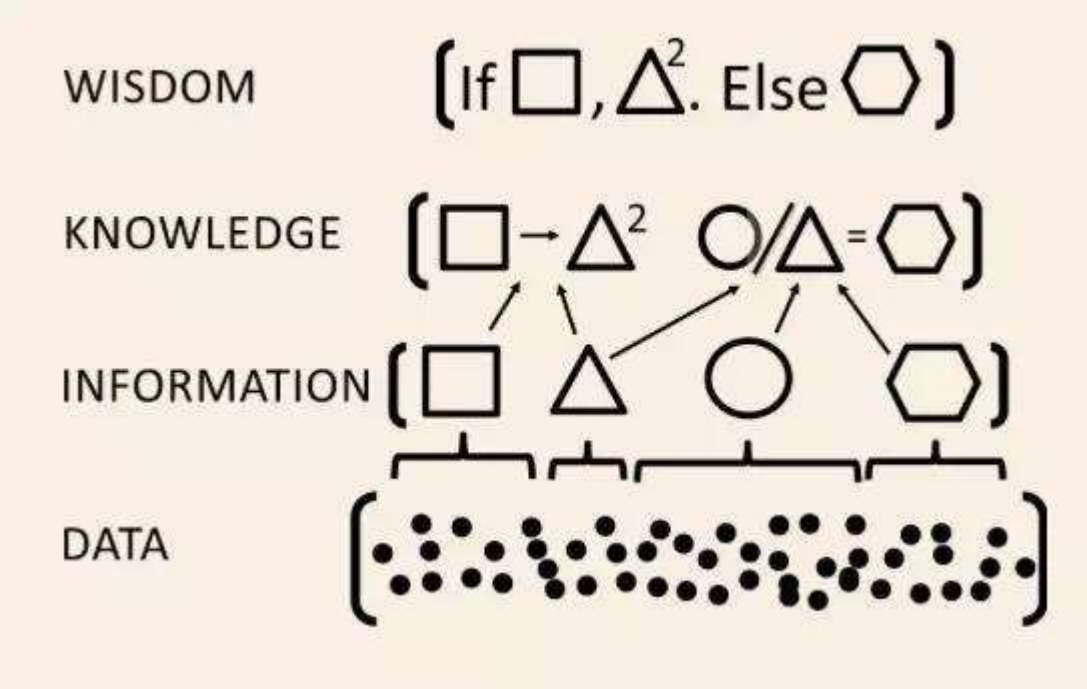
⑤ **偏好**，意味着对数据、功能和技术选择喜好的体系。

打车软件
假设：智能手机为移动终端
简化：智能手机叫车，司机使用智能手机接单
限制：只限于叫车和接单，现金支付
约束：OO方法，Java编程
偏好：C/S结构，MVC，IBM Bluemix/DevOps



什么是模型？

模型是对真实世界的抽象，明确定义了各种元素、以及元素之间的关系，可以用来做逻辑推导。



认知金字塔

- 金字塔的最底层是数据。数据代表各种事件和现象。数据本身没有组织和结构，也没有意义。数据只能告诉你发生了什么，并不能让你理解为什么会发生。
- 数据的上一层是信息。信息是结构化的数据。信息是很有用的，可以用来做分析和解读。
- 信息再往上一层是知识。知识能把信息组织起来，告诉我们事件之间的逻辑联系。
- 认知金字塔的最上一层，是智慧。智慧是识别和选择相关知识的能力。

软件需求分析的原则

① 必须能够表达和理解问题的数据域和功能域。

② 必须按自顶向下、逐层分解的方式对问题进行分解和不断细化。

③ 要给出系统的逻辑视图和物理视图。

注意：

- 必须记住分析阶段的任务是确定必须做什么，而不是确定应该怎样做。
- 因此，如果进一步分解将使你开始思考为了完成一个功能可能需要写出的代码，那么，就不应该再分解了。
- 在分析阶段分析员应该只在功能层上工作，代码已经属于物理层了。

软件需求分析方法

- 软件需求分析方法由对软件的数据域和功能域的系统分析过程及表示方法组成。
- 软件需求分析方法定义了表示系统逻辑视图和物理视图的方式。
- 常见的需求分析方法：
  - 面向数据流的结构分析方法（SA）
  - 面向数据结构的Jackson方法（JSD）
  - 面向对象的分析方法（OOA）
  - 系统动态分析方法：
    - 状态迁移图
    - Petri网

软件需求分析方法共性：

① 支持数据域分析机制
② 功能表示的方法
③ 接口的定义
④ 问题分解的机制以及对抽象的支持
⑤ 逻辑视图和物理视图
⑥ 系统抽象

3.3 需求有效性验证

- 需求有效性验证是要检验需求能否反映客户的意愿。
- 在需求有效性验证过程中，要对需求文档中定义的需求执行多种类型的检查。

- 有效性检查的内容
  - 一致性检查
  - 完备性检查
  - 现实性检查
  - 可检验性检查

- 有效性验证技术
  - 需求评审
  - 原型建立
  - 测试-用例生成
  - 自动的一致性分析

需求评审

- 需求评审是一个手工过程，需要有客户方和承包方的人员共同参与，检查文档中的不规范之处和遗漏之处。
- 需求评审可以是非正式的也可以是正式的。
- 需求评审检查的内容
  1. 可检验性
  2. 可读性
  3. 可追朔性
  4. 可调节性

高质量需求的特征

- 完整性
- 正确性
- 可行性
- 必要性
- 划分等级
- 无二义性
- 可验证性

影响需求质量的因素

- 用户需求不断增加
- 模棱两可的需求
- 用户不配合
- 过于精简的需求说明
- 忽略了用户的分类
- 不准确的计划
- 不必要的特性

3.4 需求管理

需求管理

为什么要管理需求?

① 大型软件系统的需求总是在变化的
② 大型软件系统总是要不断地完善
③ 大型软件系统通常拥有不同的用户群
④ 业务和技术环境的变化肯定会反映到系统本身上来

什么是需求管理？

- 需求管理是一个对系统需求变更了解和控制的过程。
- 需求管理的过程与其它需求工程过程相互关联的。
- 初始需求导出的同时就启动了需求管理规划，一旦形成了需求文档的草稿版本，需求管理活动就开始了。
- 持久的需求
  - 这一类需求相对平稳，一般起源于一个机构的核心活动，与该系统的领域直接相关。
- 易变的需求
  - 这是一类很不稳定的需求，在整个开发过程中以及系统投入运行以后都会不断变更。

易变需求的分类

需求类型	描述
易变的需求	需求的变化源于机构所处环境的变化。
浮现的需求	在系统的开发过程中，客户对系统有了更深入的理解，于是有新的需求浮现出来。设计过程要反映这种需求。
引发的需求	需求源于计算机系统的引进，引进的计算机系统会改变机构中的业务过程，可能产生新的工作方式，这种工作方式产生了新的需求
兼容性需求	需求依赖于机构中特别的系统或业务过程。当这些在改变时，正在使用中的系统或交付的系统兼容性需求变发生了。

需求管理的活动

1. 需求识别
2. 变更管理过程
3. 可追溯策略
4. CASE工具

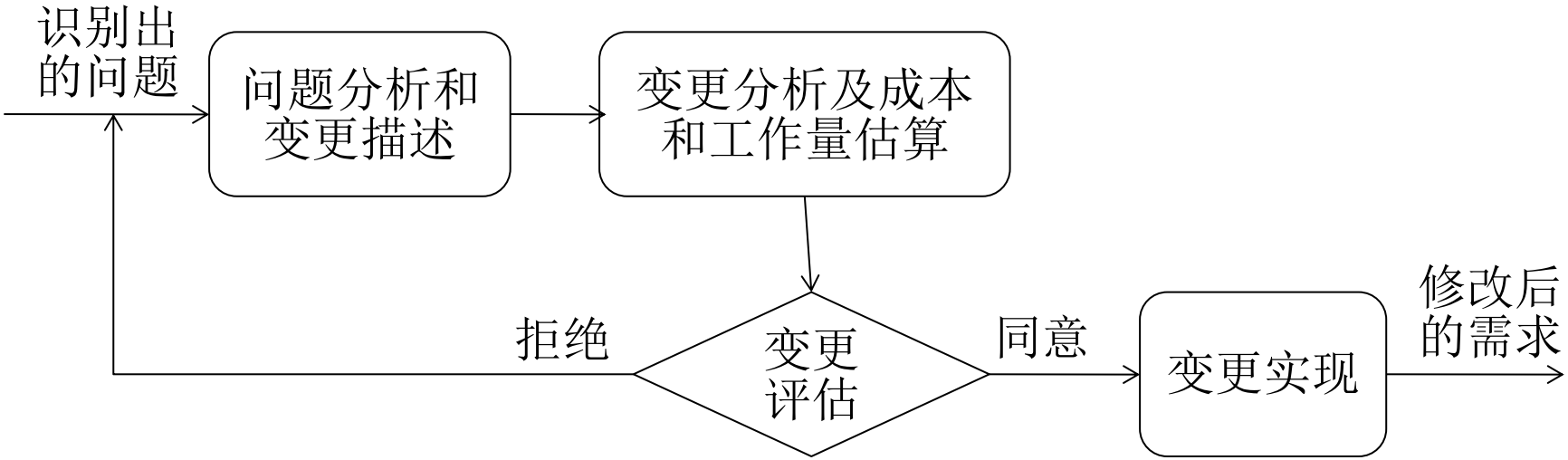


需求维护的3类可追溯性信息

1. 源可追溯性信息
   - 连接需求到提出需求的项目相关人员和这些需求的基本原则。
   - 以便变更发生时，与相关人员商讨变更事宜。
2. 依赖可追溯性信息
   - 连接需求文档中彼此依赖的需求。
   - 这个信息用来评估一个变更会对多少需求产生影响以及引发的需求变更的范围和程度。
3. 设计可追溯性信息
   - 连接需求到其实现的设计模块。
   - 这个信息用来评估需求变更对系统设计和实现带来的影响。

需求变更管理

变更管理应该处理全部的需求变更。



软件开发

4 软件需求分析

主要内容

① 需求分析的任务
② 需求内容
③ 需求文档
④ 举例：需求分析

4.1 软件需求分析的任务

1. 需求分析是借助于当前系统导出目标系统的逻辑模型，解决目标系统做什么。

2. 深入描述软件的功能和性能，确定软件设计的限制和软件同其它系统元素的接口细节，定义软件的其它有效性需求。

3. 软件需求规格说明还为评价软件质量提供依据。

需求分析的任务

4.2 需求的内容



- 功能需求
  - 系统应该提供的服务
  - 有时还要明确声明系统不应该做什么
- 非功能需求
  - 对系统提供的服务或功能的约束。包括时间约束、开发过程约束和标准等。
- 领域需求
  - 系统应用领域的需求
  - 可能是功能需求或非功能需求

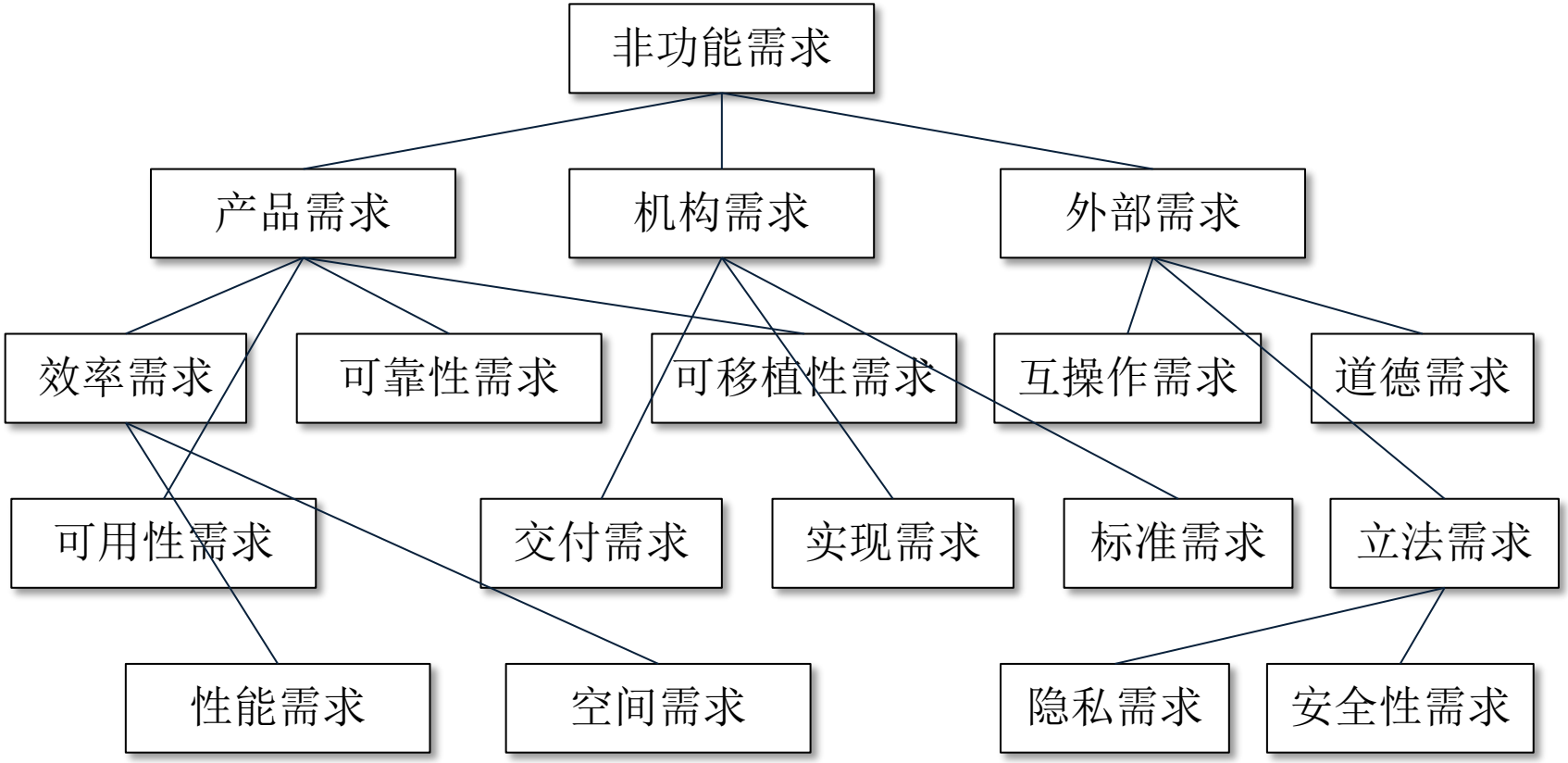


软件需求的3个不同层次

什么是非功能性需求

- 非功能性需求（Non-functional requirement）是指依据一些条件判断系统运作情形或其特性，而不是针对系统特定行为的需求。
- 一般而言，功能需求会定义系统的行为，而非功能性需求会定义系统的特性。
- 非功能性需求一般被称为系统的品质，或限制、品质属性、品质目标、品质属性、品质服务需求、非行为性的需求。
- 非功能性需求可以分为以下二类：
  - 运行品质（Execution qualities），在系统运行时观察到的品质，例如安全性及易用性等。
  - 发展品质（Evolution qualities），与软件系统结构及开发过程有关的品质，例如软件可测试性（testability）、可维护性、可扩展性、可伸缩性（scalability）等。

非功能需求的类型


非功能需求
产品需求
机构需求
外部需求
效率需求
可靠性需求
可移植性需求
互操作需求
道德需求
可用性需求
交付需求
实现需求
标准需求
立法需求
性能需求
空间需求
隐私需求
安全性需求

例：非功能性需求实例

产品需求

4.C.8 它应该能将所有JEE和用户之间的必需的通信用标准的Java字符表达式。

机构需求

9.3.2 系统开发过程和交付的文档将遵照YXZCo-SP-STAN-95中的相关定义。

外部需求

7.6.5 系统不应该对系统的操作人员公开客户除名字和索引代码之外的任何个人信息。

例：系统目标和可验证的需求

系统目标
系统应该很好用，即使对一个没有经验的用户，而且应该使得用户错误降到最少。
可检验的非功能需求
对一个没有经验的用户来说，经过两个小时的培训就应该能使用系统的所有功能。在这样的培训之后，一个有经验的用户每天的出错平均数不应超过两次。



例：指定非功能需求的度量

性质	度量方法
速度	每秒处理的事务 用户/事件响应时间 屏幕刷新时间
规模	K字节 RAM芯片数
易用性	培训时间 帮助画面数
可靠性	失败平均时间 无效的概率 失败发生率 有效性
健壮性	失败之后的重启次数 事件引起失败的百分比 失败中数据崩溃的可能性
可移植性	依赖于目标的语句百分比 目标系统数

4.3 需求文档

需求规格说明书

主要内容

① 概述
② 任务概述
③ 数据描述
④ 功能需求
⑤ 性能需求
⑥ 运行需求
⑦ 其它需求

需求描述所使用的符号

- 结构化自然语言
  - 该方法依赖于定义标准格式或模板来表达需求
- 设计描述语言
  - 该方法使用一种类似于程序设计语言，但是具有更多抽象特征，通过定义系统的操作模型来定义需求。
- 图形化符号
  - 图形语言辅之以文本注释来定义系统的功能需求。如：SADT、Use-case等。
- 数学描述
  - 基于像有限状态机或集合这样的数学概念的符号。
  - 无二义性描述减少了客户和承包商之间关于功能的争论。然而，绝大多数客户不懂形式化描述，因而不愿接受这样的系统合同。

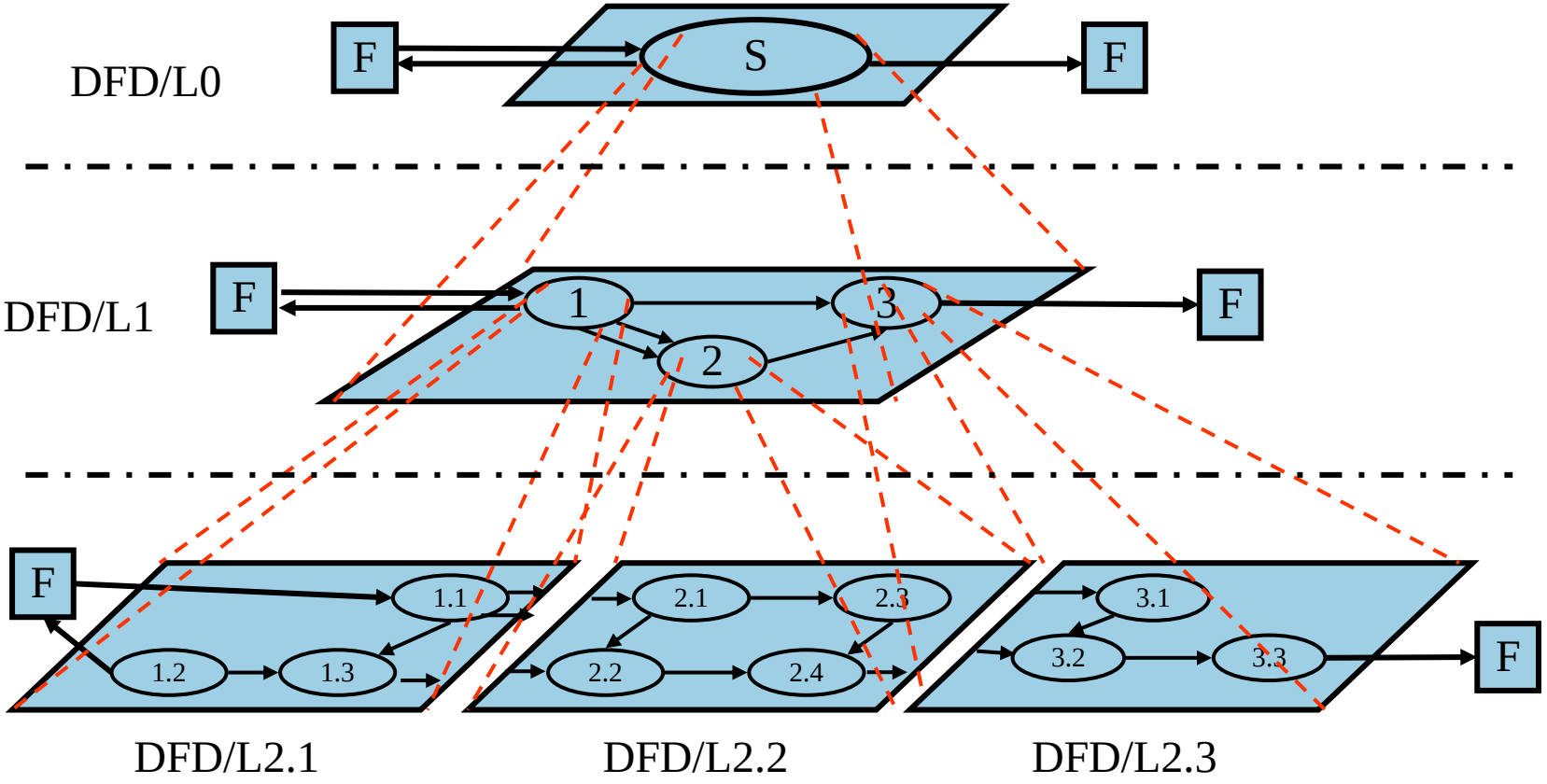
例：使用标准格式的系统需求描述

功能	
描述	
输入	
来源	
输出	
目的地	
要求	
前条件	
后条件	
副作用	

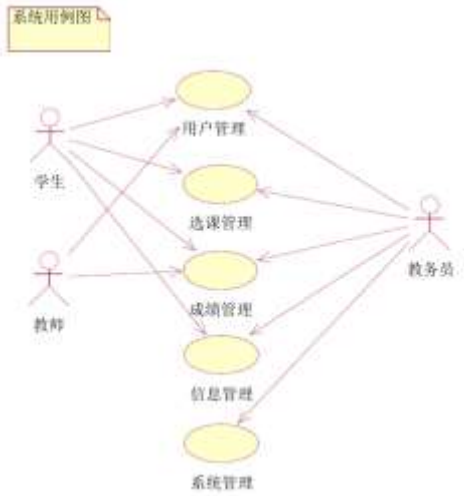
例：打印服务器接口的Java PDL描述

```
interface PrintServer {
// defines an abstract printer server
// requires: interface Printer, interface PrintDoc
// provides: intialize, print, displayPrintQueue, cancel PrintJob, switchPrint
    void intialize ( Printer P );
    void print ( Printer p, PrintDoc d );
    void displayPrintQuene (Printer p );
    void cancelPrintJob (Printer p, PrintDoc d );
    void switchPrinter ( Printer p1, Printer p2, PrintDoc d );
} //PrintServer
```

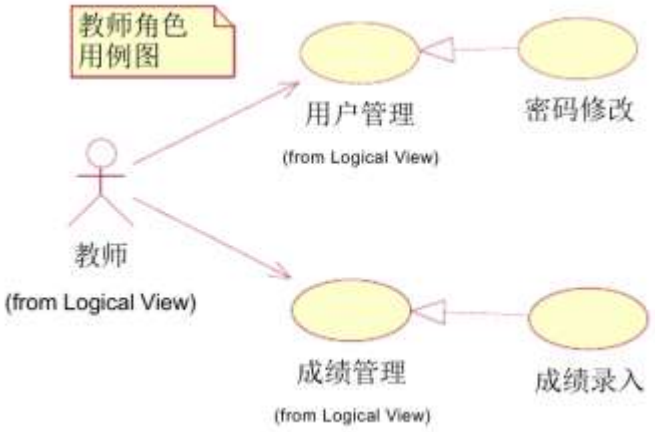
例：分层数据流图

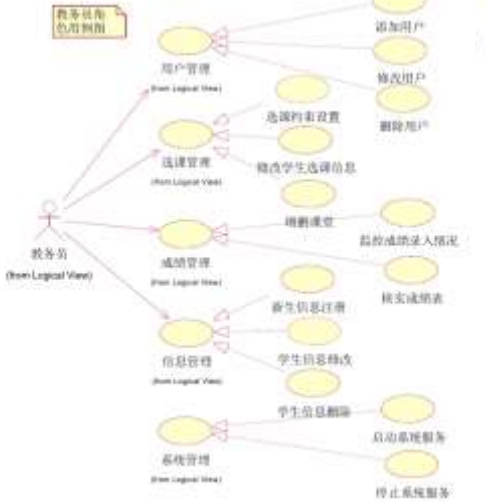

DFD/L0
F
S
F
DFD/L1
F
1
2
3
F
F
1.1
1.2
1.3
2.1
2.2
2.3
2.4
3.1
3.2
3.3
F
DFD/L2.1
DFD/L2.2
DFD/L2.3

例：UML描述的模型


系统用例图
学生
教师
用户管理
选课管理
成绩管理
信息管理
系统管理
教务员


学生角色
用例图
用户管理
密码修改
网上选课
个人课表查询
选课管理
课程评情查询
全部成绩
及格成绩
不及格成绩
查询成绩
成绩管理
专业计划完成情况
信息管理
学生信息查询
个人信息查询
学生
(from Logical View)


教师角色
用例图
用户管理
(from Logical View)
密码修改
教师
(from Logical View)
成绩管理
(from Logical View)
成绩录入


教务员角色
用例图
用户管理
添加用户
修改用户
删除用户
选课约束设置
选课管理
修改学生选课信息
增删课堂
成绩管理
监控成绩录入情况
核实成绩表
教务员
(from Logical View)
新生信息注册
信息管理
学生信息修改
学生信息删除
系统管理
启动系统服务
停止系统服务

4.4 需求分析举例

- 结构化分析方法（SA，Structure Analysis）是面向数据流进行需求分析的方法，适合于数据处理类型软件的需求分析。
- SA利用图形来表达需求，清晰、简明、易于学习和掌握。
- SA的特点：
  - 抽象概念
  - 面向数据流
  - 自顶向下逐层分析

SA常用的工具

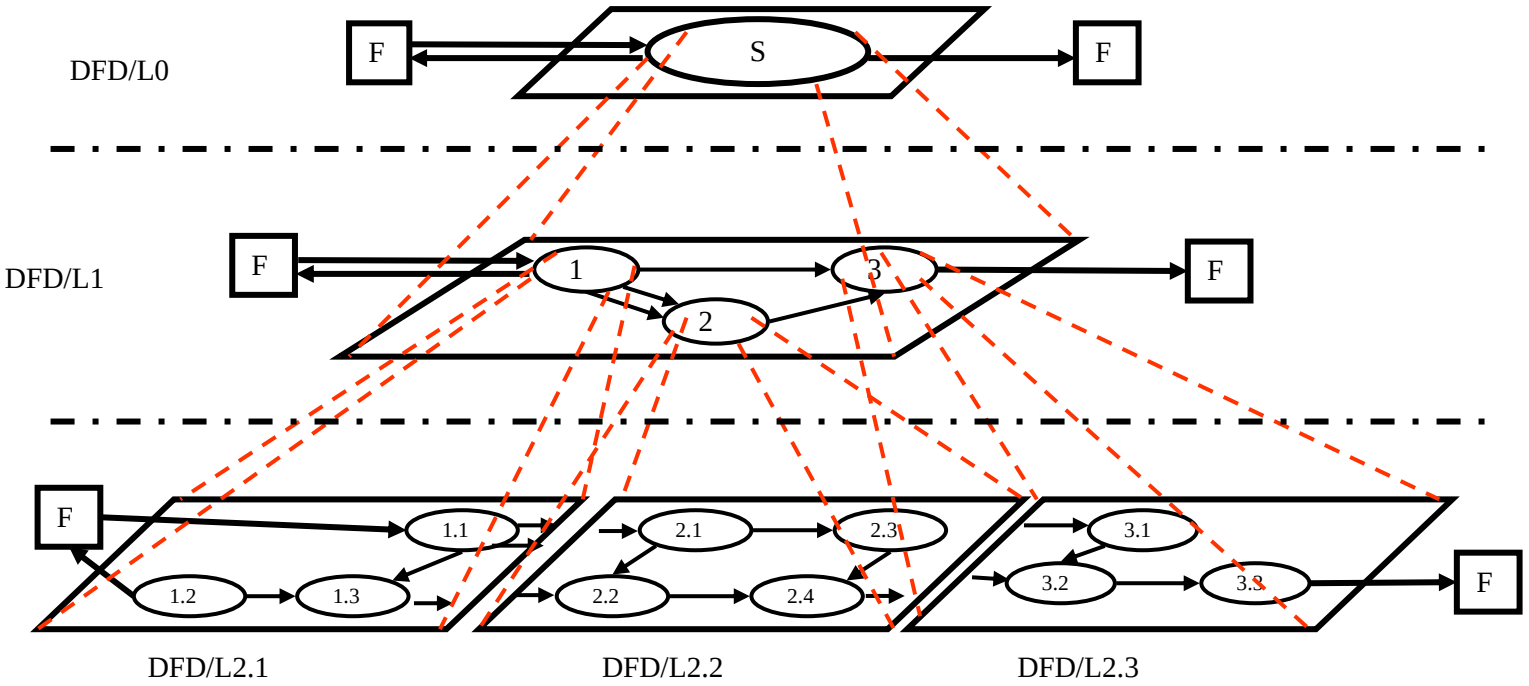
- 数据流图，DFD
- 数据字典，DD
- 结构化英语
- 判定树和判定表

SA方法的不足

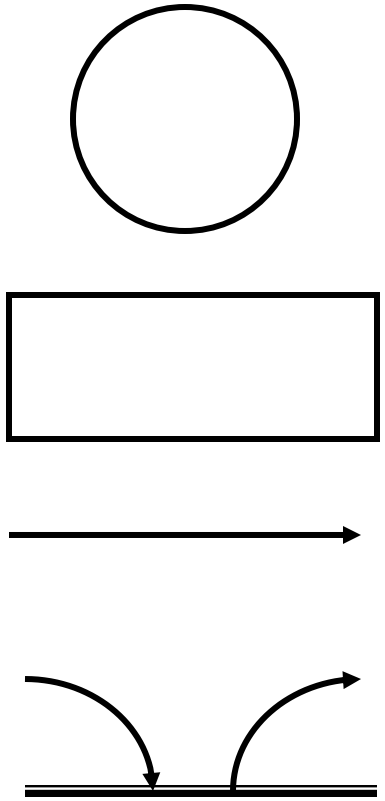
- 不提供对非功能性系统需求的有效理解和建模。
- 不提供对用户选择的合适方法的指导，也没有对方法适用的特殊环境的忠告。
- 往往产生大量的文档，系统的需求要素被隐藏在一大堆具体细节的描述中。
- 产生的模型太注重细节以致用户觉得难以理解，因而很难去验证这些模型的真实性。

1）数据流图

- 数据流图（DFD，Data Flow Diagram）也称Bubble Chart或Data Flow Graph，是描述数据处理的工具。
- DFD从数据传递和加工的角度，以图形的方式刻图数据流从输入到输出的移动变换过程。
- DFD只反映系统必须完成的逻辑功能，是一种功能模型。

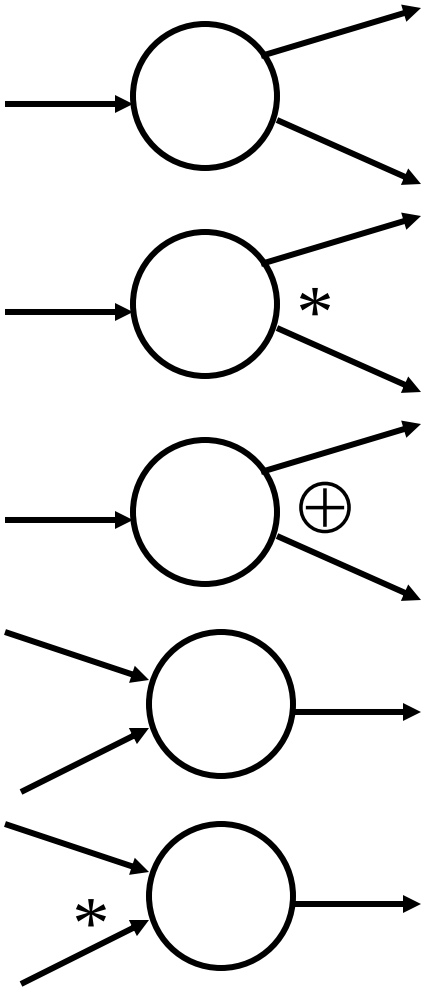


DFD的主要图形元素：



- 加工
  - 输入数据在此进行变换产生输出数据
  - 其中要注明加工的名字
- 源点和汇点
  - 其中要注明源点和汇点的名字
- 数据流
  - 被加工的数据与流向
  - 要注明数据流名字，可用名词、名词性短语。
- 数据存储文件
  - 要注明文件名字，用名词或名词短语

表明多个数据流与加工之间关系的符号



有A则有B或C，或两者都有

有A则有B与C，两者都有

有A则有B或C，但不会同时有B与C

当A或B有一个存在，就有C

当A与B都存在，就有C

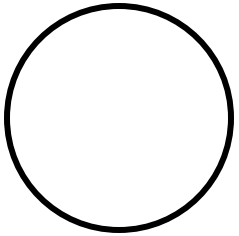
符号对照



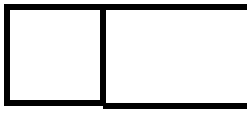


数据源点和汇点





加工



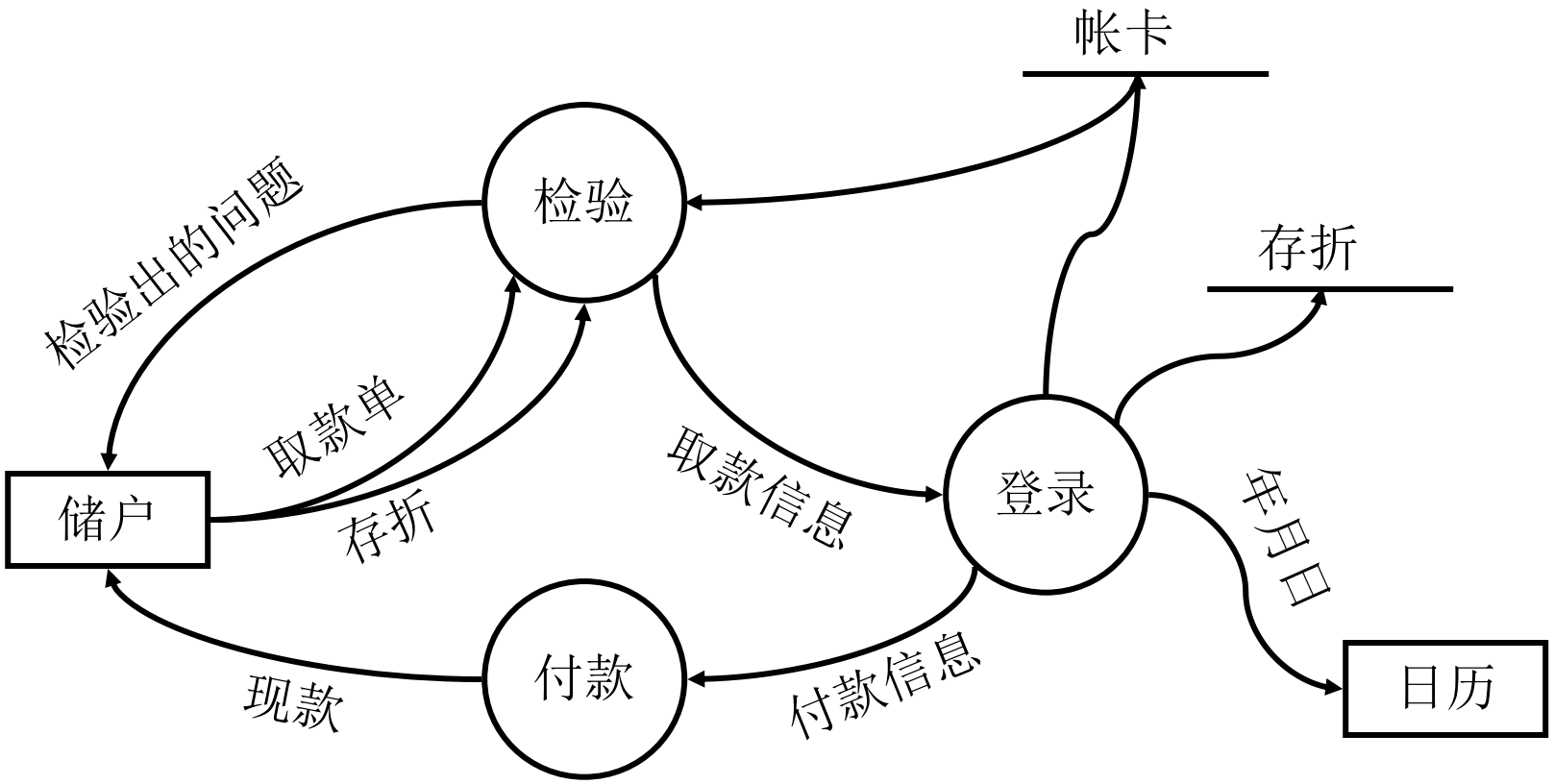
数据文件





数据流

办理取款手续的数据流图



数据流图的作用：

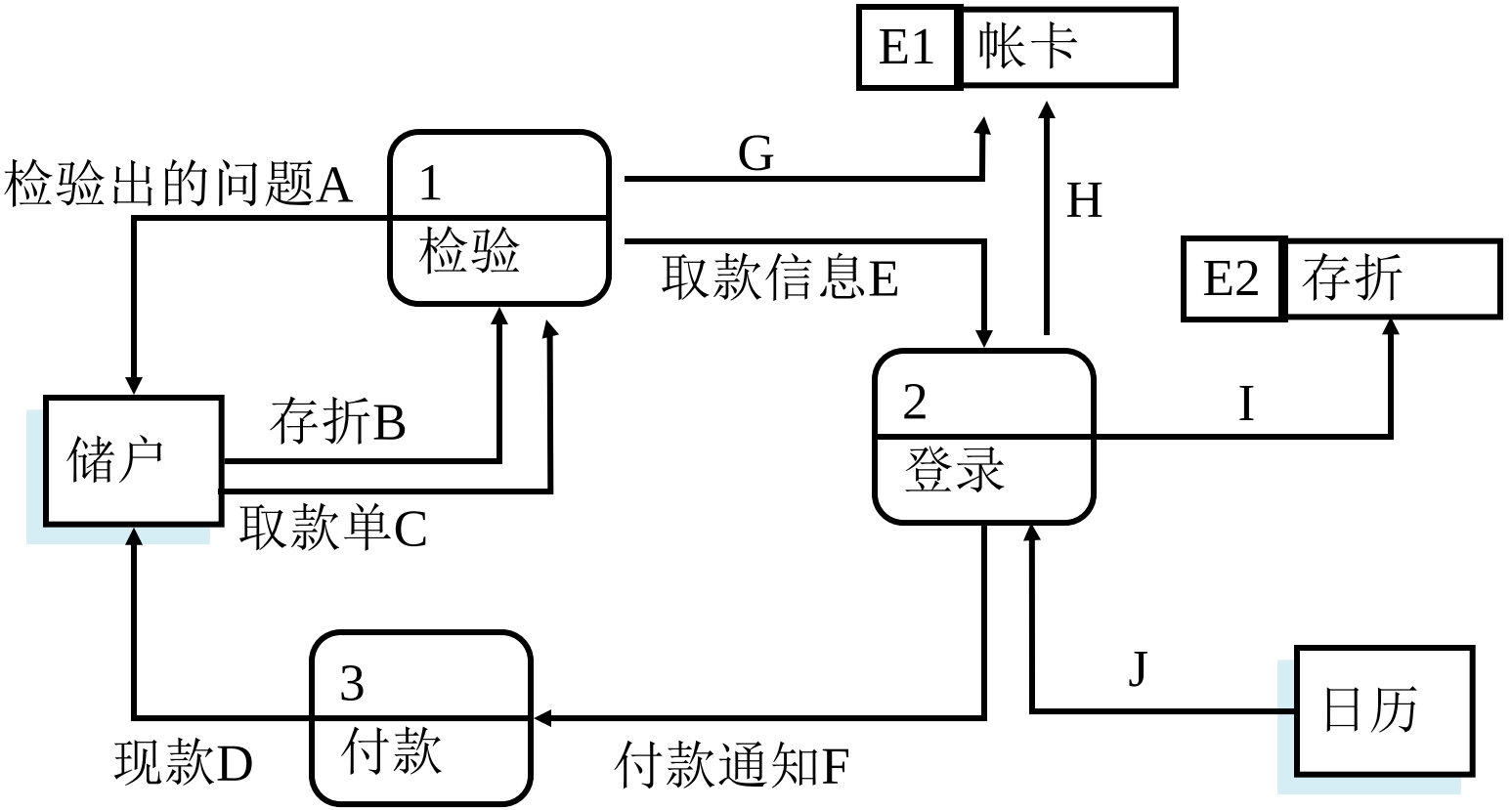
1）便于用户表达功能需求和数据需求及其联系；

2）便于两类人员共同理解现行系统和规划系统的框架；

3）清晰表达数据流的情况；

4）有利于系统建模。

银行取款过程的数据流图



等价矩阵

	储户	日历	E1帐卡	E2帐卡	检验	登录	付款
储户	—					B，C	
日历		—				J	
E1帐卡			—		G		
E2存折				—			
检验	A		H	I	—	E	
登录						—	F
付款	D						—

画单张数据流图时，必须注意以下原则：

1. 一个加工的输出数据流不应与输入数据流同名，即使它们的组成成分相同。
2. 保持数据守恒。也就是说，一个加工所有输出数据流中的数据必须能从该加工的输入数据流中直接获得，或者说是通过该加工能产生的数据。
3. 每个加工必须既有输入数据流，又有输出数据流。
4. 所有的数据流必须以一个外部实体开始，并以一个外部实体结束。
5. 外部实体之间不应该存在数据流。

DFD的画法

一般遵循“由外向里”的原则，即先确定系统的边界或范围，再考虑系统的内部，先画加工的输入和输出，再画加工的内部。即：

(1)识别系统的输入和输出。

(2)从输入端至输出端画数据流和加工，并同时加上文件。

(3)加工的分解“ 由外向里”进行分解。

(4)数据流的命名，名字要确切，能反映整体。

(5)各种符号布置要合理，分布均匀，尽量避免交叉线。

(6)先考虑稳定态，后考虑瞬间态。如系统启动后在正常工作状态，稍后再考虑系统的启动和终止状态。

2） 数据字典

- DD（Data Dictionary）就是用来描述数据流和加工的具体含义。
- DFD只描述了系统的“分解”，它没有表达出各个数据和加工的具体含义。
- 有了DFD和DD，才算是较完整的地描述了一个系统。
- 对于DFD中出现的所有命名的图形元素在DD中作出一个词条加以定义，使得每一个图形元素的名字有一个确切的解释。

数据流词条描述

- 数据流名：
  - 说明：简要介绍作用，即它产生的原因和结果。
  - 数据流来源：来自何方。
  - 数据流去向：去向何处。
  - 数据流构成：数据结构。
  - 每个数据量流通量：数据量，流通量。
- 数据元素名：
  - 类型：数字（离散值，连续值），文字（编码类型）
  - 长度
  - 取值范围
  - 相关的数据元素及数据结构
- 数据文件名：
  - 简述：存放的是什么数据。
  - 输入数据
  - 输出数据
  - 数据文件组成：数据结构
  - 存储方式：顺序，直接，关键码
  - 存取频率

加工逻辑词条描述

- 加工名：
  - 加工编号：反映该加工的层次。
  - 简要描述：加工逻辑及功能简述。
  - 输入数据流
  - 输出数据流
  - 加工逻辑：简述加工程序，加工顺序。
- 源点及汇点词条描述
  - 名称：外部实体名称。
  - 简要描述：什么外部实体。
  - 有关数据流
  - 数目

在数据字典的定义式中出现的符号

符号	含义	解释
=	被定义为	
+	与	x=a+b，表示x由a和b组成
[…,…]	或	x=[a,b]，x=[a|b]，表示x由a或由b组成
[…|…]	或	
{…}	重复	x={a}，表示x由0个或多个a组成
m{…}n	重复	x=3{a}8,表示x中至少出现3次a,至多出现8次a
(…)	可选	x=(a)，表示a可在x中出现，也可不出现
"…"	基本数据元素	x="a"，表示x可取值为a的数据元素
..	连接符	x=1..9，表示x可取1到9之中的任一值

例：存折格式

户名　　所号　　帐号

开户日　　性质　　印密

日期 处月日	摘要	支出	存入	余额	操作	复核



例：存折在DD中的定义

- 存折=户名+所号+帐号+开户日+性质+(印密)+1{存取行}50
- 户名=2{字母}24
- 所号="001"…"99999999"
- 开户日=年+月+日
- 性质="1"…"6"
- 印密="0"
- 存取行=日期+(摘要)+支出+存入+余额+操作+复核
- 日期=年+月+日
- 年="00".."99"
- 月="01".."12"
- 日="01".."31"
- 摘要=1{字母}4
- 支出=金额
- 金额="0000000.01".."9999999.99"
- 操作="00001".."99999"

3）加工逻辑说明

写基本加工逻辑的说明时，应满足如下要求：

- 对DFD的每一个基本加工，必须有一个加工逻辑说明；
- 加工逻辑说明必须描述基本加工如何把输入数据流变换为输出数据流的加工规则；
- 加工逻辑说明必须描述实现加工和策略而不是实现加工的细节。

结构化英语（Structure English）

- 是在自然语言基础上加入一些限制而得到的语言，是使用有限的词汇和有限的语句来描述加工逻辑。
- 由命令动词、数据词典中定义的名字、有限的自定义词和控制结构关系词等组成。

例：商店业务处理系统中“检查发货单”

```
IF the invoice exceeds $500 THEN
        IF the account has any invoice more than 60 days overdue THEN
                the confirmation pending resolution of the debt
        ELSE (account is in good standing)
                issue confirmation and invoice
        ENDIF
ELSE (invoice $500 or less)
        IF the account has any invoice more than 60 days overdue THEN
                issue confirmation, invoice and write message on credit action report
        ELSE (account is in good standing)
                issue confirmation and invoice
        ENDIF
ENDIF
```

判定表（Decision Table）

- 判定表能够把在什么条件下系统应完成哪些操作，表达得十分清楚、准确、一目了然。这是用语言说明难以准确、清楚表达的。判定表的组成如下：

I条件类别	II条件组合
III操作	IV操作执行

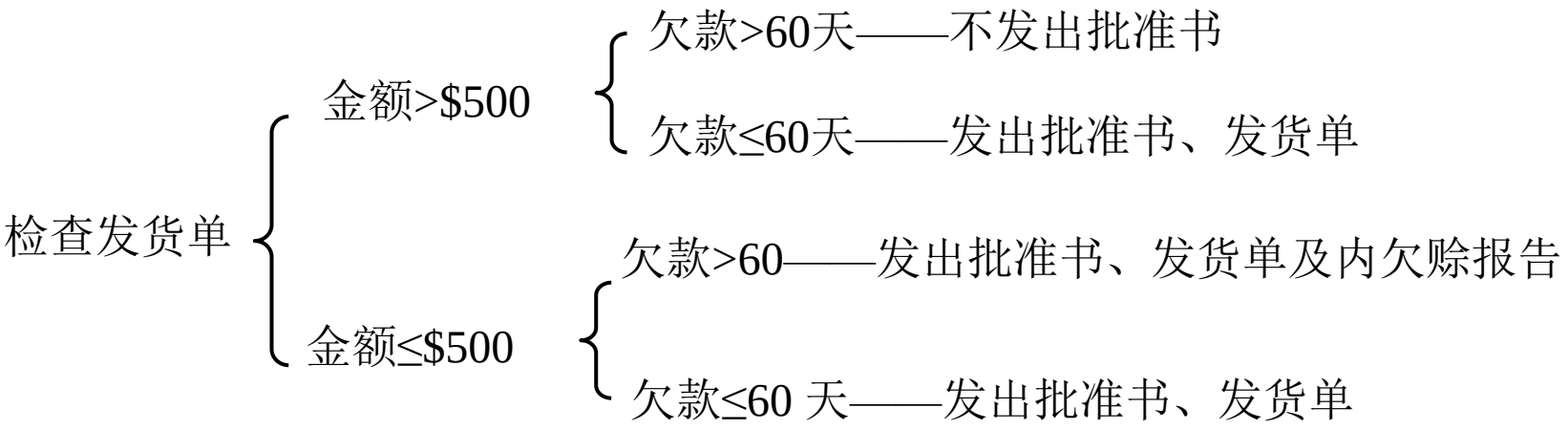
- 判定表分4个区：
  - I条件类别：列出所有条件类别
  - II条件组合：列出所有条件组合
  - III操作：列出所有操作
  - IV操作执行：标出在相应的组合条件下，某个操作是否执行或执行情况。

例：“检查发货单”的判定表

		1	2	3	4
条件	发货单金额	＞$500	＞$500	≤$500	≤$500
	赊欠情况	＞60天	≤60天	＞60天	≤60天
操作	不发出批准书	√			
	发出批准书		√	√	√
	发出发货单		√	√	√
	发出赊欠报告			√	

判定树（Decision Tree）

- 用图形描述加工逻辑，其特点是结构清晰，易读易懂。
- 例：“检查发货单”的判定树



练习题：连锁超市进货操作流程为：厂家持“定货单”及“厂家送货单”送货，库管科检查货物。

- 如果来货相符，库管科做收货确认，（包括：在“定货单”之“点核数量”栏手工填写收货数量；在电脑中找出“定货单”，在“点核数量”栏输入收货数量；在“收货日报表”上记录该收货；在“定货单”、“厂家送货单”上加盖库管科公章），然后库管科打印商品条码，收送货组与卖场办理移交手续，卖场收货人在“定货单”上签名盖章，日末，库管可将单据汇总，连同“收货日报表”分别送卖场、财会，并自留一份，第二天电脑科打印“验收进货单”，分送卖场、采购、财会。
- 如果来货不符，通知负责该次定货之采购主任，采购主任填写“定货修正单”，报采购部主管签名同意，“定货修正单”交库管科，库管科从电脑中提出原“定货单”，按“定货修正单”内容进行修改，将修改后之新“定货单”打印（一式三份），和原“定货单”连在一起，最后进行上述来货相符后的工作。
- 要求：

  （1）画出该系统的数据流图。

  （2）给出该系统主要处理、数据存储或外部实体的数据字典。

课堂练习——教学管理系统建模

一个简化的教学管理系统的描述如下：

●与学生相关的活动：

◇学生向系统提出注册申请

◇系统向学生反馈学籍通知

◇系统向部分优秀学生发出奖励通知

●与教务处相关的活动：

◇教务处将学籍审批结果输入系统

◇系统向教务处显示统计报告

◇教务处向系统提供奖励凭证

●与系教学办相关的活动：

◇系统显示学籍报告管理情形

◇系教学办将学籍处理意见和奖励建议输入系统

●与教师相关的活动：

◇系统将教学安排通知教师

◇教师将学生成绩输入系统

1.根据以上描述，画出该系统的顶层数据流图；

2.写出相关的数据文件；

3.如果系统的主要功能是注册管理、学籍管理、成绩管理、奖励管理，请画出系统的第一层数据流图。

软件开发

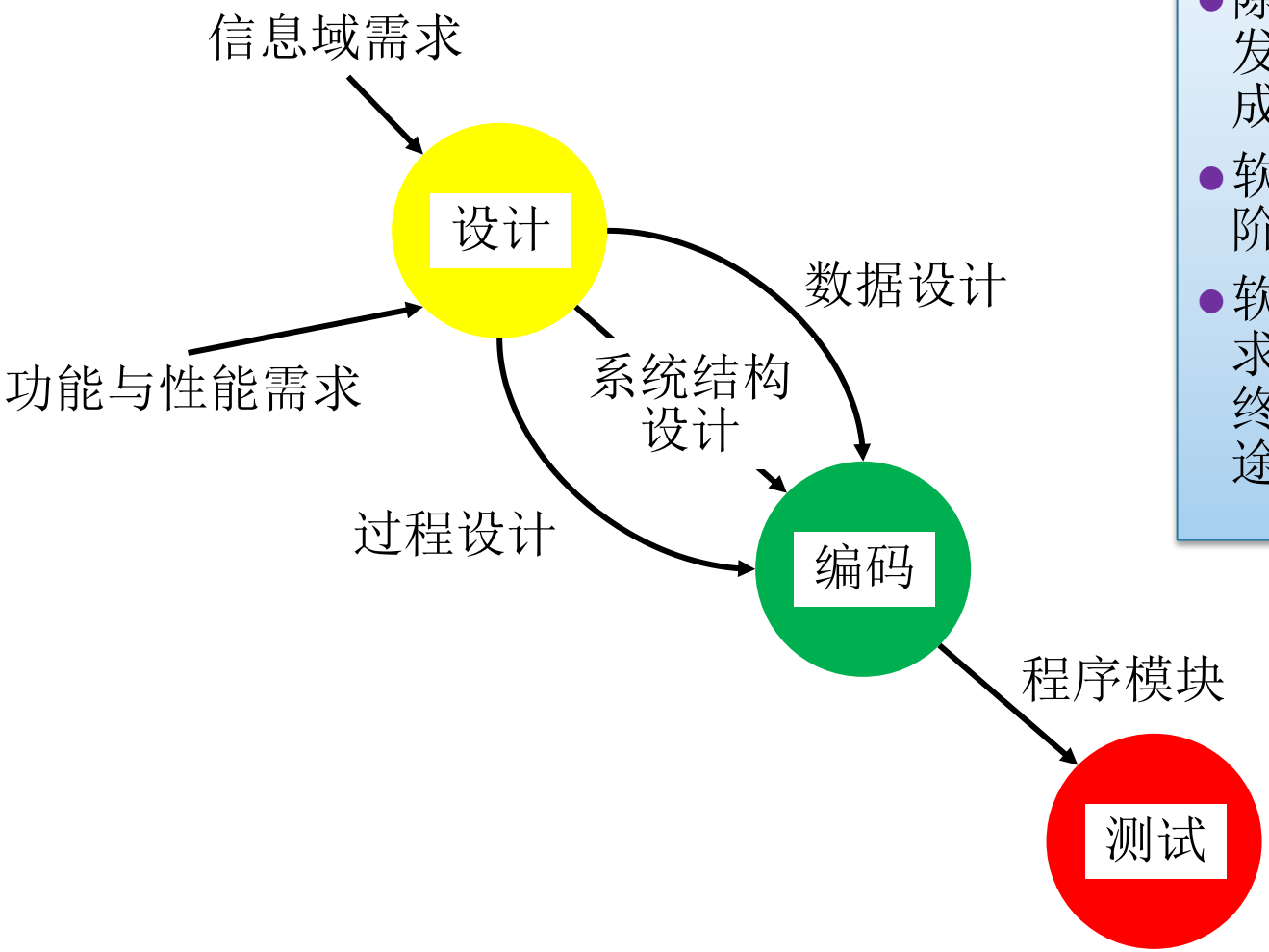
5 软件设计

主要内容

① 软件设计任务
② 软件设计要素
③ 体系结构设计
④ 结构化设计方法

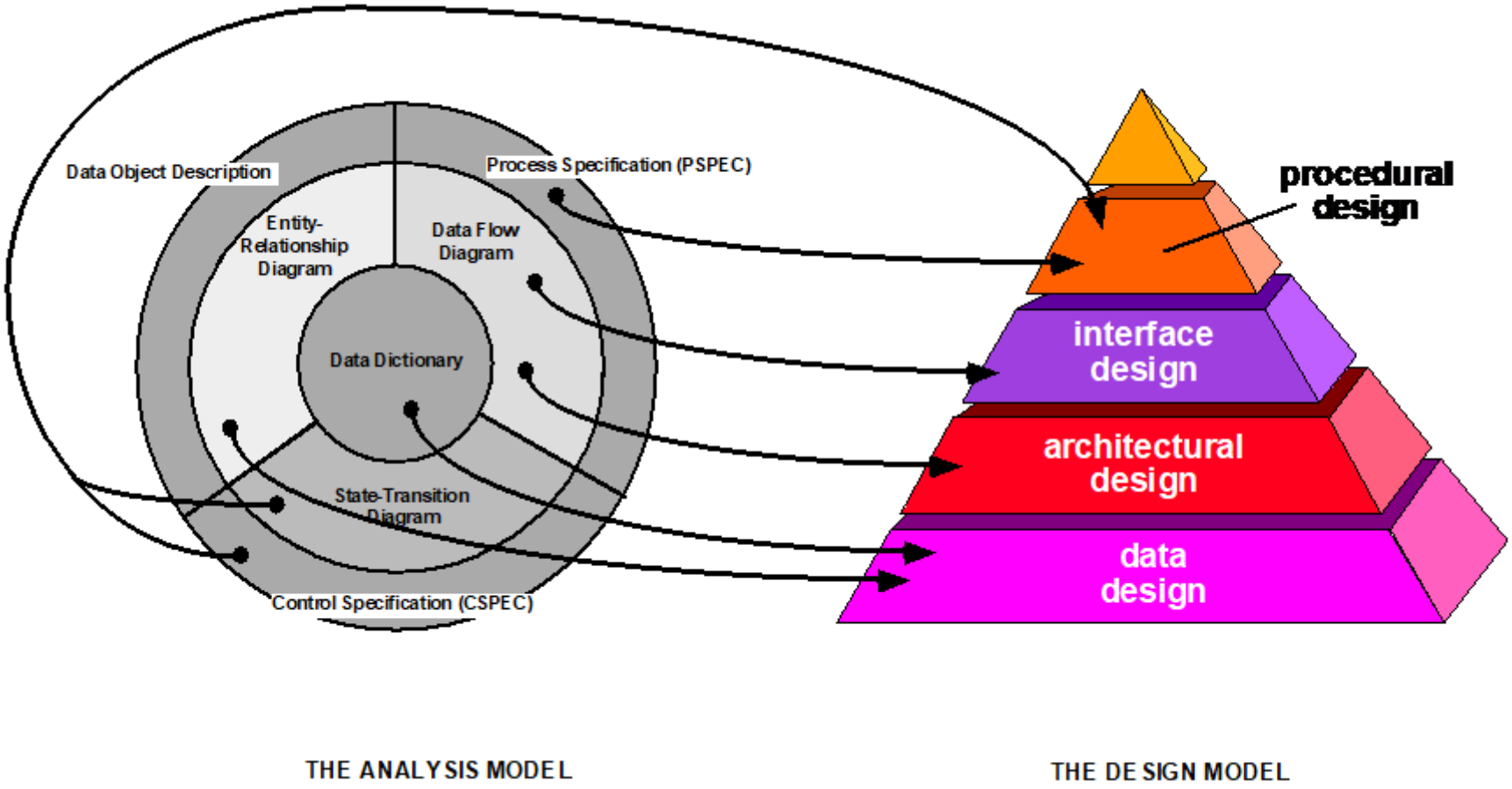


5.1 软件设计的重要性



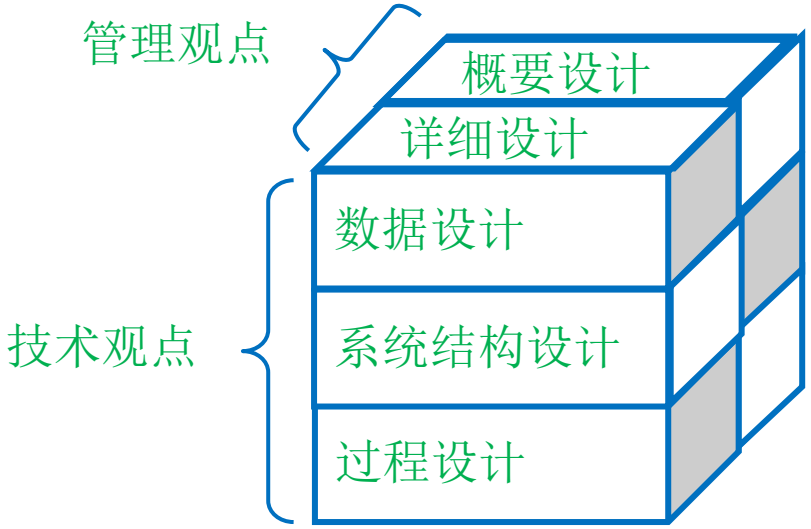
- 除维护阶段以外，开发阶段占软件工程总成本的75%以上。
- 软件设计是软件开发阶段最重要的步骤。
- 软件设计是将用户要求准确地转化顾为最终的软件产品的唯一途径。

Analysis to Design



设计分类：

- 数据设计
  - 侧重于数据结构的定义
- 系统结构设计
  - 定义软件系统各主要成份之间的关系。
- 过程设计
  - 把结构成份转换成软件的过程性描述。



5.2 软件设计任务

- 软件设计的目标是要取得最佳解决方案。
- 最佳是指所候选方案中，就节省开发费用，降低资源消耗，缩短开发时间的条件，选择能够赢得较高生产率、较高的可靠性和可维护性的方案。

概要设计的任务：

① 制定规范
② 软件系统结构的总体设计
③ 处理方式设计
④ 数据结构设计
⑤ 数据的保护性设计
⑥ 可靠性设计
⑦ 编写概要设计文档

详细设计的任务：

① 确定软件各个组成部分内算法以及部分的内部数据组织。
② 选定某种过程表达形式来描述各种算法。
③ 进行详细设计评审。

5.3 软件设计的要素



软件结构

体系结构

程序结构

数据结构

软件过程

设计思想

模块化

抽象化

信息隐蔽

自顶向下，逐步细化

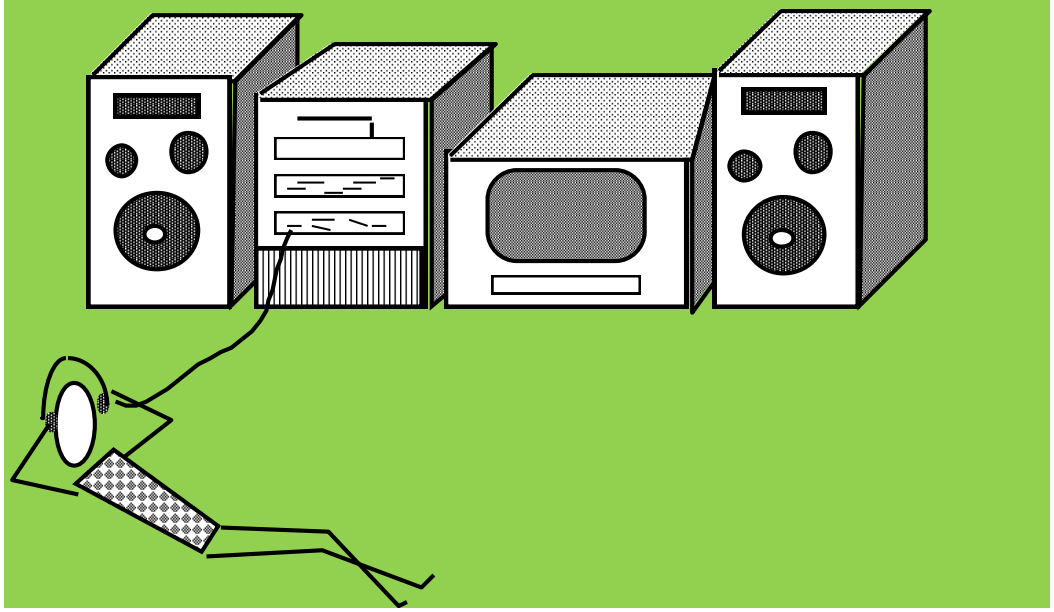
Design Principles

- The design process should not suffer from 'tunnel vision.'
- The design should be traceable to the analysis model.
- The design should not reinvent the wheel.
- The design should "minimize the intellectual distance" between the software and the problem as it exists in the real world.
- The design should exhibit uniformity and integration.
- The design should be structured to accommodate change.
- The design should be structured to degrade gently, even when aberrant data, events, or operating conditions are encountered.
- Design is not coding, coding is not design.
- The design should be assessed for quality as it is being created, not after the fact.
- The design should be reviewed to minimize conceptual (semantic) errors.

From Davis [DAV95]

Modular Design

easier to build, easier to change, easier to fix ...



模块是模块化设计和制造的功能单元，具有三大特征：

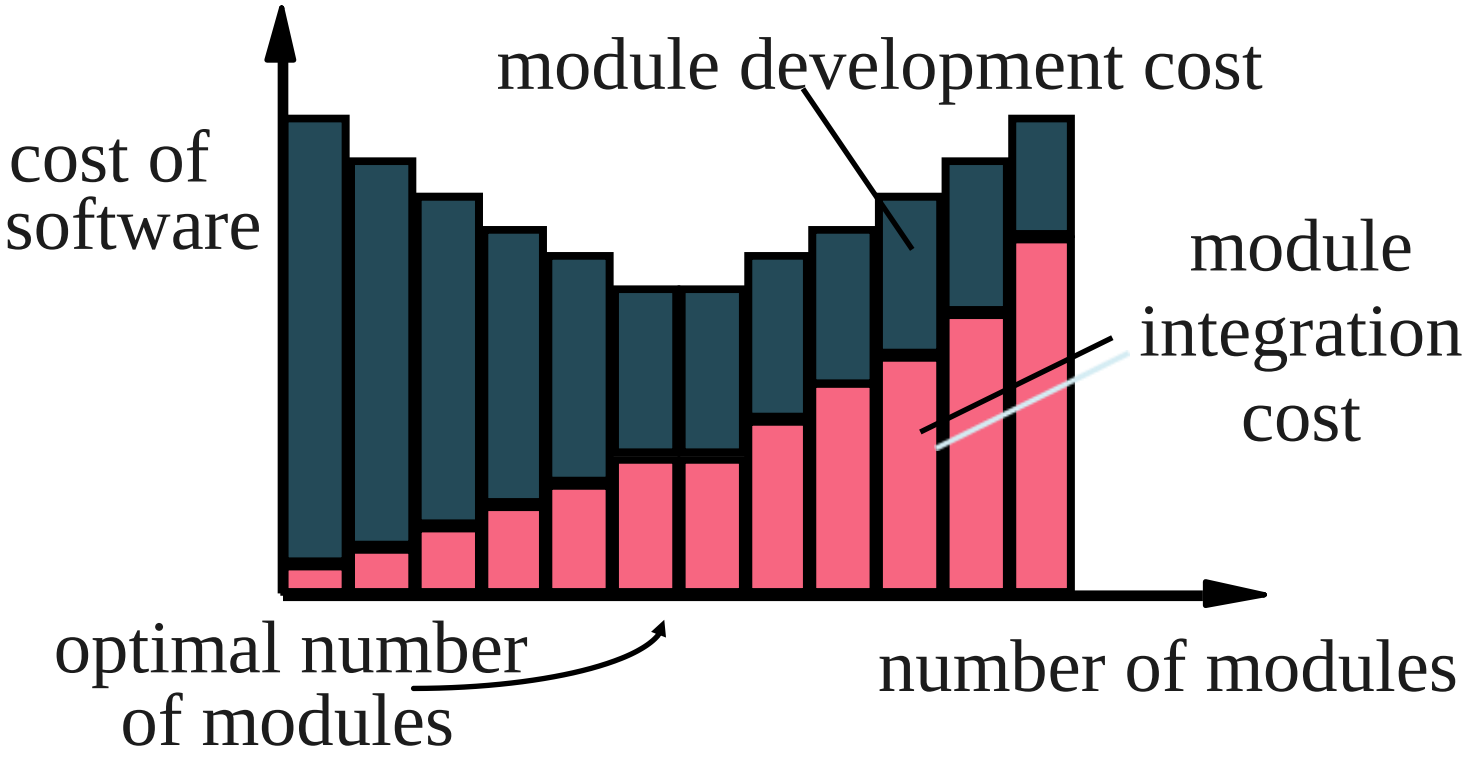
① 相对独立性，可以对模块单独进行设计、制造、调试、修改和存储，这便于由不同的专业化企业分别进行生产；

② 互换性，模块接口部位的结构、尺寸和参数标准化，容易实现模块间的互换，从而使模块满足更大数量的不同产品的需要；

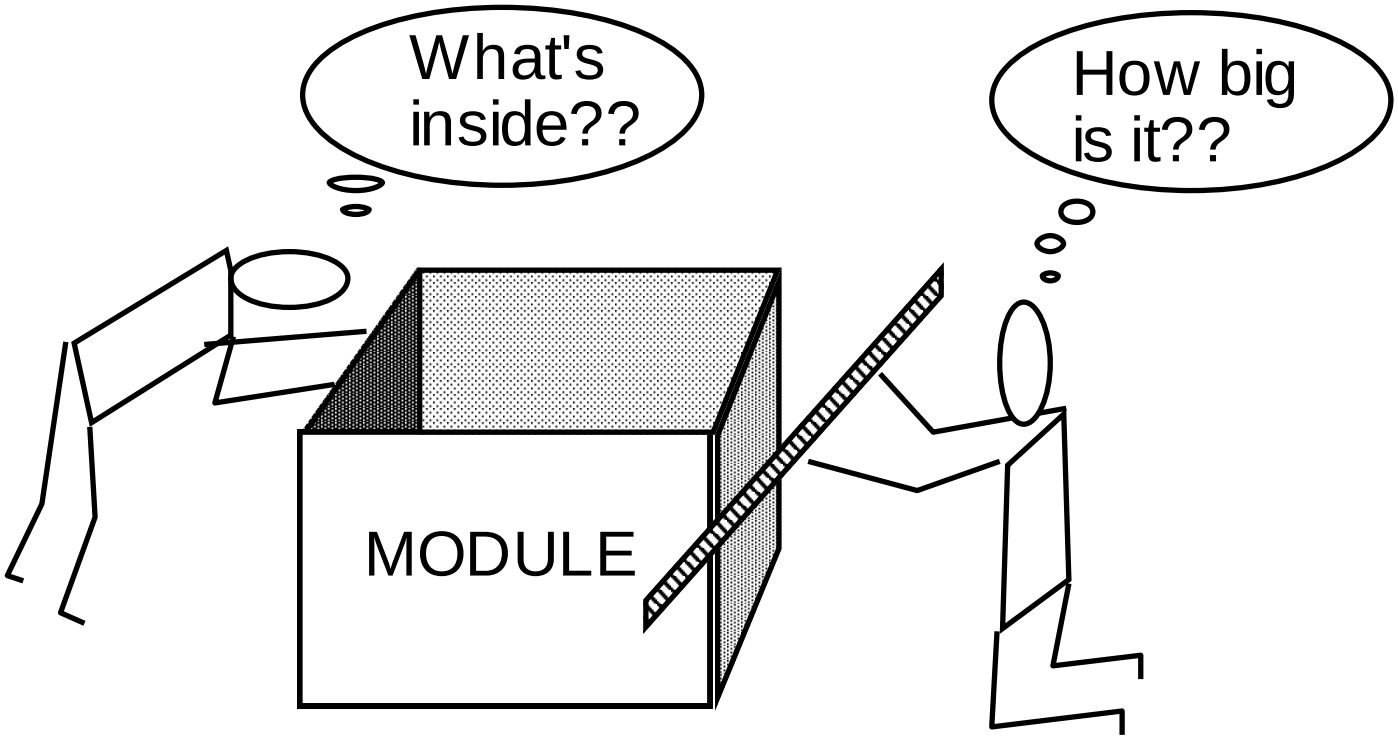
③ 通用性，有利于实现横系列、纵系列产品间的模块的通用，实现跨系列产品间的模块的通用。

Modularity: Trade-offs

What is the "right" number of modules for a specific software design?



Sizing Modules: Two Views

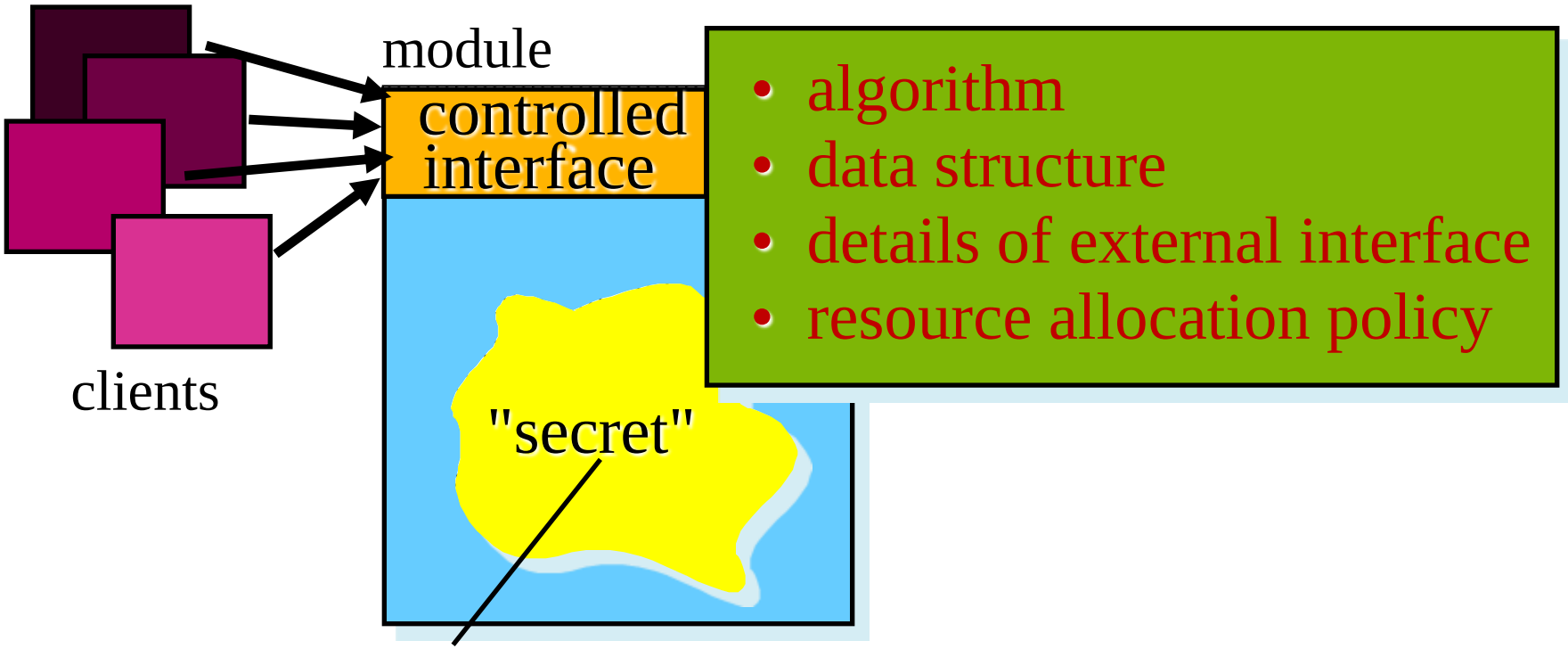


Functional Independence

COHESION - the degree to which a module performs one and only one function.

COUPLING - the degree to which a module is "connected" to other modules in the system.

Information Hiding



a specific design decision

5.4 体系结构设计

讨论问题：

1.一般来说，软件设计工作中最先开展什么设计，为什么？（概要设计、详细设计；数据设计、过程设计、结构设计）

2.软件体系结构设计要哪些工作？

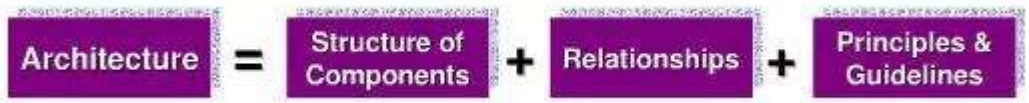
- 大型系统总是要分解成一些子系统，这些子系统提供一些相关的服务。
- 初始设计过程的任务是识别出这些子系统控制和通信的框架，这个过程叫体系结构设计（Software Architecture Design），其输出是软件体系结构的描述。
- 体系结构影响性能、健壮性、分布能力和可维护性等。
- 一个应用程序的结构和风格依赖于系统的非功能性需求。
- 对于绝大多数大型软件的体系结构而言，不可能由一个模型描述清楚。系统的不同部分可能使用不同的体系结构模型来设计。

什么是架构？

ISO/IEC/IEEE 42010中对架构的定义如下：

> The fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution.

ISO/IEC/IEEE 42010:2011, *Systems and software engineering — Architecture description*



定义了架构的三要素：

- 职责明确的模块或者组件
- 组件间明确的关联关系
- 约束和指导原则

体系结构设计方法



设计过程

- 系统结构化
- 控制模块
- 模块分解

设计的内容

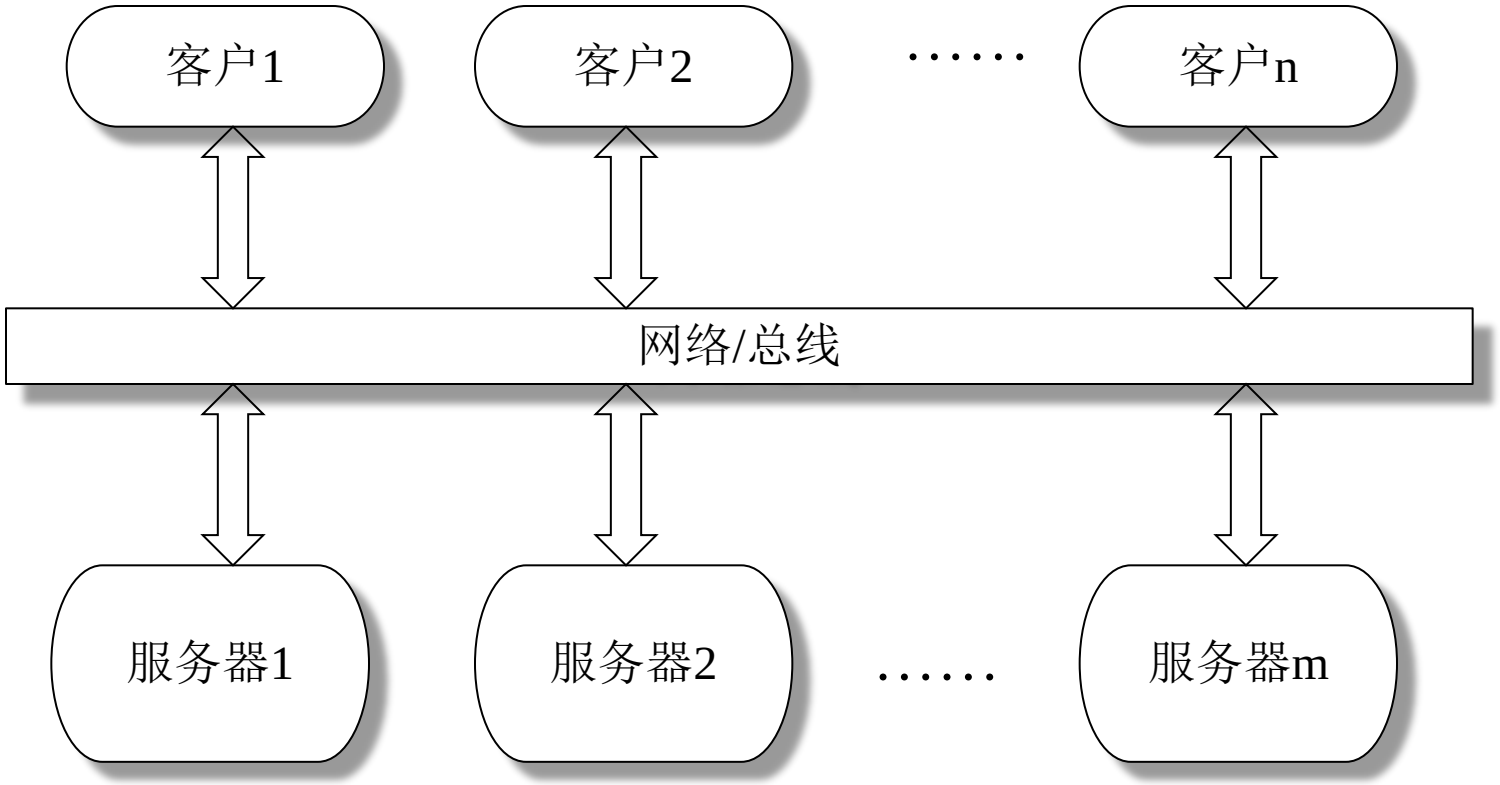
- 静态结构模型
- 动态结构模型
- 接口模型
- 关系模型

体系结构标准模型

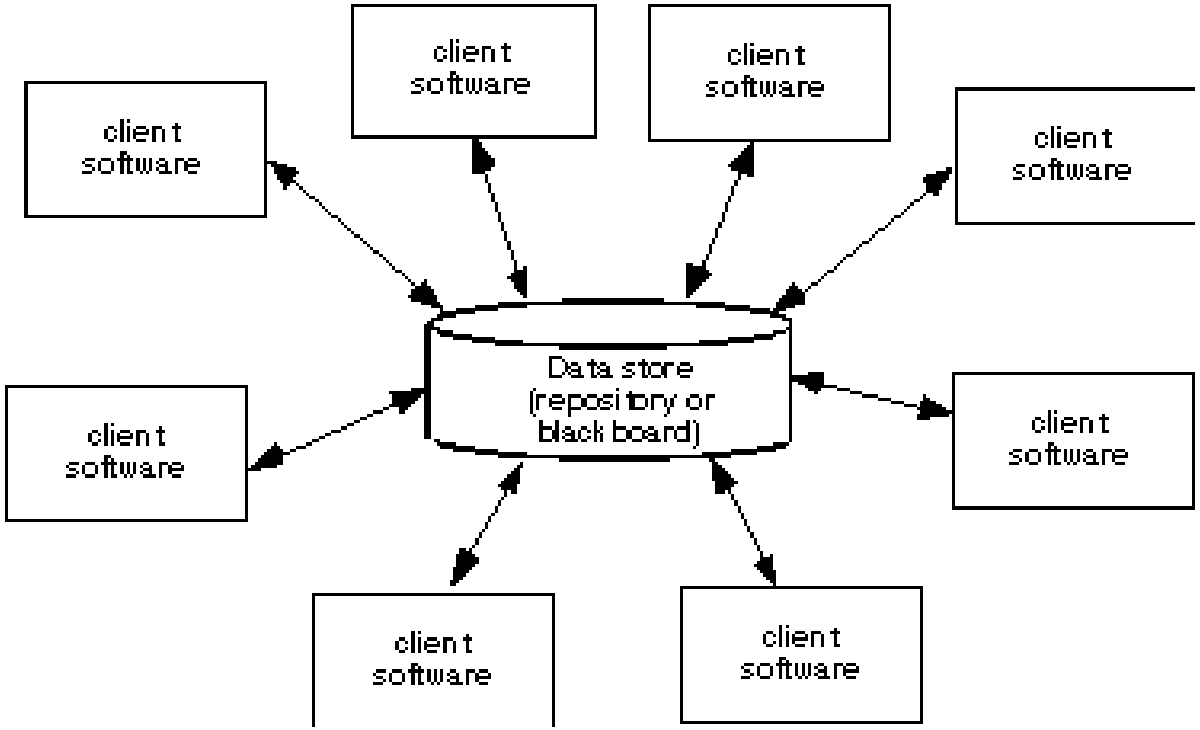
- 客户/服务器（client/server architecture）
- 数据中心结构（data-centered architecture）
- 数据流结构（data flow architecture）
- 调用返回结构（call and return architecture）
- 事件驱动架构（Event-driven Architecture）
- 微服务架构（μ-services Architecture）
- 层次结构（layered architecture）
- 模型-视图-控制器架构（MVC）



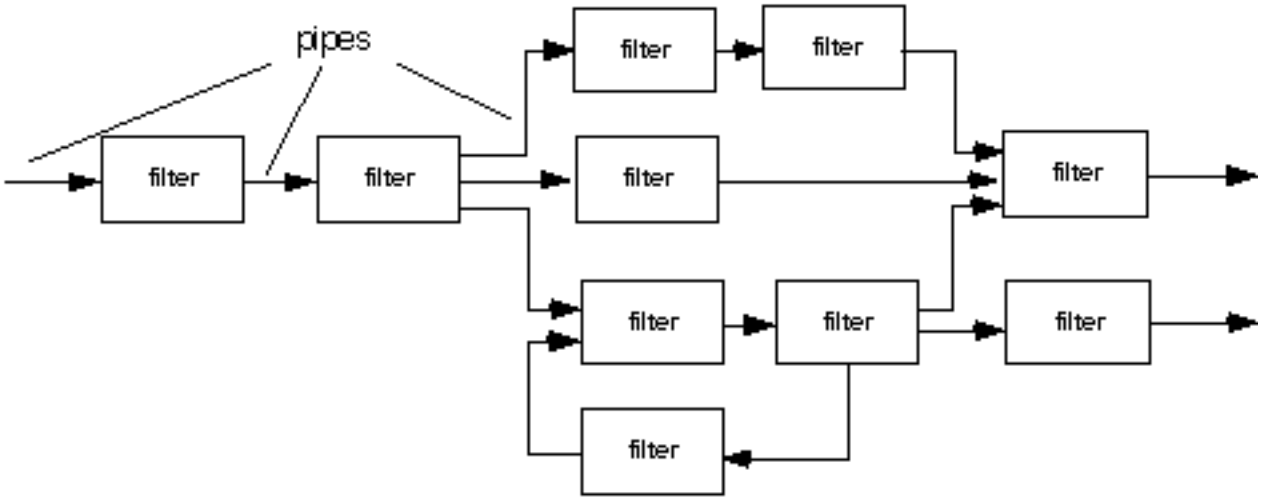
Client/Server Architecture



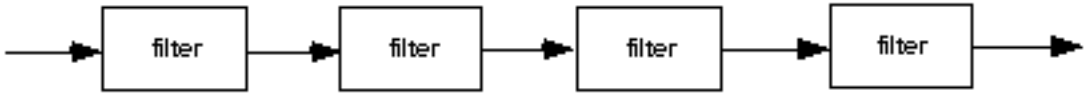
Data-Centered Architecture



Data Flow Architecture

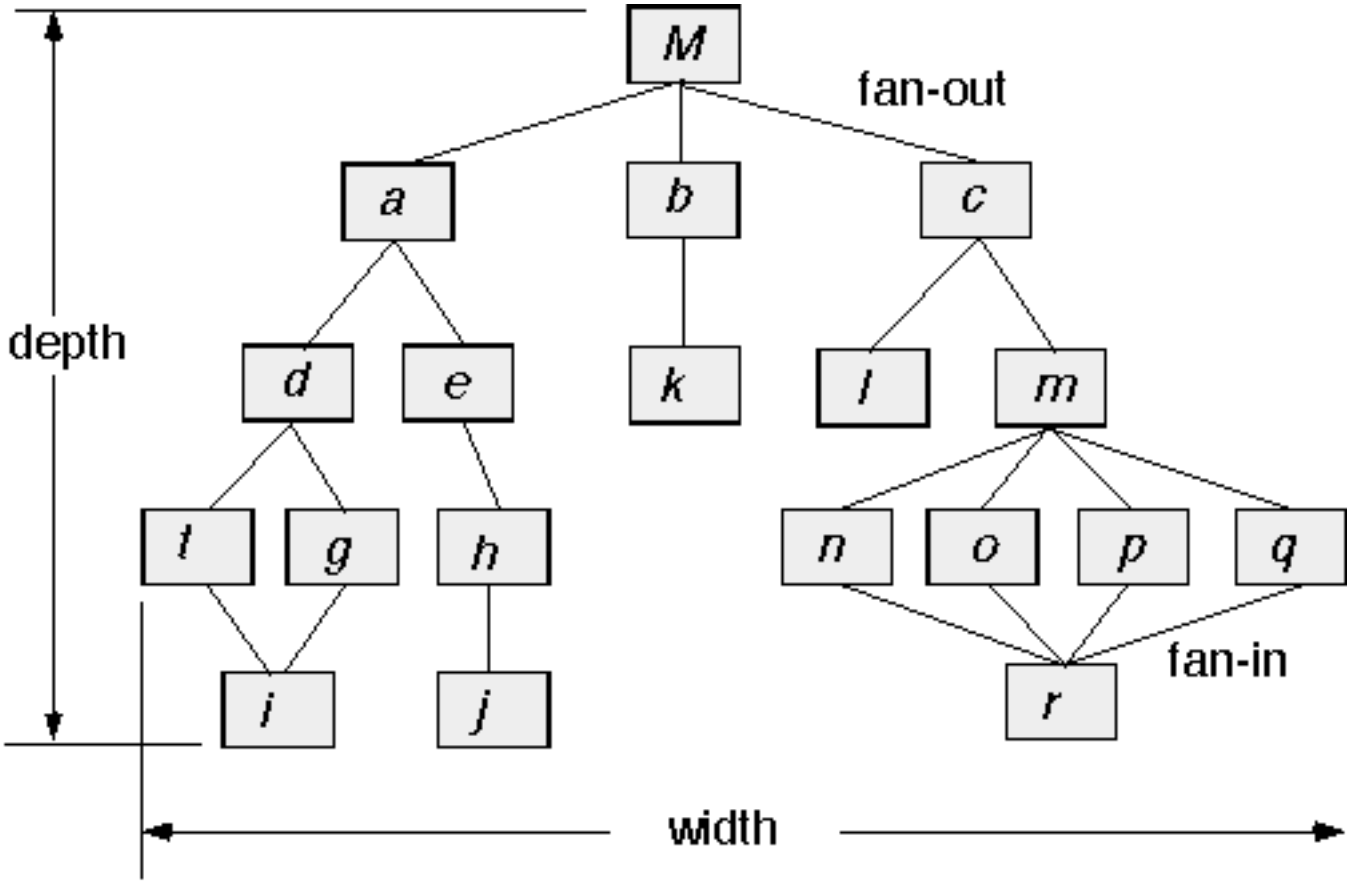


(a) pipes and filters

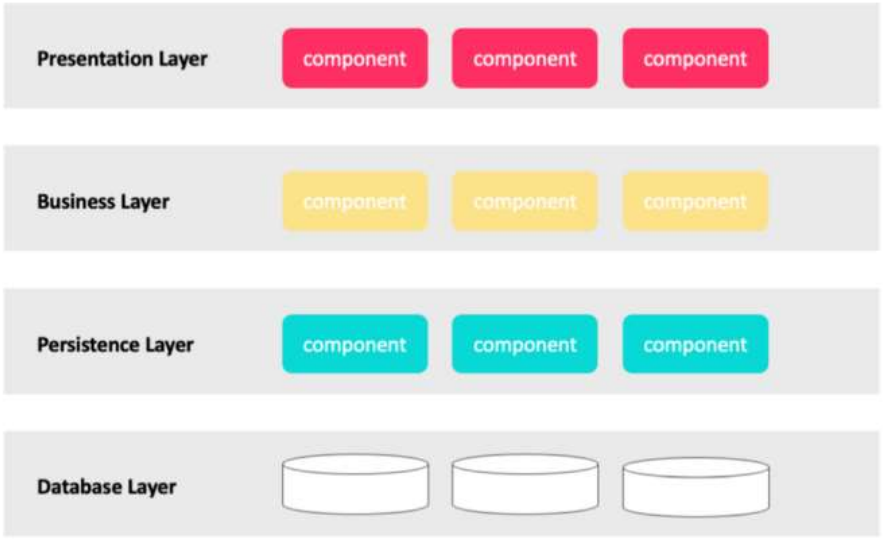


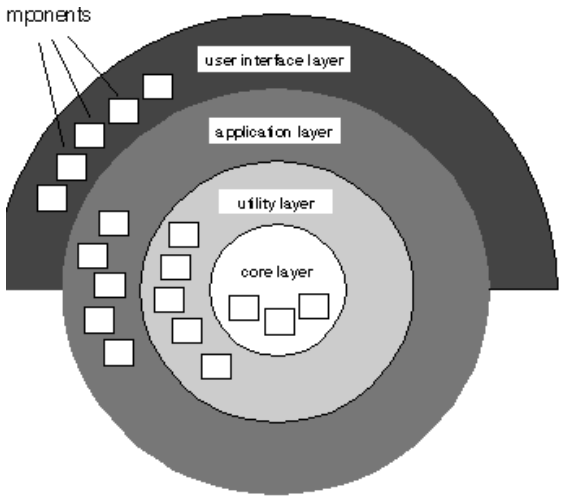
(b) batch sequential

Call and Return Architecture

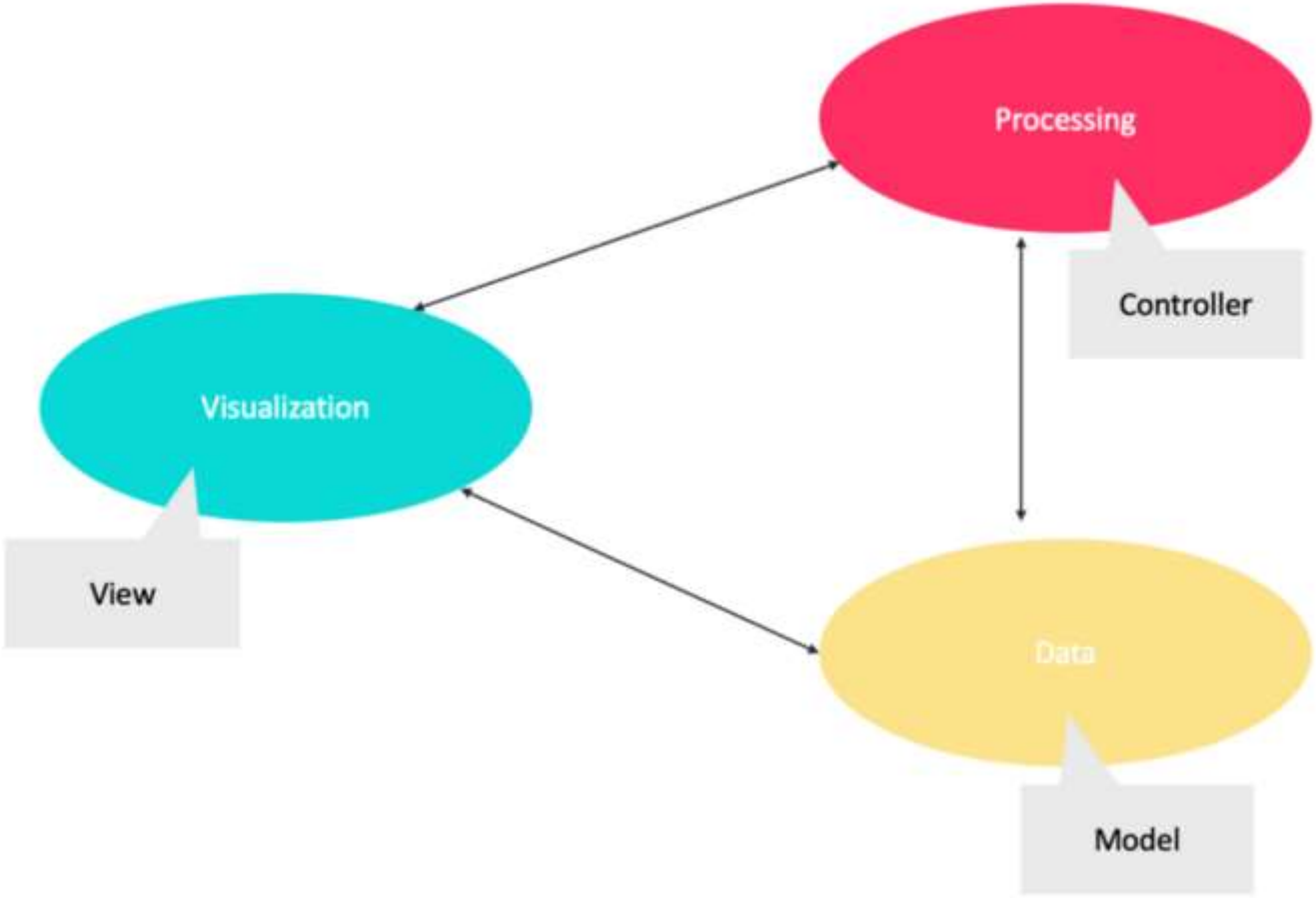


Layered Architecture

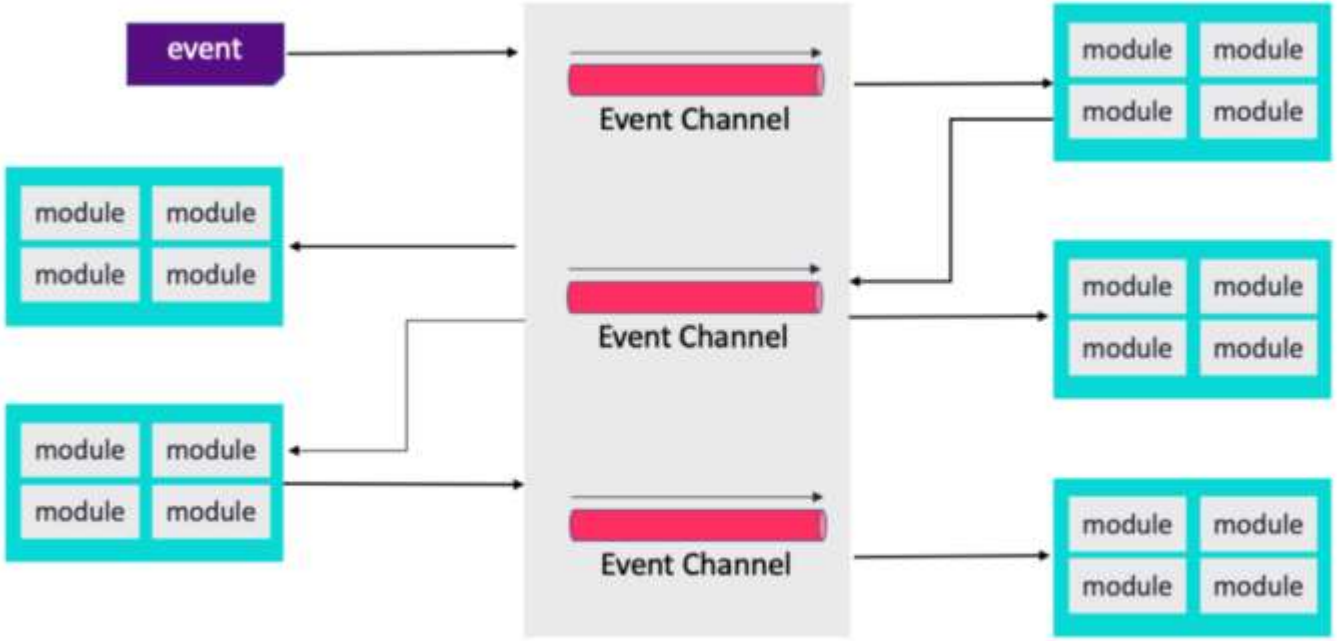




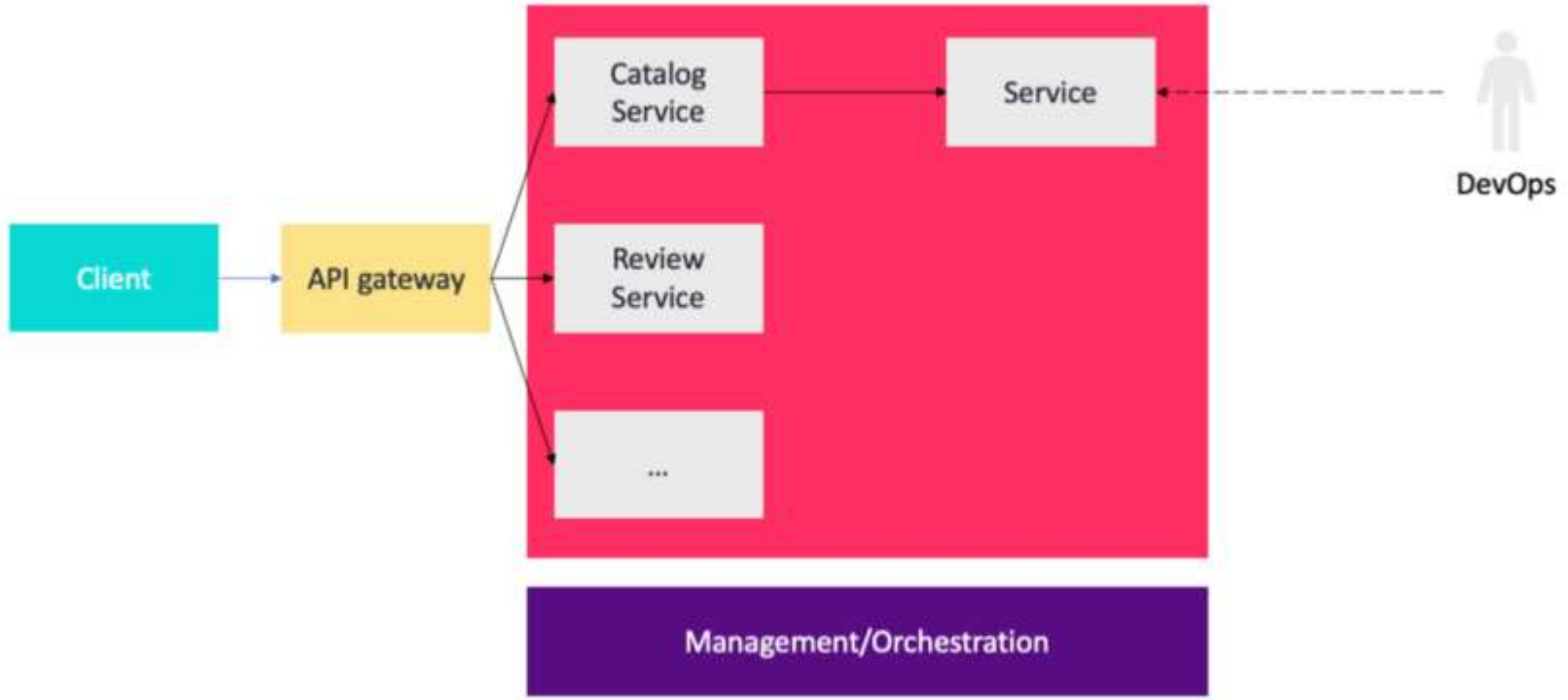
MVC（模型 - 视图 - 控制器架构）



Event-driven Architecture



μ-services Architecture

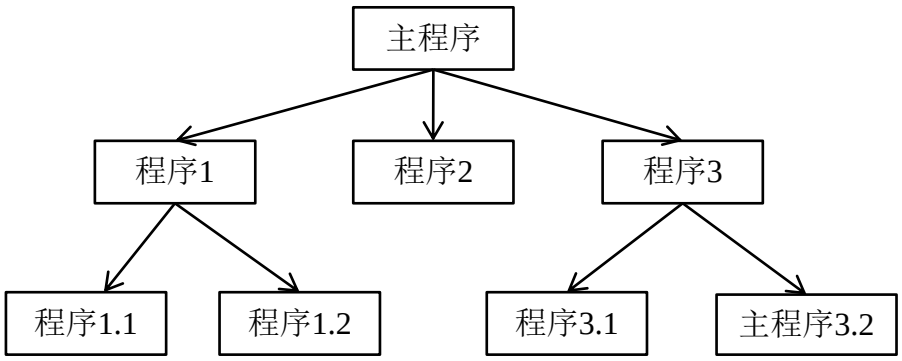


5.5 控制模型

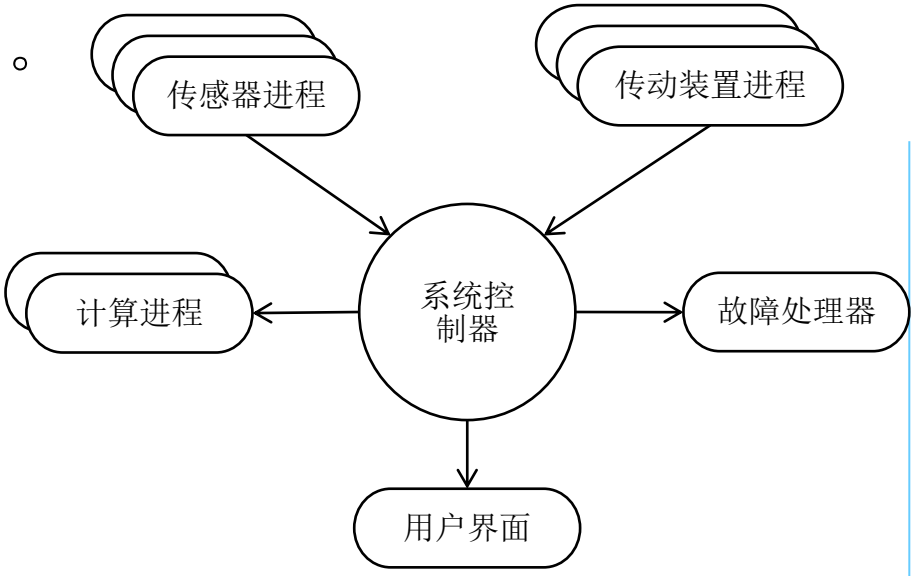
- 在体系结构层次上的控制模型关心的是子系统之间的控制流。
- 控制建模有2个一般方法：
  ① **集中式控制：** 一个子系统专门负责控制，控制其它子系统的启动和停止；它也可能将控制交给另一个子系统，但是在控制完成后，控制权仍要归还给它。
  ② **事件驱动系统：** 每个子系统都接受来自外部子系统的事件并对此做出响应。事件可能 来自其它子系统或来自系统的环境。

集中式控制

- 根据子系统的顺序是否并发执行而分为2类：
  - ① 调用-返回模型：自上而下的子过程模型
  - ② 管理者模型 ：适用于并发系统的模型。
- 优点：容易分析控制流，从而找出对特别的输入系统将会做什么响应。
- 缺点：对异常处理显得笨拙。



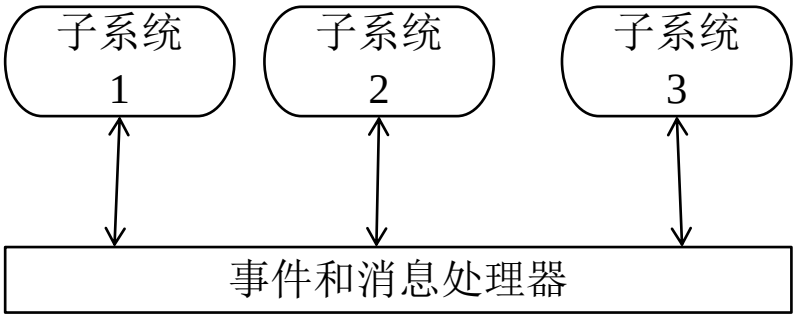
控制的调用-返回模型



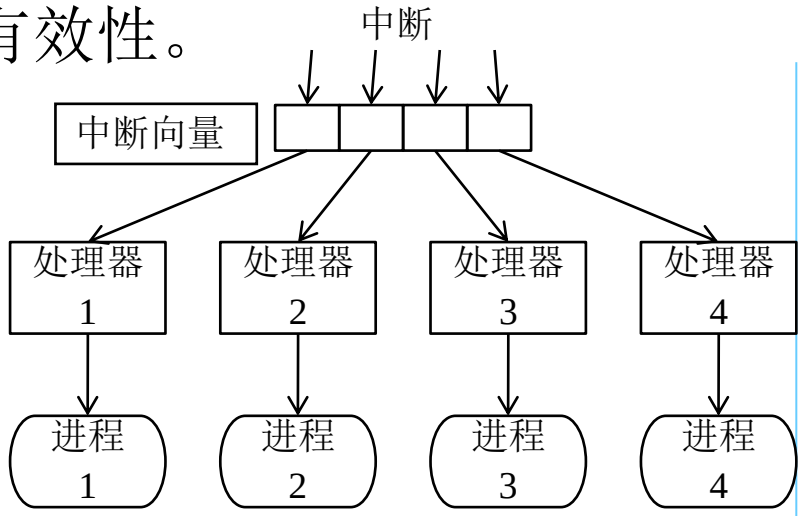
管理者模型

事件驱动系统

- 讨论2种事件驱动系统：
  - ① 广播模型：事件广播到所有子系统，任何能处理该事件的子系统都会响应。
  - ② 中断驱动模型：专用于实时系统中，由中断处理器对来自外部的中断进行检测，然后在其它组件中处理这些中断。
- 优点：能够迅速响应事件。
- 缺点：编程复杂，且不易验证有效性。



基于选择广播的控制模型



中断驱动控制模型

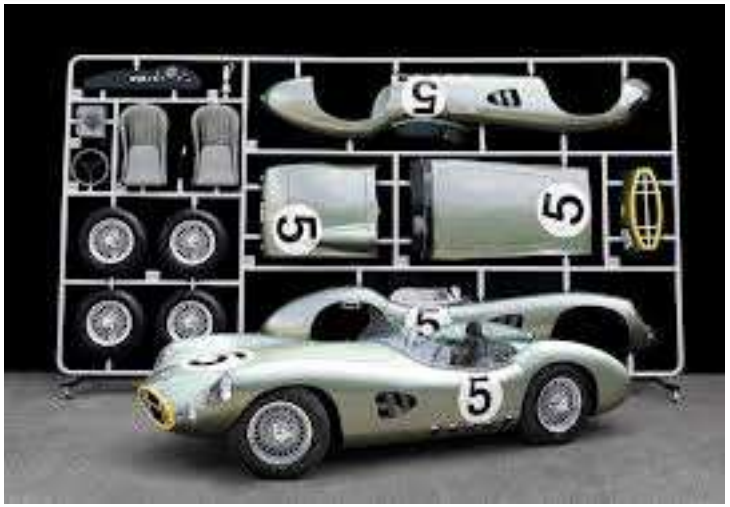
软件开发

6 结构化设计方法

6 结构化设计方法的主要内容

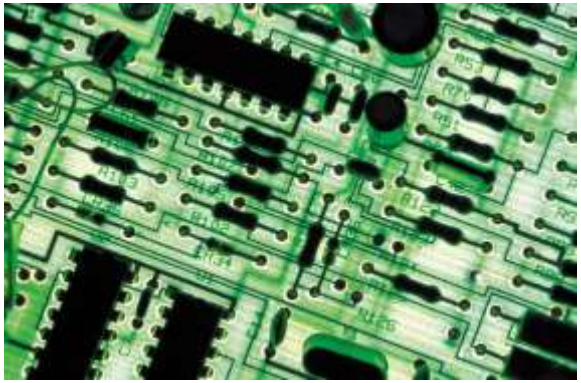
- 自顶向下，逐步细化
- 软件结构
- 程序结构
- 数据结构
- 软件过程
- 模块化
- 抽象化
- 信息隐蔽

结构化设计方法（Structure Design，SD）是基于模块化、自顶向下逐层细化、结构化程序设计等程序设计技术基础上发展起来的。



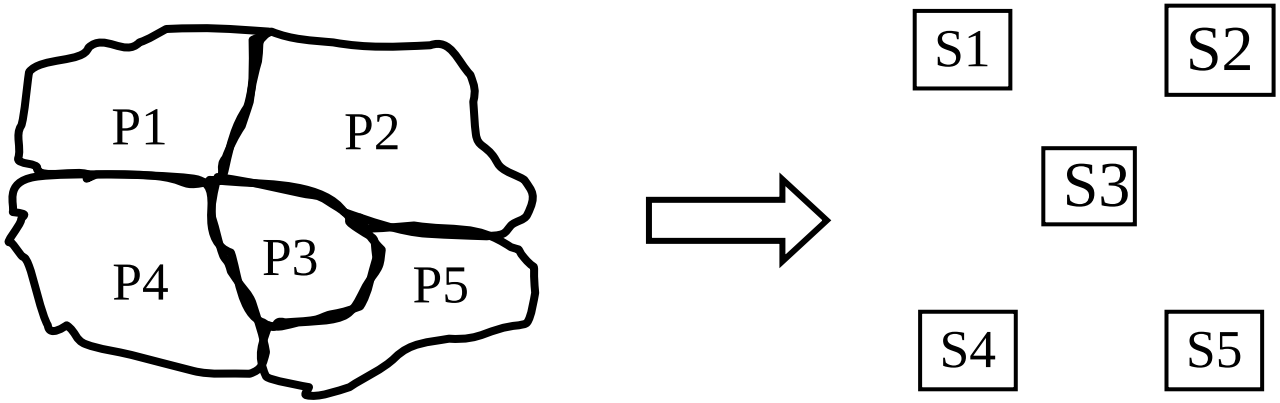
6.1 自顶向下，逐步细化

- 将软件的结构按自顶向下方式，对各个层次的过程细节和数据细节逐层细化，直到用程序设计语言的语句能够实现为止，从而最后确立整个的体系结构。
- 自顶向下是一个反复推敲的过程。
- 由抽象到具体。



6.2 软件结构

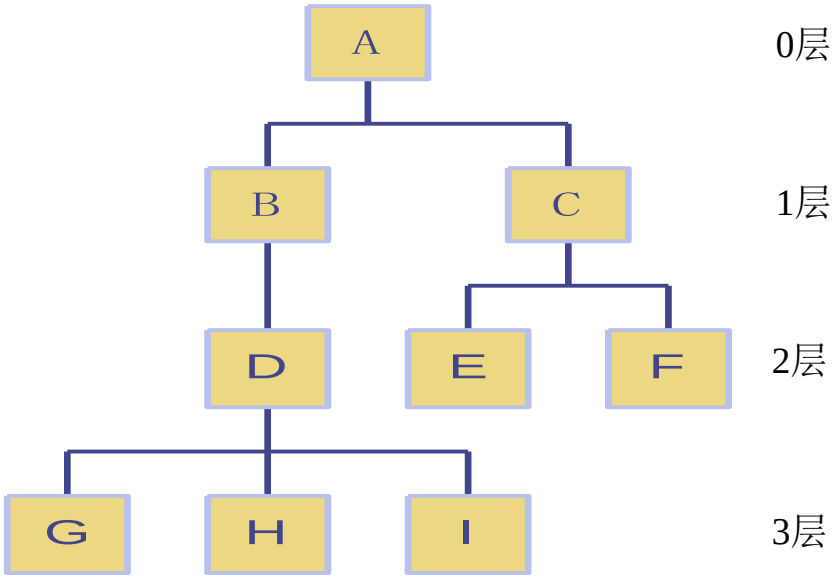
- 软件的结构包括2部分：
  - 程序模块的层次结构
  - 数据的结构
- 对于同一组软件需求，用不同的软件设计方法，可能得到不同的软件结构。



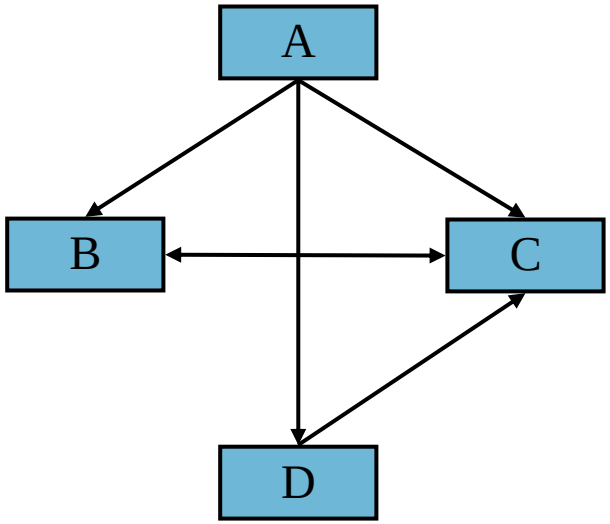
软件结构的形成

6.3 程序结构

- 程序结构表明了程序各个成份（模块）的组织情况，它通常是树状结构或网状结构。



树状结构

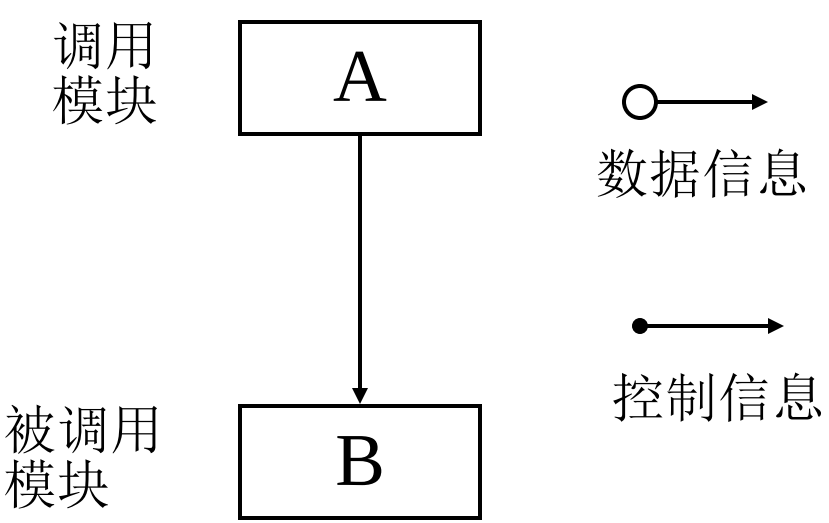


网状结构

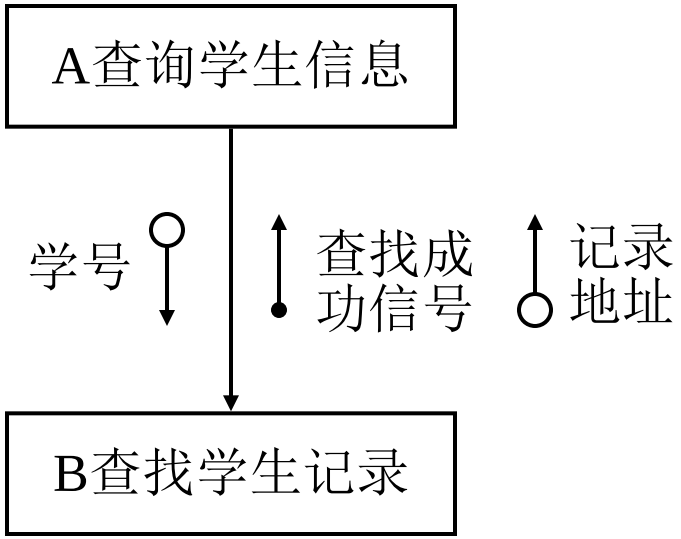
结构图

- 程序结构图：
  - 清楚地反映出程序中模块间的层次调用关系和联系
  - 定义了各个模块的名字、功能和接口
  - 反映了设计思想
- 结构图的形态特征
  - **结构图的深度：** 在多层次的结构图中，其模块结构的层次称为结构图的深度。
  - **结构图的宽度：** 结构图中同一层模块的最大模块数称为结构图的宽度。
  - **模块的扇入：** 调用一个给定模块的模块个数。
  - **模块的扇出：** 扇出表示一个模块直接调用的其它模块数目。

模块间的调用关系和接口表示

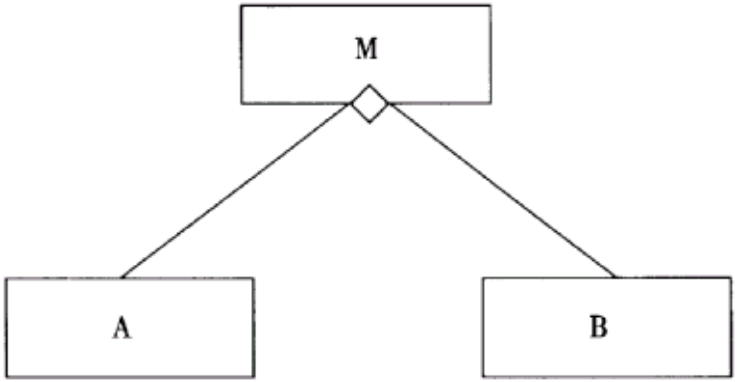


模块间调用关系

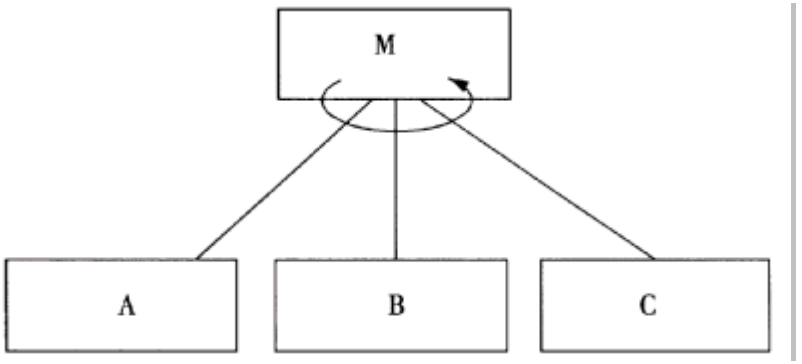


模块间接口的表示

条件调用和循环调用表示



条件调用



循环调用

6.4 数据结构

- 数据结构是数据的各个元素之间的逻辑关系的一种表示。
- 数据结构影响过程设计，它与过程结构同等重要。
- 数据结构可以在不同的抽象层次上表示。
- 数据结构设计的内容：
  - 数据组织
  - 存取方式
  - 相关程序
  - 处理方法

6.5 软件过程

- 软件过程设计：描述模块的处理细节，提供精确的处理说明，包括事件的顺序、正确的判定点、重复的操作、数据的组织和结构等。
- **模块最重要特征：抽象和信息隐藏。**
- 通过抽象，可以确定组成软件的过程（信息）实体；通过信息隐蔽，可以定义和实施对模块的过程细节和局部数据结构的限制。

模块化设计

- 将子系统分解为模块时有2种模型：
  ① 面向对象
  ② 数据流模型
- 模块类似于高级语言中的：
  - procedure　　过程
  - function　　函数
  - subroutine　　子程序
  - section　　程序段
  - block　　程序块

模块（Module）

- 用一个名字就可调用的一段程序称为模块。
- 模块具有3个基本属性：
  - 功能：要完成的工作
  - 逻辑：描述模块内部的执行过程
  - 状态：使用模块时的环境和条件。
- 模块特性：
  - 外部特性：指模块的名、参数表，以及对程序和整个系统造成的影响。
  - 内部特性：程序的功能代码和模块内部使用的函数。

模块个数与工作量间的关系

- 设：
  - *C(X)* 是确定问题*X*的复杂程度函数；
  - *E(X)* 是决定解决问题*X*所需要工作量的函数。
- 对于两个问题P_1和P_2，如果

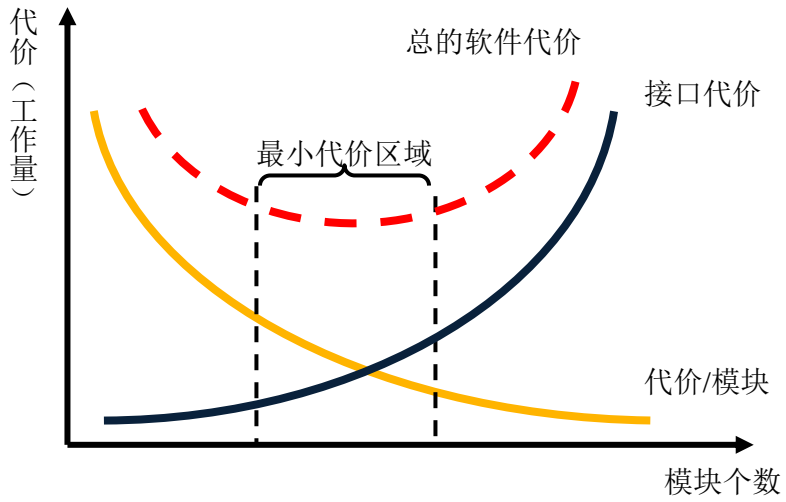
  $C(P_1)>C(P_2)$

  则有

  $E(P_1)>E(P_2)$
- 另一个规律

  $C(P_1+P_2)>C(P_1)+C(P_2)$

  则有 $E(P_1+P_2)>E(P_1)+E(P_2)$



模块独立性（Module Independence）

- 模块独立性是模块抽象和信息隐藏的直接结果。
- 模块独立性是软件质量好坏的关键。
- 度量模块独立性的2个定性的标准：
  - 耦合：指模块间相互关联的程度。
  - 内聚：指一个模块内部各个元素之间关系的紧密程度。
- 模块独立性的基本原则：
  - 为得到好的模块结构，理想的情况是块间联系尽可能少，块内联系尽可能大。
  - 块间联系和块内联系是同一事物的两个方面。

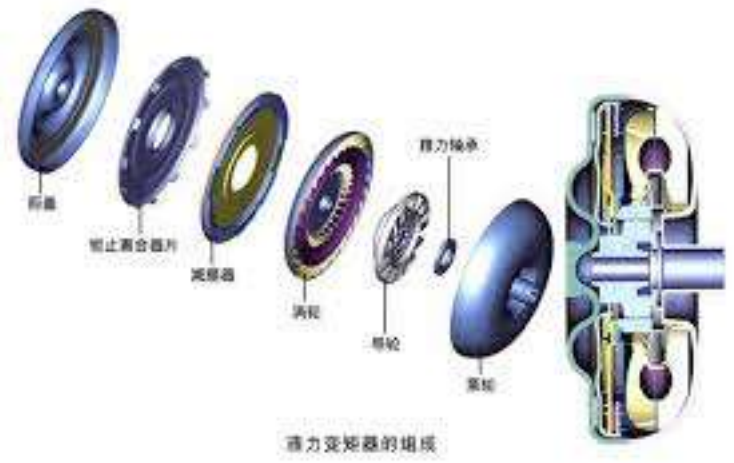
耦合性

耦合性（Coupling）是程序结构中各个模块之间相互关联的度量。它取决于各个模块之间接口的复杂程序、调用模块的方式以及哪些信息通过接口。

低 耦合性 高

- 非直接耦合
- 数据耦合
- 标记耦合
- 控制耦合
- 外部耦合
- 公共耦合
- 内容耦合

强 模块独立性 弱



模块之间的连接方式

降低模块间耦合度的方法：

- 根据问题的特点，选择适当的耦合类型
  - 传递数据信息比传递控制信息的耦合度低。
  - 不能盲目地追求松散耦合。
- 降低模块间接口的复杂性
  - 接口复杂性包括3个因素：传送的信息量、联系方式、传送数据的结构。
- 把模块的通信信息放在缓冲区中。
  - 把缓冲区作为每次通信流的媒介，可以减少模块间在执行速度和频率方面的相互影响。

内聚性（Cohesion）

- 内聚标志一个模块内各个元素彼此结合的紧密程度，它是信息隐蔽和局部化概念的自然扩展。简单地说，理想内聚的模块只做一件事情。
- 设计时应该力求做到高内聚，通常中等程度的内聚也是可以采用的，而且效果和高内聚相差不多；但是，低内聚很坏，不要使用。

高 内聚性 低

- 功能内聚
- 信息内聚
- 通信内聚
- 过程内聚
- 时间内聚
- 逻辑内聚
- 巧合内聚

强 模块独立性 弱

功能单一

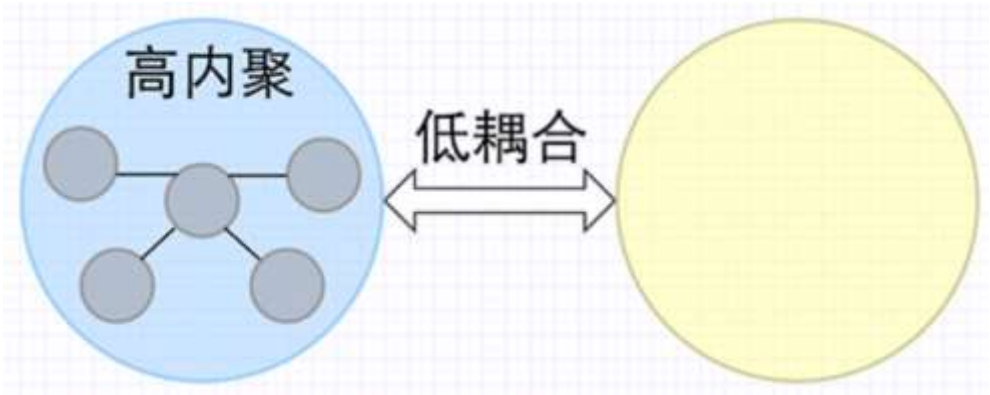
功能分散

模块内聚性的7个类型



设计原则

尽量使用数据耦合，少用控制耦合，限制公共环境耦合的范围，完全不用内容耦合。



6.6 抽象化

- 抽象就是将事务的相似方面集中和概括起来，暂时忽略它们之间的差异。或者说，抽象是抽象事务的本质特性而暂时不考虑它们的细节。
- 在软件设计时，可以有不同的抽象层次：
  - 在最高的抽象层次上，可使用自然语言描述问题。
  - 较低的抽象层次上，采用过程化方法描述问题解决方法。
  - 最低层的抽象层次上，使用能够直接实现的方式来描述解决问题的方法。

过程的抽象（1/3）

- 抽象层次I：用问题所处环境的术语来描述这个软件。

- 例：

  一个CAD软件包括一个计算机绘图界面，向绘图员显示图形，以及一个数字化仪界面，用以代替绘图板和丁字尺。所有直线、折线、矩形、圆及曲线的描画、所有的几何计算、所有的剖面图和辅助图都可以用这个CAD软件实现。

过程的抽象（2/3）

- 抽象层次II：任务需求描述。

- 例：

CAD SOFTWARE TASK:
    user interaction task;
    2-D drawing creation task;
    graphics display task;
    drawing file management task;
END.

过程的抽象（3/3）

- 抽象层次III．程序过程表示。

- 例：以2-D绘图生成任务为例。

```
PROCEDURE: 2-D drawing creation
    REPEAT UNTIL (drawing creation task terminates)
        DO WHILE (digitizer interaction occurs)
          digitizer interface task;
        DETERMINE drawing request CASE;
           line: line drawing task;
        rectangle: rectangle drawing task;
        circle: circle drawing task;
        ……
     END;
    DO WHILE (keyboard interaction occurs)
          keyboard interaction task;
       PROCESS analysis/computation CASE;
            view: auxiliary view task;
            section: cross sectioning task;
            ……
          END;
          ……
   END REPETITION;
END PROCEDURE.
```

数据抽象

- 数据抽象在不同层次上描述对象的细节，可以定义与该数据对象相关的操作来规定数据对象。
- 例：

```
TYPE drawing IS STRUCTURE DEFIND
    number IS STRING LENGTH(12);
    geometry DEFIND……
    notes IS STRING LENGTH(256);
    BOM DEFIND
END drawing TYPE;
```

6.7 信息隐蔽

- 模块化原理的一个重要原则是信息尽量隐蔽在模块中。
- 信息隐蔽的核心内容是：一个模块中所包含的信息（包括数据和过程），不允许其它不需要这些信息的模块访问。
- 信息隐蔽就意味着对程序处理细节的隐蔽，有助于提高模块的独立性。

软件开发

7 结构化分析

主要内容

- 系统结构图
- 变换分析
- 事务分析
- 数据设计

SD方法的要点：

1. 研究、分析和审查数据流图。
2. 根据数据流图决定问题的类型。
3. 由数据流程图导出初始的系统结构图。
4. 利用一些试探性原则来改进系统的初始结构图，直到得到符合要求的结构图为止。
5. 修改和补充数据词典。
6. 制定测试计划。

7.1 典型的系统结构图

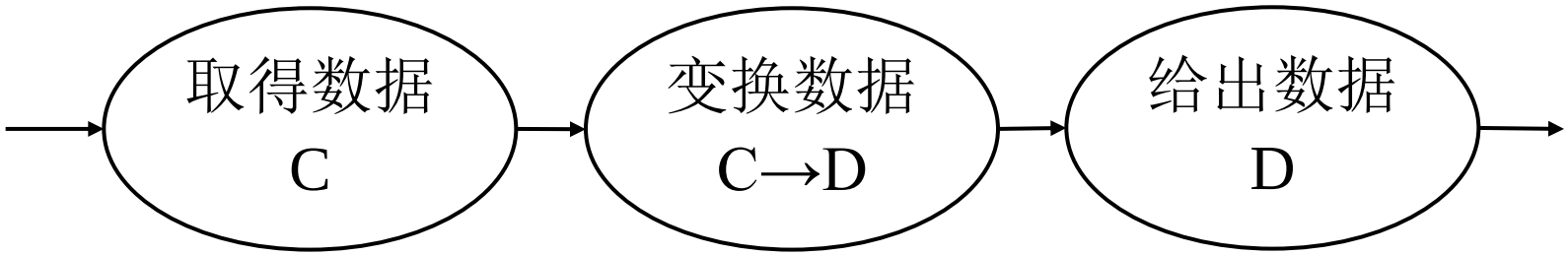
- **原子模块**：在系统结构图中，不能再分解的底层模块。
- **完全因子分解的系统**：如果一个软件系统的全部实际加工都由原子模块来完成，而其它所有非原子模块仅仅执行控制或协调功能。
- 如果系统结构图是完全因子分解的，就是最好的系统。

在系统结构图中有4种类型的模块：

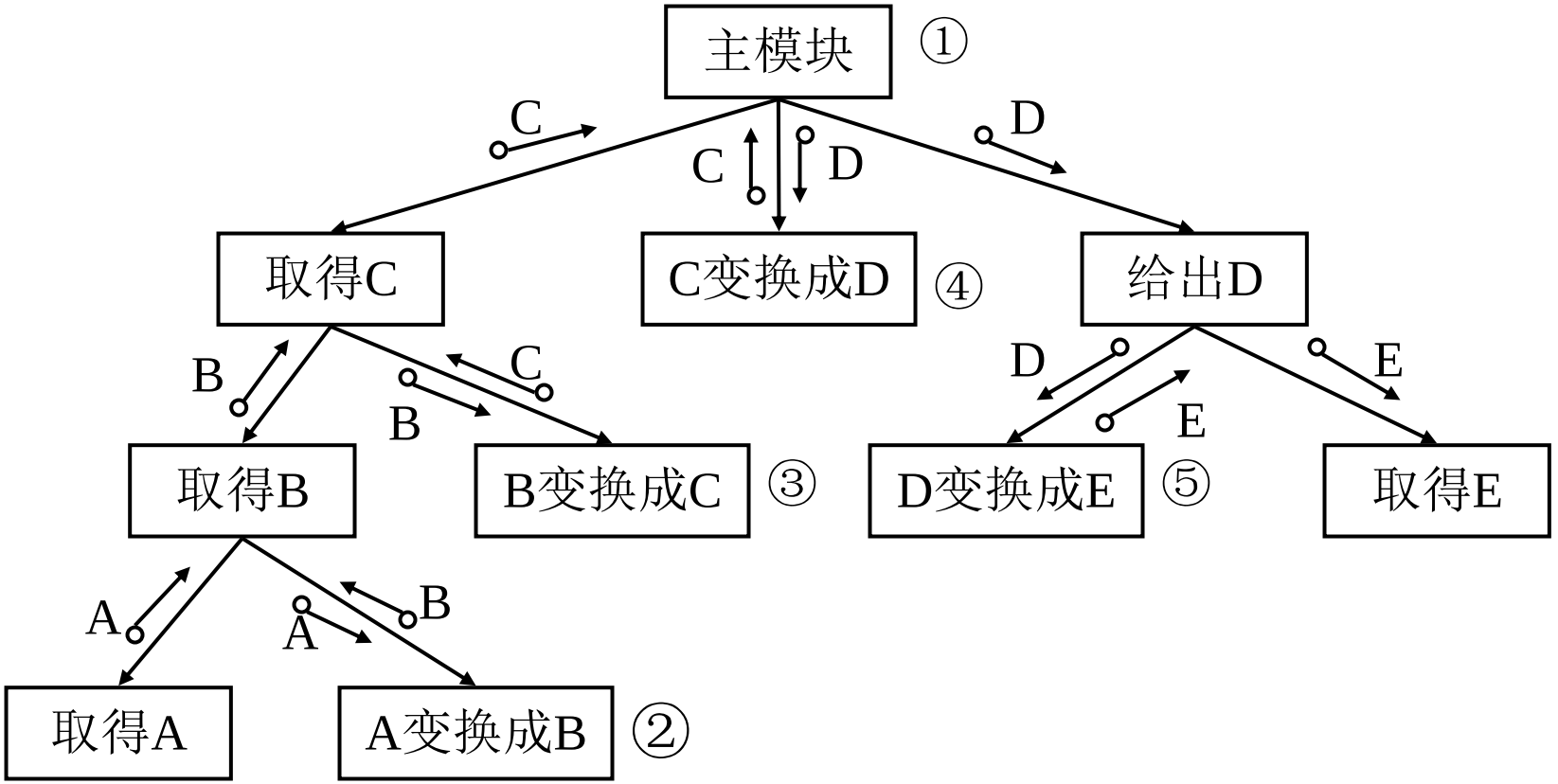
- 传入模块
  - 从下属模块取得数据，经过某些处理，再将其传送给上级模块。
- 传出模块
  - 从上级模块获得数据，进行某些处理，再将其传送给下属模块。
- 变更模块
  - 它从上级模块取得数据，进行特定的处理，转换成其它形式，再传送回上级模块。
- 协调模块
  - 对所有下属模块进行协助调和管理的模块。

变换型系统结构图

- **变换型数据流图**呈一种线性状态，信息沿输入通路进入系统，同时由外部形式变换成内部形式，进入系统的信息通过变换中心，经加工处理以后再沿输出通路变换成外部形式离开软件系统。
- 工作过程：
  1. 取得数据
  2. 变换数据
  3. 给出数据



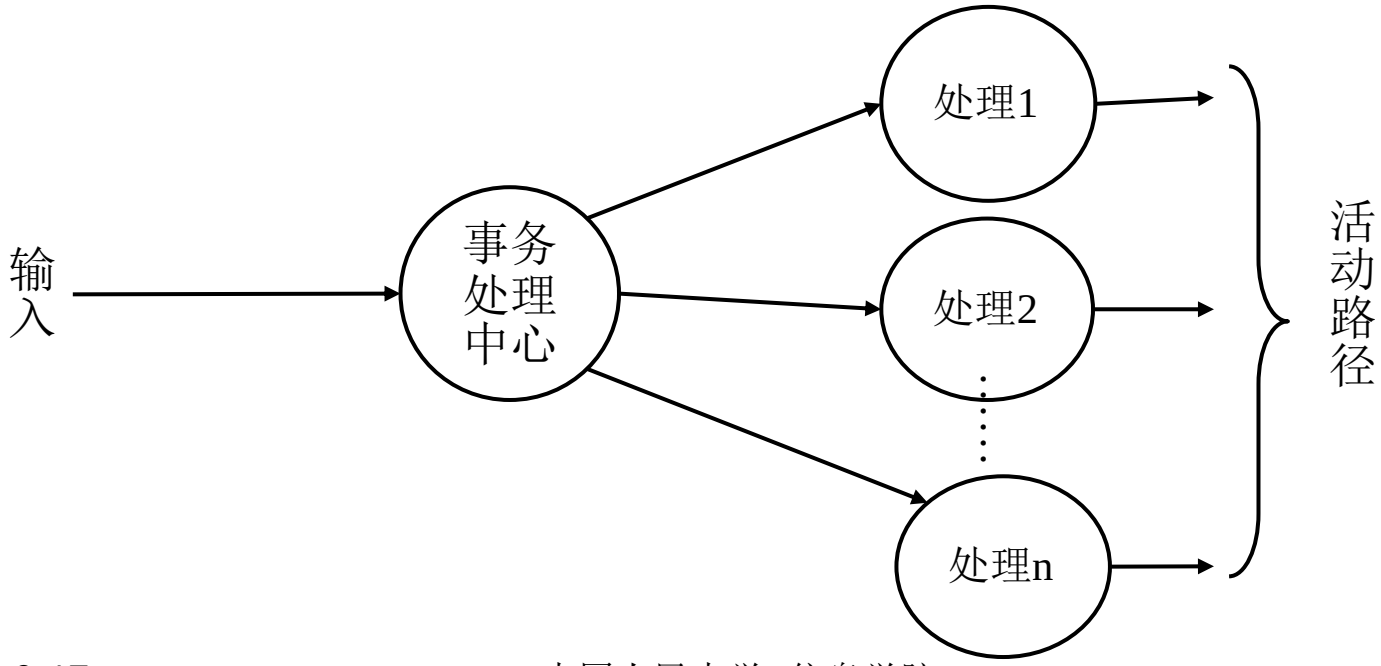
变换型的系统结构图



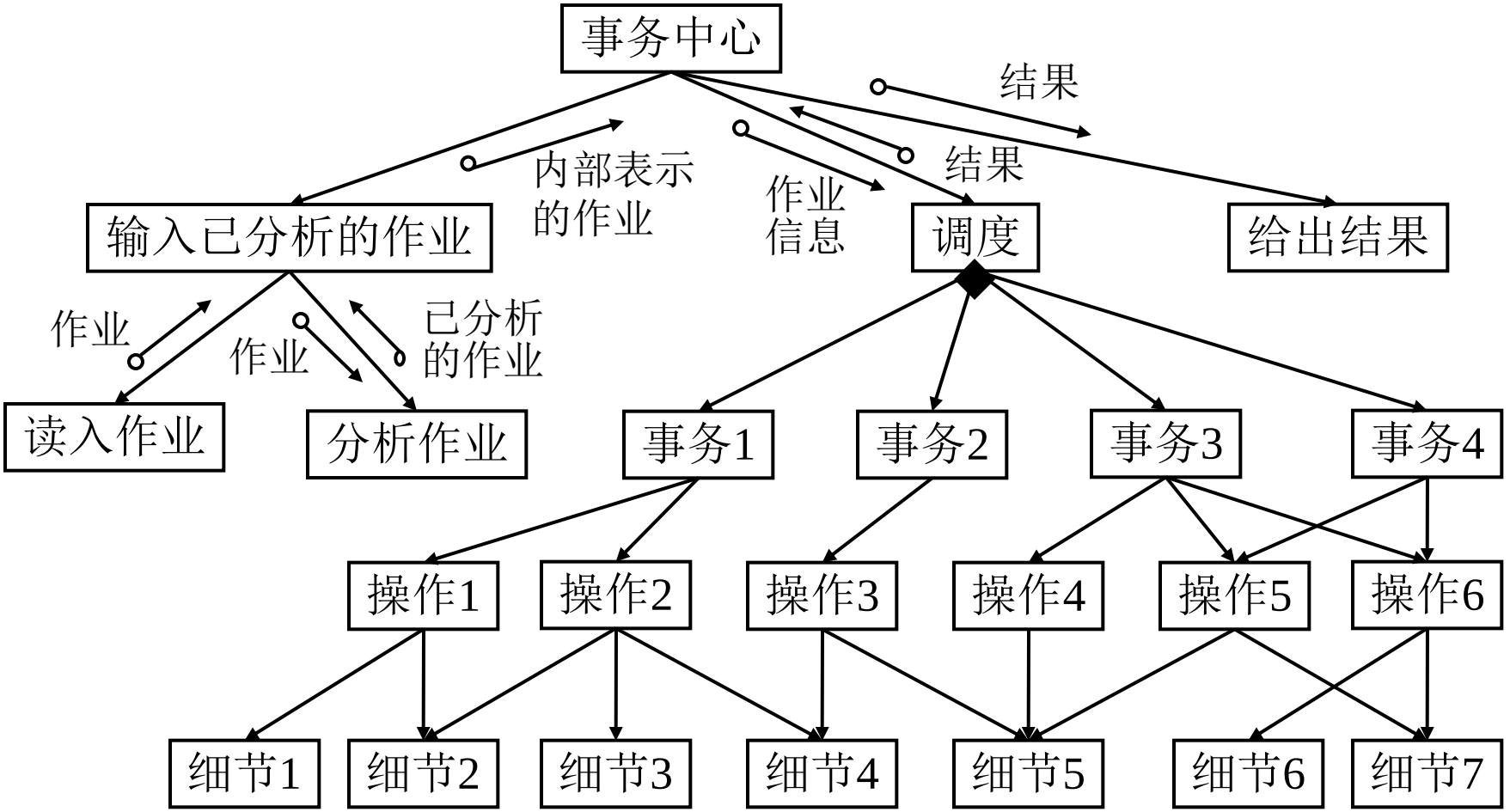
变换过程：$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

事务型系统结构图

- 某个加工将它的输入分离成一串发散的数据流，形成许多活动路径，并根据输入的值选择其中一条路径。具有这样特征的数据流图称为**事务型数据流图**。
- 通常把完成选择分派任务的部分叫做**事务处理中心**。



事务型系统结构图


事务中心
结果
内部表示
的作业
作业
信息
结果
输入已分析的作业
调度
给出结果
作业
作业
已分析
的作业
读入作业
分析作业
事务1
事务2
事务3
事务4
操作1
操作2
操作3
操作4
操作5
操作6
细节1
细节2
细节3
细节4
细节5
细节6
细节7

7.2 变换分析设计方法

- 变换分析方法步骤：
  1. 重画数据流图
  2. 区分有效（逻辑）输入、有效（逻辑）输出和中心变换部分
  3. 进行一级分解，设计上层模块
  4. 进行二级分解，设计中、下层模块

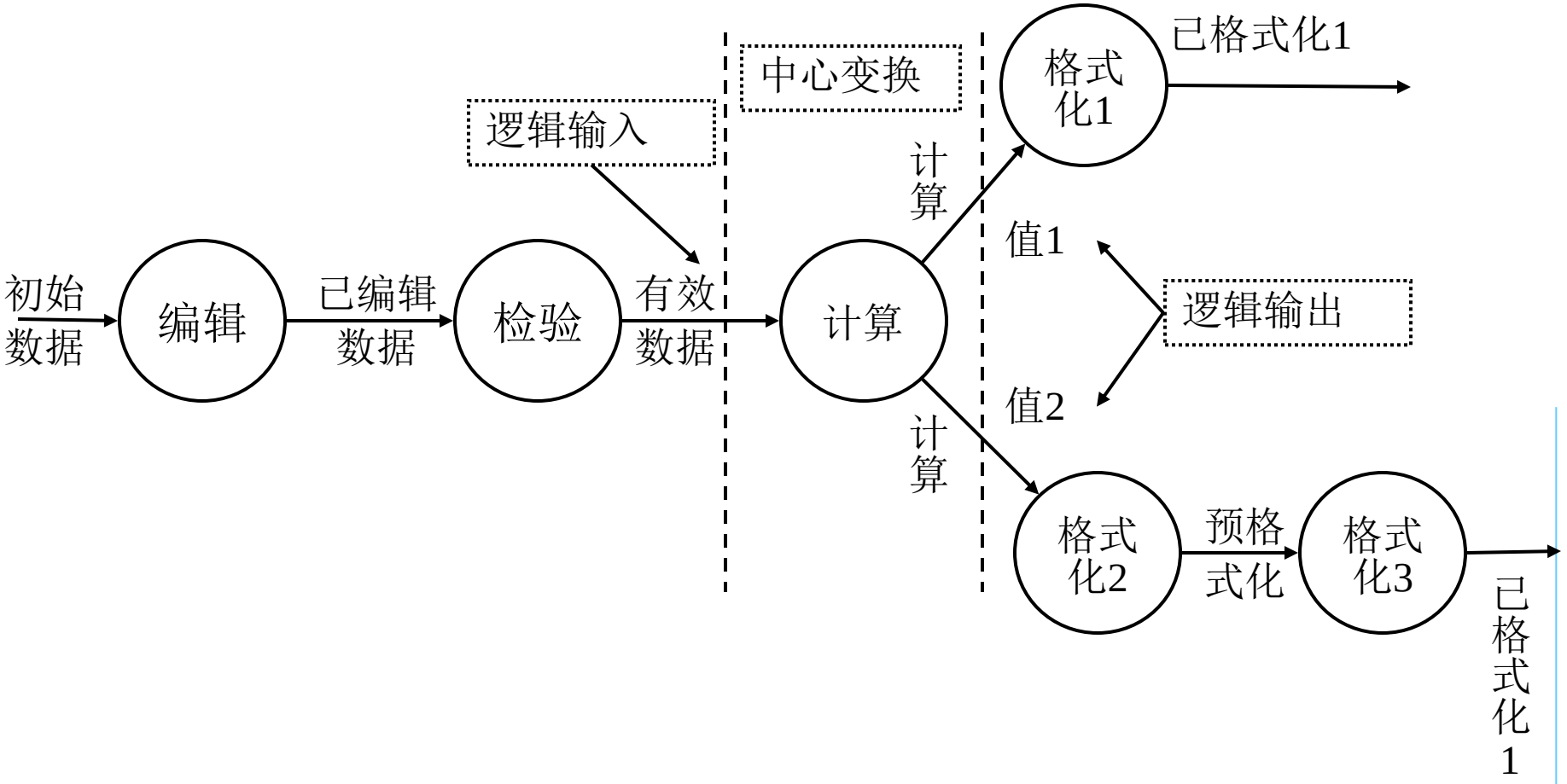
（1）重画数据流图

- 要点：
  - 以需求分析阶段得到的数据流图为基础重画数据流图时，可以从物理输入到物理输出，或者相反，还可以从项层加工框开始，逐层向下。
  - 在图上不要出现控制逻辑（如判定、循环）。用箭头表示的是数据流，不是控制流。
  - 不要去管理系统的开始和终止。
  - 当数据流进入和离开一个加工时，要仔细地标记它们，不要重名。
  - 如有必要，可以使用逻辑运算符*和⊕。

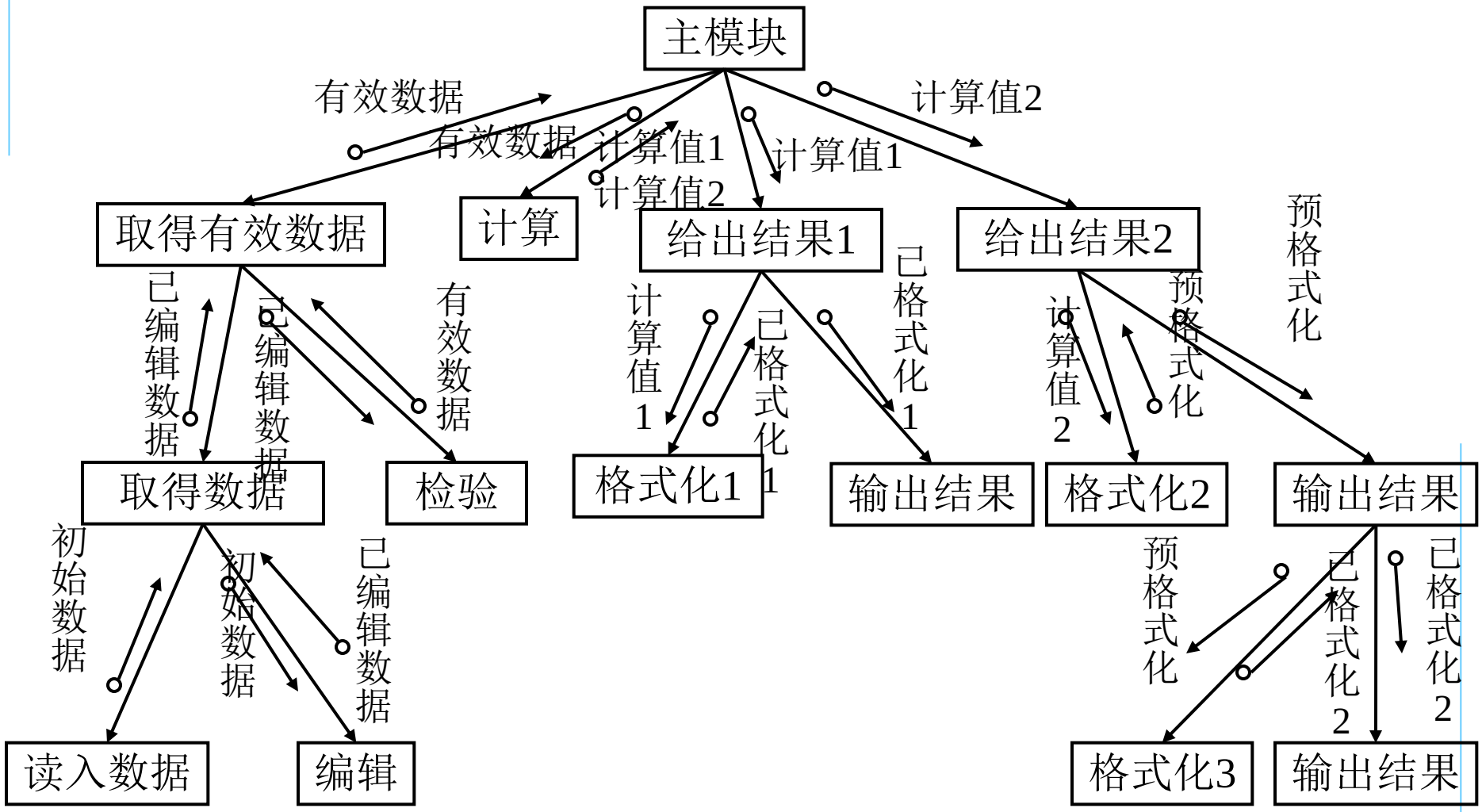
（2）在数据流图上区分系统逻辑输入、逻辑输出和中心变换部分

- 逻辑输入就是离物理输入端最远的，但仍被看作是系统输入的数据流。
- 逻辑输出就是离物理输出端最远的，但仍被看用是系统输出的数据流。
- 从物理输入端到逻辑输入，构成系统的输入部分；从物理输出端到逻辑输出，构成输出部分；夹在输入部分和输出部分之间的就是中心变换部分。
- 有些系统仅仅有输入和输出部分没有中心变换部分。

变换型问题的数据流图


初始
数据
编辑
已编辑
数据
检验
有效
数据
逻辑输入
中心变换
计算
计算
计算
值1
值2
逻辑输出
格式
化1
已格式化1
格式
化2
预格
式化
格式
化3
已格式化1

变换型问题的数据流图导出结构图



（续）

（3）进行一级分解，设计系统模块结构的顶层和第一层

（4）进行二层分解，设计输入、中心变换、输出部分的中心、下层模块

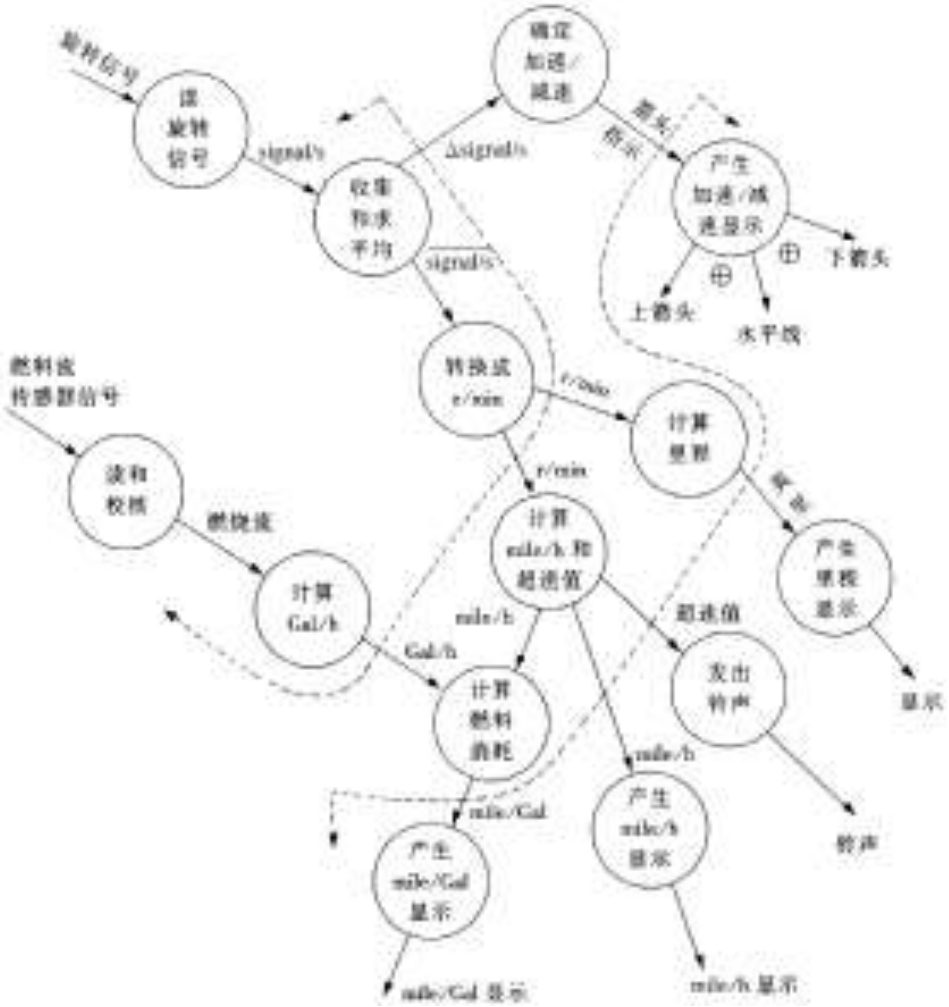
例：微处理器汽车数字控制系统（1/8）

- 变换设计示例
- 系统的功能：
  ① 通过数字转换，实现传感器与微处理器的接口。
  ② 用发光二极管在面板上显示各种数据。显示的数据有：每小时时英里数（mph），里程数（m），每加仑里数（mpg），每里的费用，旅行的费用及功能选择等 。
  ③ “超速”指示，如果速度超过55mph，发出与时速和55mph之差同步的蜂鸣声。

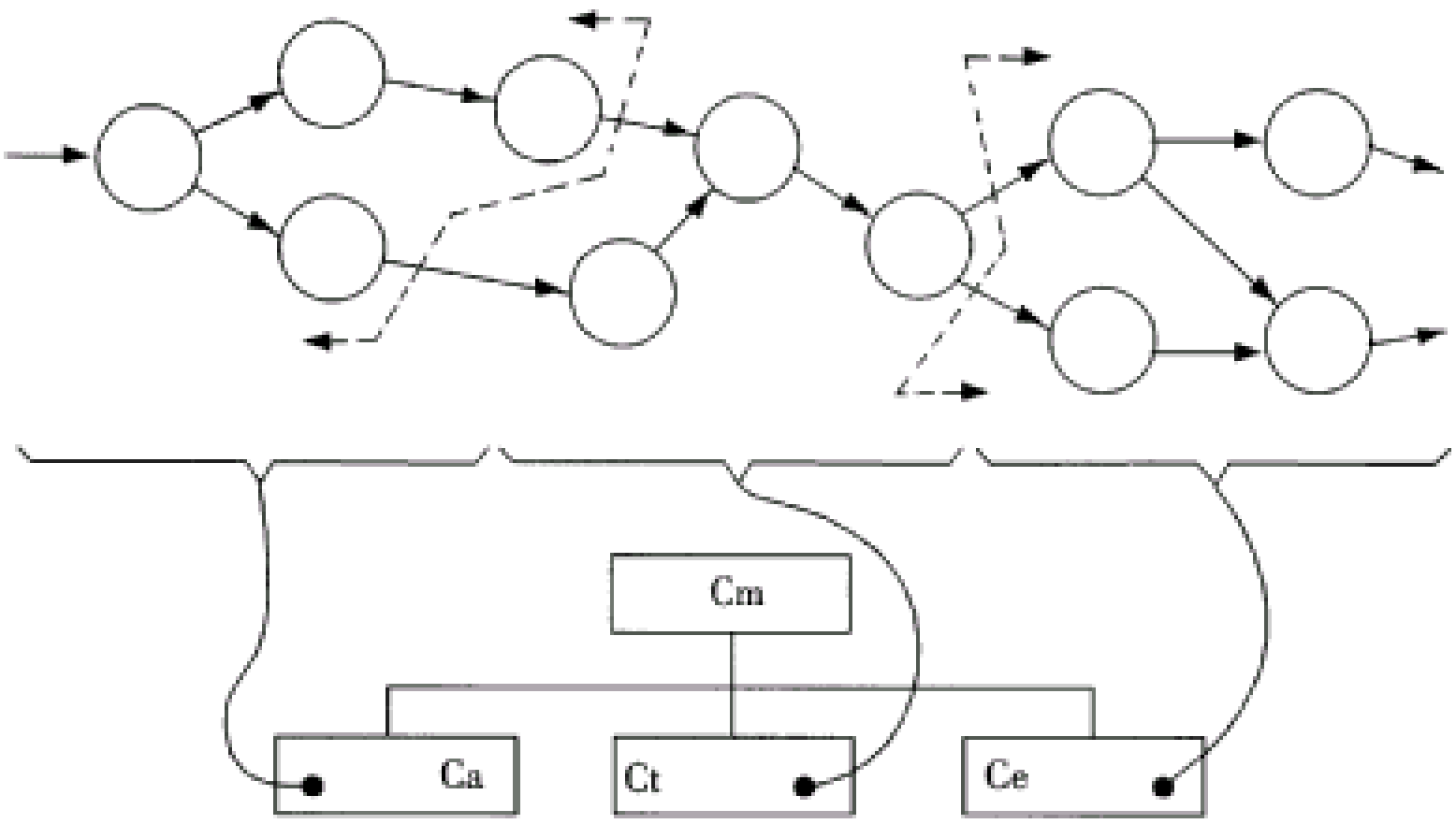
数字仪表板系统的数据流图(2/8)



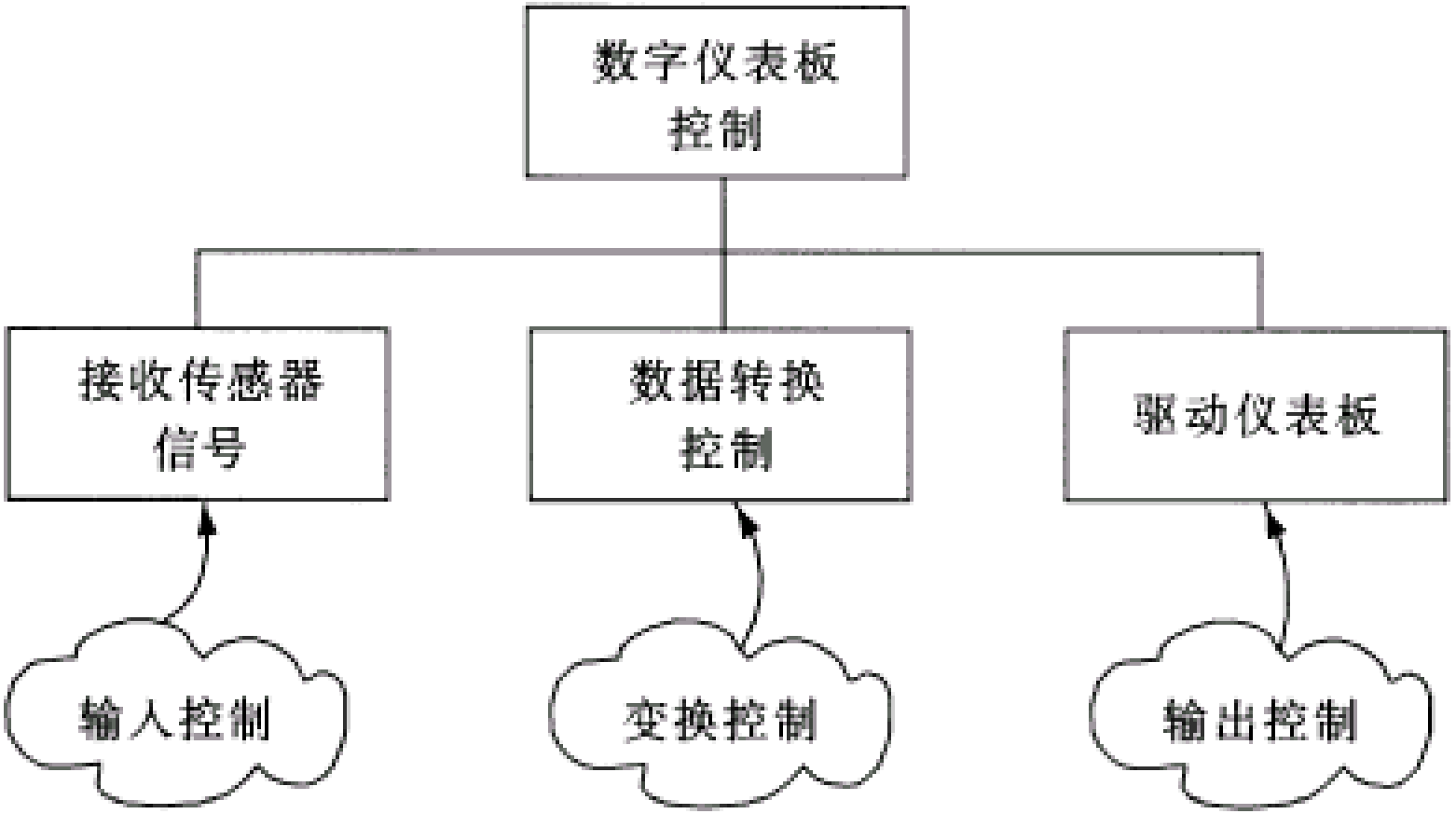
具有边界的数据流图(3/8)



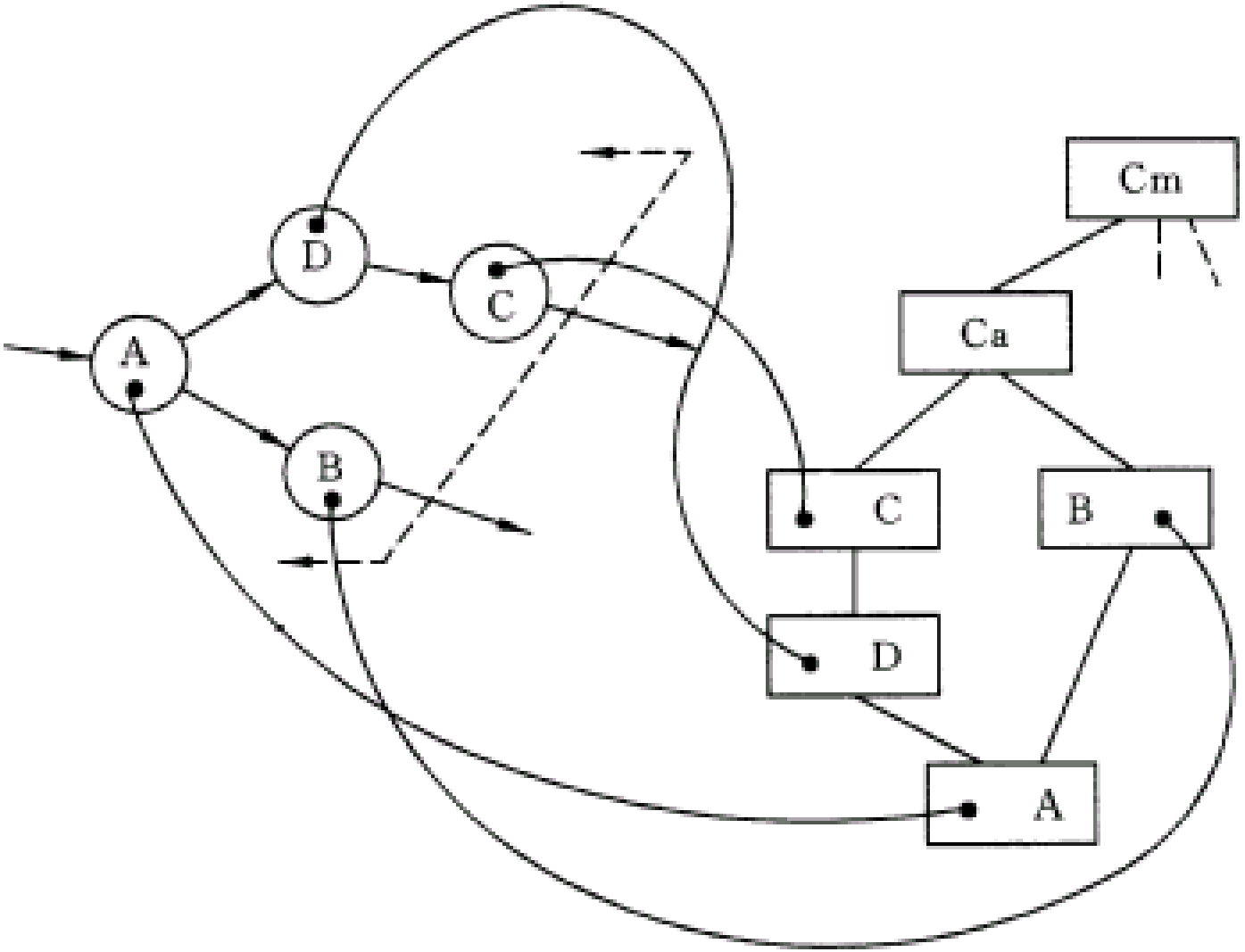
第一级分解的方法(4/8)



数字仪表板系统的第一级分解(5/8)



第二级分解的方法(6/8)



未经精化的输入结构(7/8)

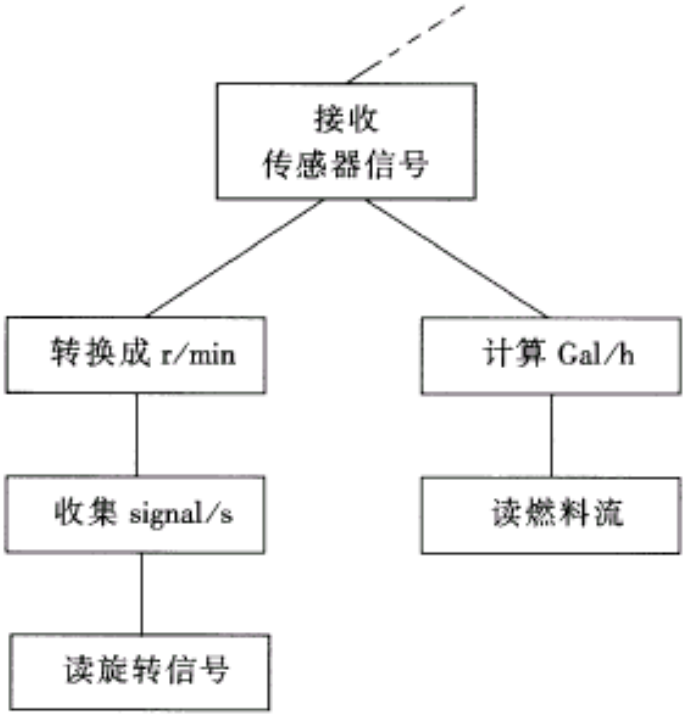


图 a

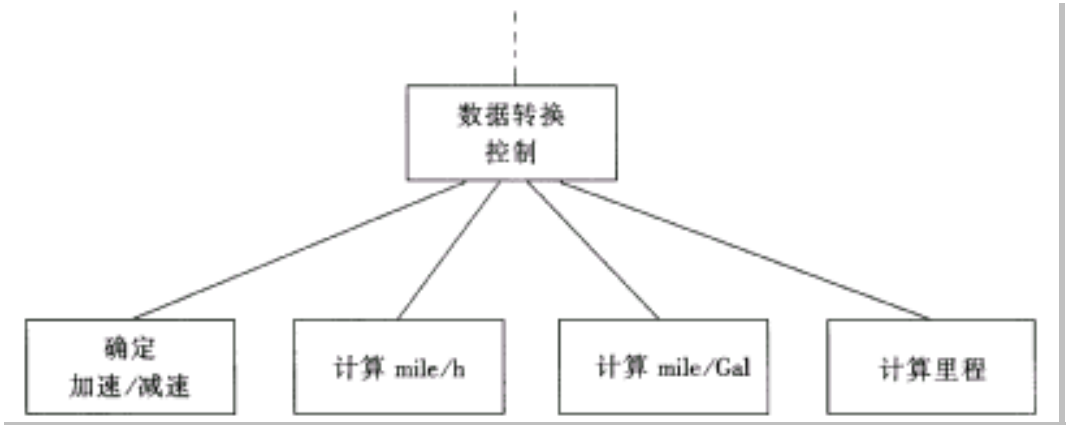
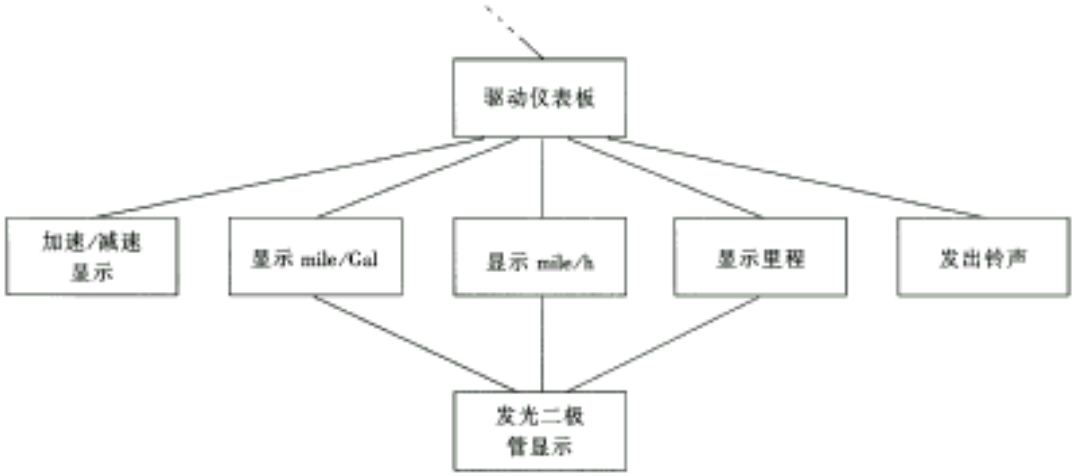
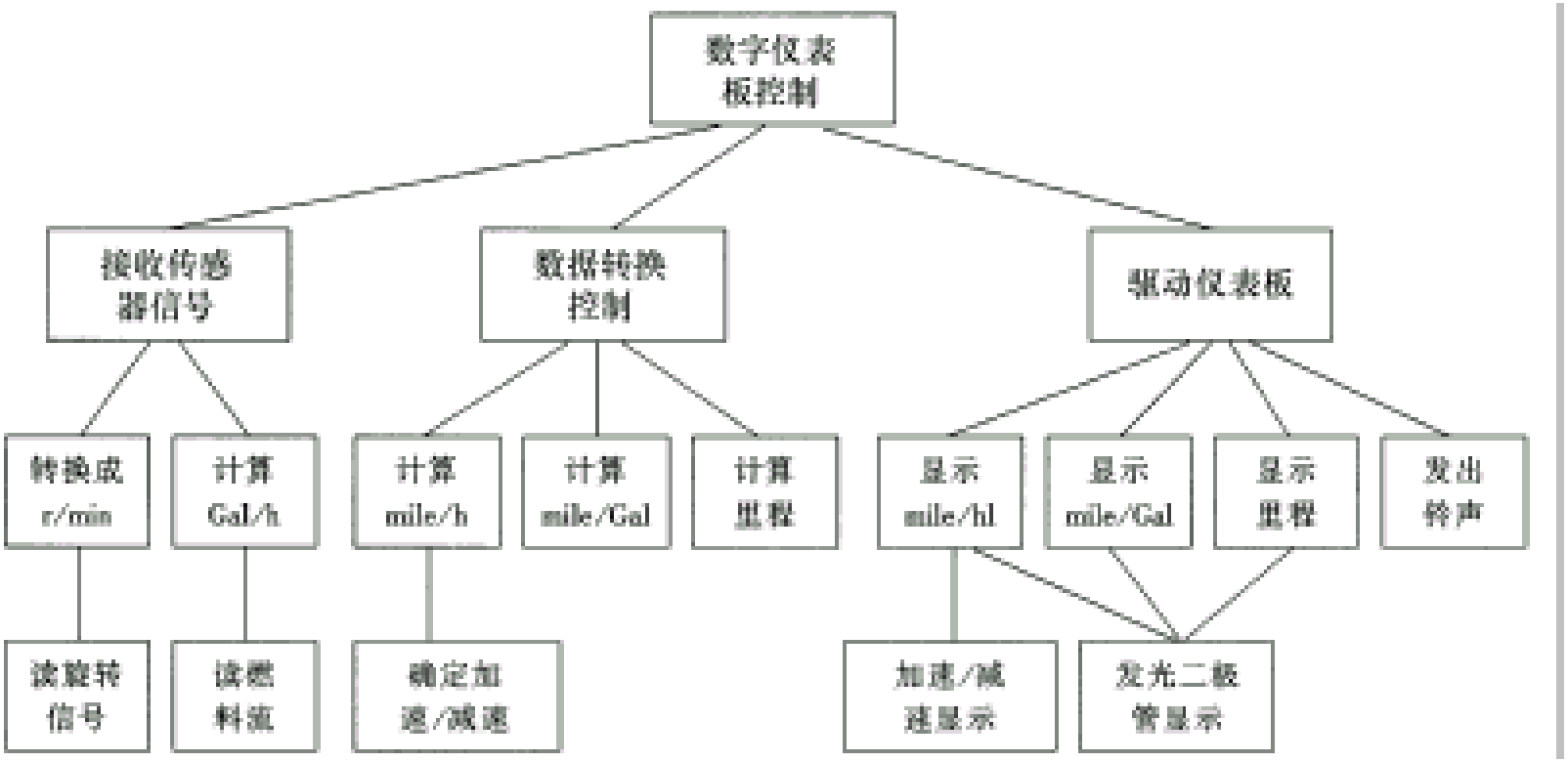


图 b



图c

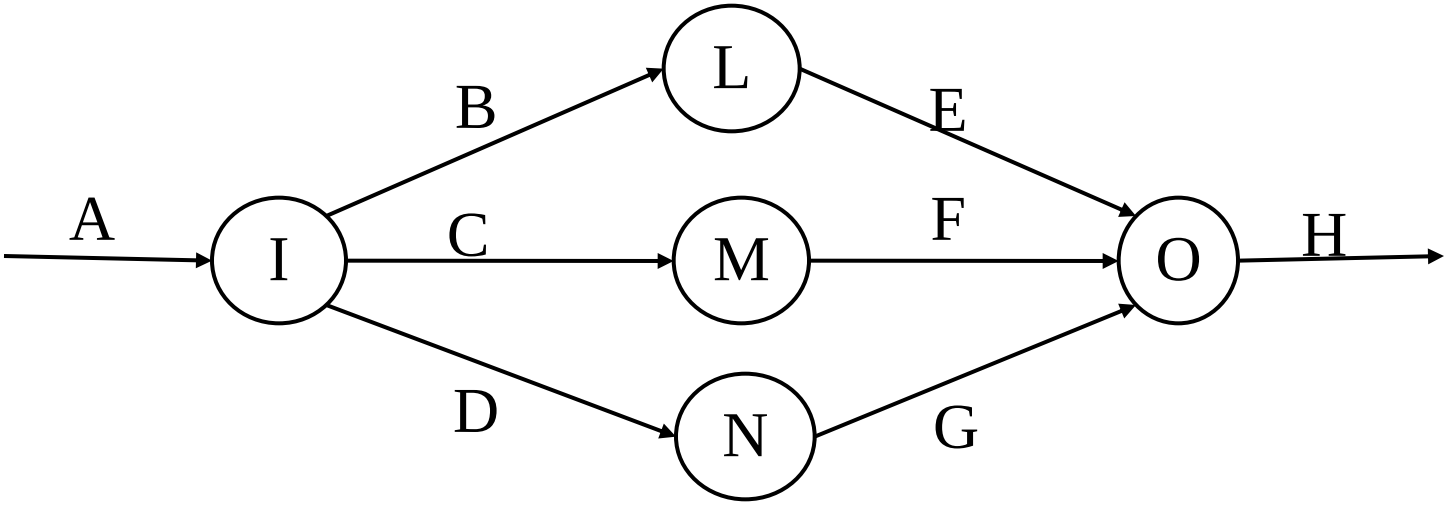
精化后的数字仪表板系统的软件结构(8/8)

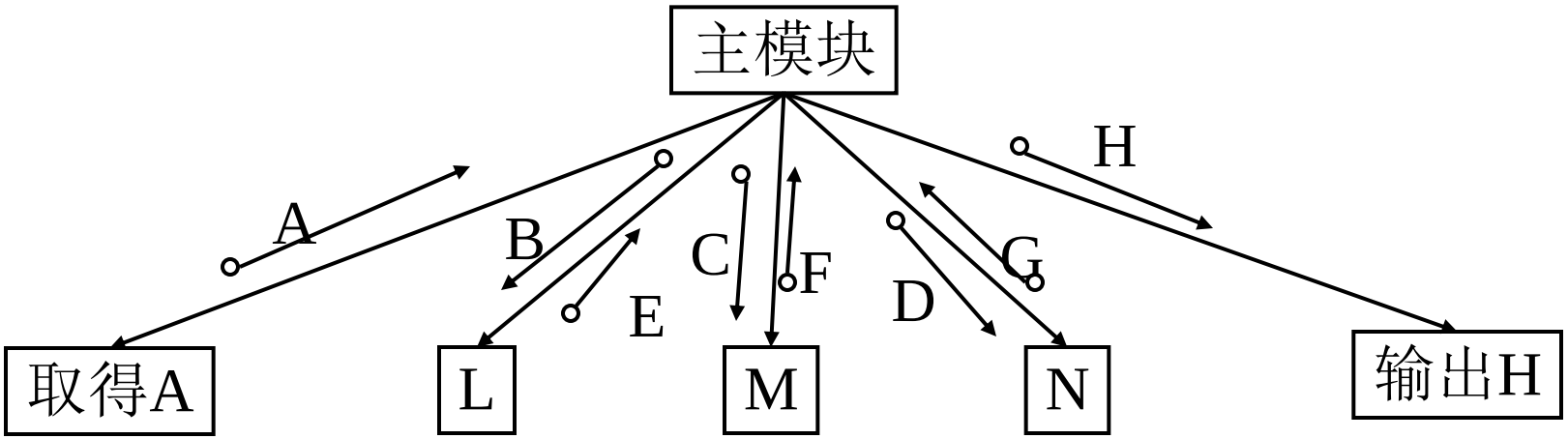


7.3 事务分析

- 事务分析从数据流图开始，自顶向下，逐步分解，建立到结构图。
- 事务分析方法的步骤：
  1. 识别事务源。
  2. 规定适当的事务型结构。
  3. 识别各种事务和它们定义的操作。
  4. 注意利用公用模块。
  5. 对每一事务，或对联系密切的一组事务，建立一个事务处理模块。
  6. 对事务处理模块规定它们全部的下层操作模块。
  7. 对操作模块规定它们的全部细节。

事务型问题导出的系统结构图





设计准则

1) 评价软件的初始结构，通过模块的分解和合并，以求减少块间联系，增大块内联系；
2) 模块调用个数最好不要超过5个；
3) 一个模块的作用范围应处在这个模块的控制范围之内；
4) 降低模块接口的复杂性；
5) 力求设计单入口和单出口的模块，避免“病态连接”；
6) 模块的大小要适中。

7.4 软件模块结构的改进

1) 模块功能的完善。
2) 消除重复功能，改善软件结构。
3) 模块的作用范围应在控制范围之间。
4) 尽可能减少高扇出结构，随着深度增大扇入。
5) 模块大小要适中。
6) 设计功能可预测的模块，但避免过分受限制的模块。

7.5 设计的后处理

1) 处理说明
2) 接口说明
3) 数据结构说明
4) 设计评审
5) 设计优化



错误的扩大模型

7.6 数据设计及文件设计

- 数据结构对程序结构和过程复杂性的影响使得数据设计对软件质量产生不容忽视的影响。
- 数据设计原则：
  1) 用于功能和行为的系统分析原则也适用于数据。
  2) 应该标识所有的数据结构以及其上的操作。
  3) 应当建立数据字典，并用于数据设计和程序设计。
  4) 低层的设计决策应该推迟到设计过程的后期。
  5) 只有那些需要直接使用数据结构内部数据的模块才能看到该数据结构的表示。
  6) 应该开发一个由有用的数据结构和应用于其上的操作组成的库。
  7) 软件设计和程序设计语言应该支持抽象数据类型的定义和实现。

文件设计

- 文件设计是指数据存储文件的设计。
- 主要工作：

  根据使用要求、处理方式、存储的信息量、数据的活动性，以及所能提供的设备条件等，来确定文件类型，选择文件媒体，决定文件组织方式，设计文件格式，并估算文件的容量。
- 文件设计讨论的问题：
  1) 文件设计过程（逻辑设计和物理设计）
  2) 文件存储媒体的选择
  3) 确定文件的组织方式
  4) 确定记录的布局
  5) 预估存取时间
  6) 文件容量估算

软件开发

8 过程设计

主要内容

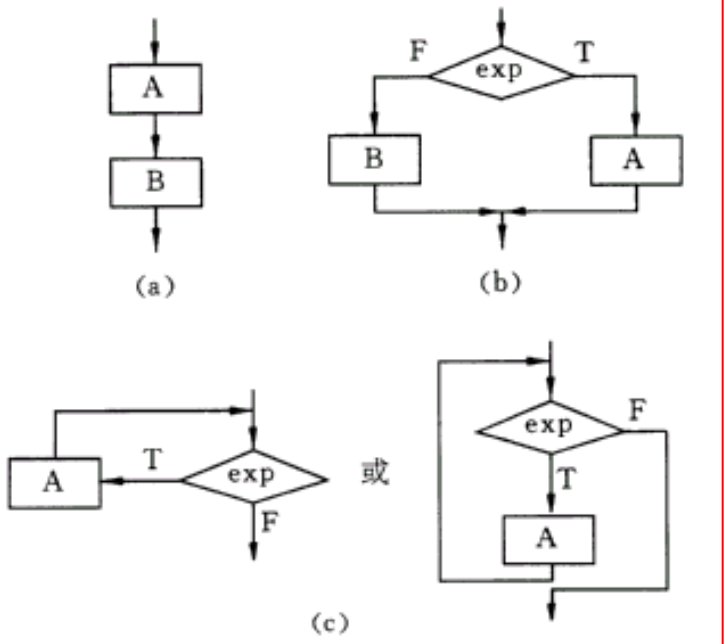
- 程序流程图
- N-S图
- PAD
- PDL
- 判定表
- 判定树

过程设计

- 过程设计也叫详细设计或程序设计。
- 在过程设计阶段要决定各个模块的实现算法，并精确地表达这些算法。
- 详细设计工具分3类：
  - 图形工具
  - 表格工具
  - 语言工具

8.1 程序流程图

程序流程图又称为程序框图，它是历史最悠久使用最广泛的描述过程设计的方法，然而它也是用得最混乱的一种方法。



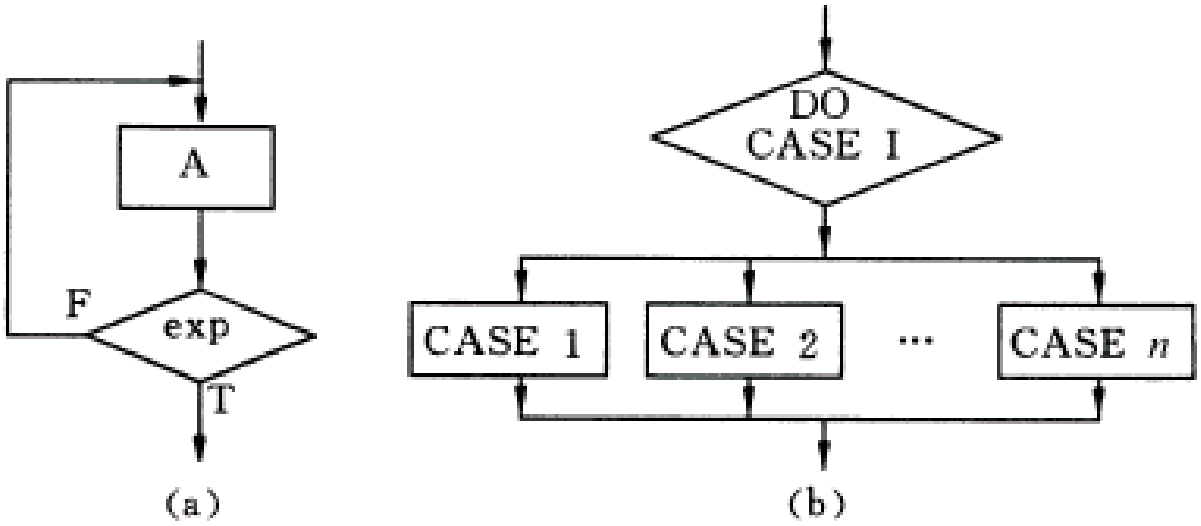
三种基本的控制结构：

(a)顺序结构，先执行A再执行B；

(b)IF-THEN-ELSE型选择(分支)结构；

(c)DO-WHILE型循环结构

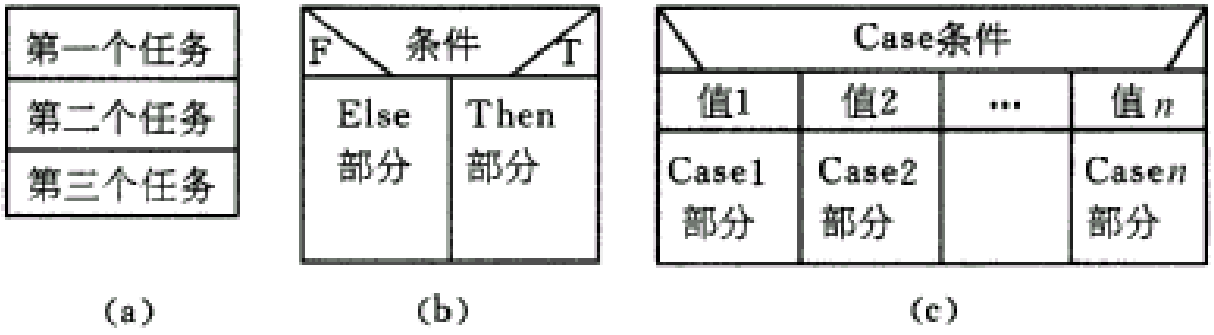
其他常用的控制结构

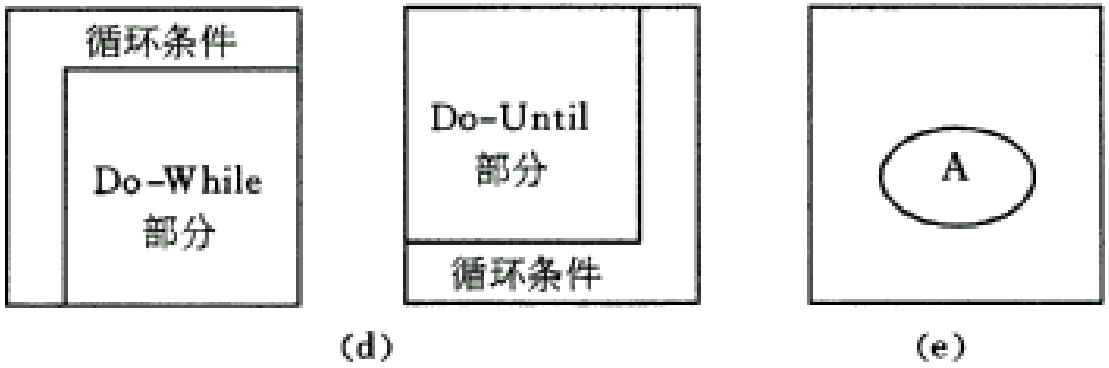


(a) DO-UNTIL型循环结构；(b)多分支结构

8.2 N-S图

- 由于流程图不是一种有效的结构化设计工具，Nassi和Shneiderman提出了盒图，又称为N-S图。
- N-S是一种结构化程序设计的图形描述工具。

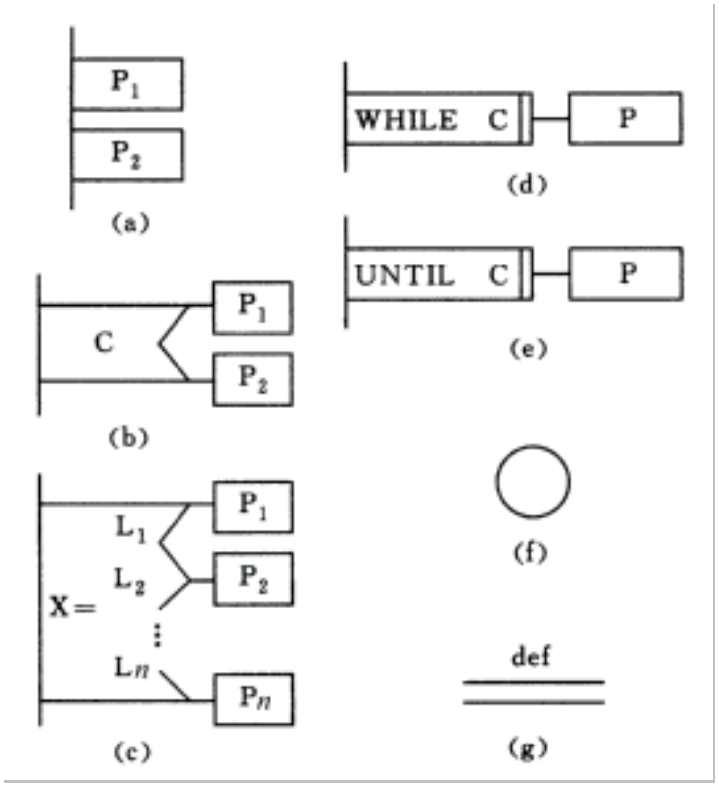




(a) 顺序；
(b) IF-THEN-ELSE型分支；
(c) CASE型多分支；
(d) 循环；
(e) 调用子程序A

8.3 PAD

- PAD(Problem Analysis Diagram)，自1973年由日本日立公司发明以后，已得到一定程度的推广。
- 它用二维树形结构的图来表示程序的控制流，将这种图翻译成程序代码比较容易。



(a) 顺序(先执行P1后执行P2)；

(b) 选择(IF C THEN P_1 ELSE P_2)；

(c) CASE型多分支；

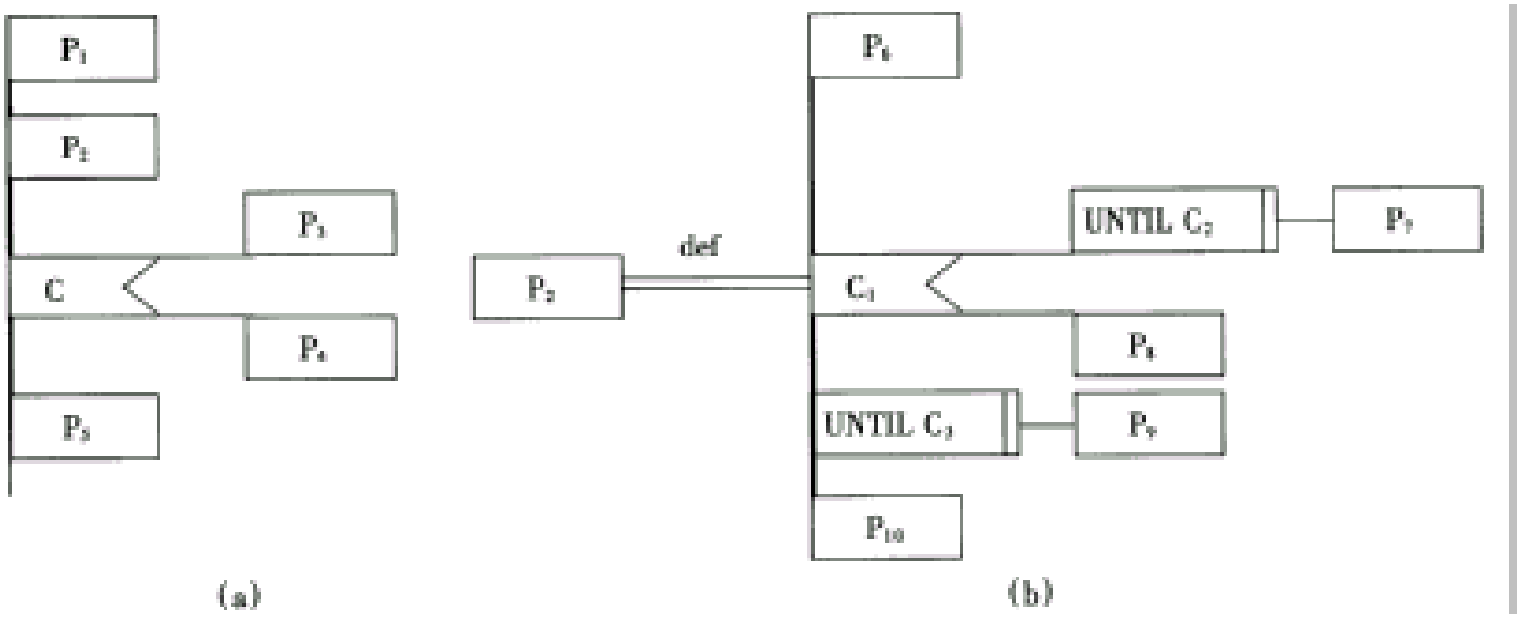
(d) WHILE型循环(WHILE C DO P)；

(e) UNTIL型循环(REPEAT P UNTIL C)；

(f) 语句标号；

(g) 定义

PAD例子



(a) 初始的PAD图；（b） 使用def符号细化处理框P2

8.4 判定表

- 当算法中包含多重嵌套的条件选择时，用程序流程图、N-S图、PAD图或PDL都不易清楚地描述。
- 然而判定表却能够清晰地表示复杂的条件组合与应做的动作之间的对应关系。
- 判定表由4部分组成：
  1. 左上部列出所有条件
  2. 左下部是所有可能做的动作
  3. 右上部是表示各种条件组合的一个矩阵
  4. 右下部是和每种条件组合相对应的动作。判定表右半部的每一列实质上是一条规则，规定了与特定的条件组合相对应的动作。

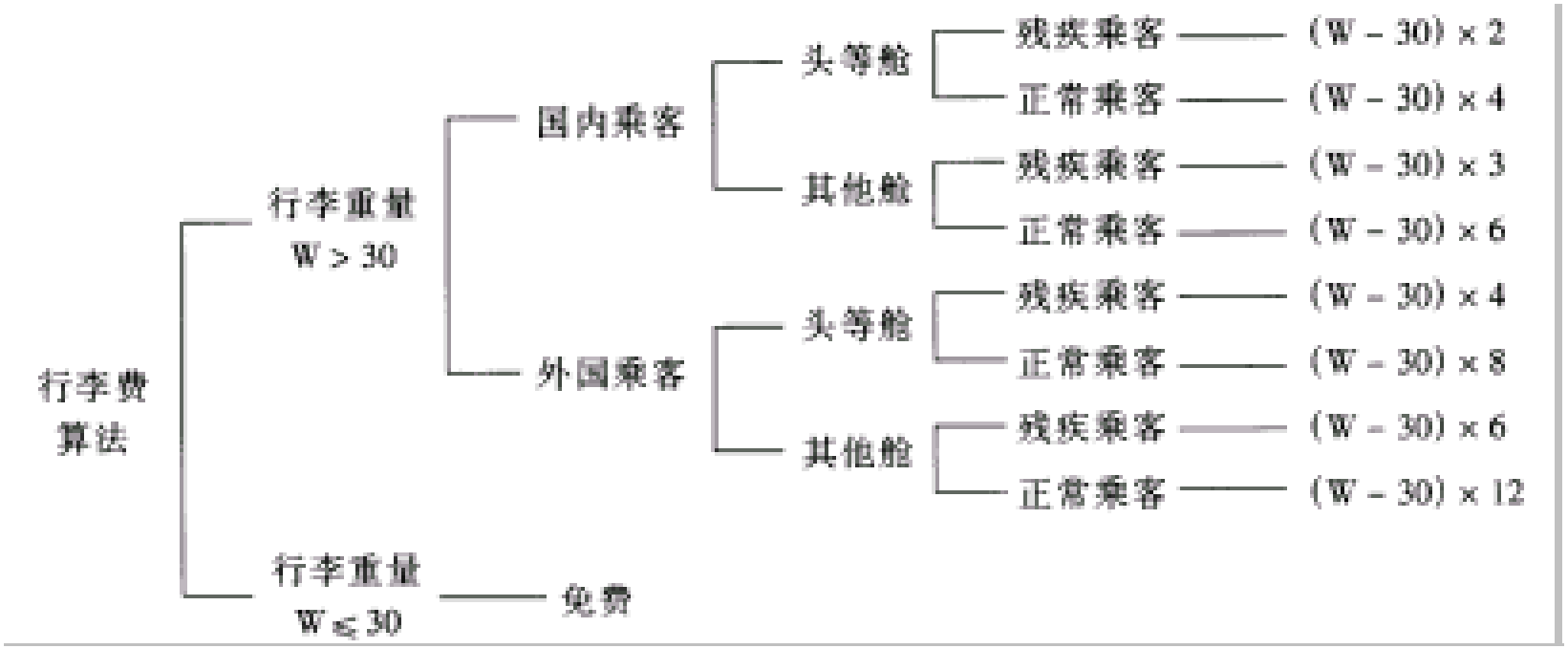
例：用判定表表示计算行李费的算法

	1	2	3	4	5	6	7	8	9
国内乘客		T	T	T	T	F	F	F	F
头等舱		T	F	T	F	T	F	T	F
残疾乘客		F	F	T	T	F	F	T	T
行李重量 W≤30	T	F	F	F	F	F	F	F	F
免费	×								
(W−30)×2				×					
(W−30)×3					×				
(W−30)×4		×						×	×
(W−30)×6			×						
(W−30)×8						×			
(W−30)×12							×		

判定树

- 判定表虽然能清晰地表示复杂的条件组合与应做的动作之间的对应关系，但其含义并不直观。此外，当数据元素的值多于两个时，判定表的简洁程度也将下降。
- 判定树是判定表的变种，也能清晰地表示复杂的条件组合与应做的动作之间的对应关系。

用判定树表示计算行李费的算法



8.5 PDL（Program Design Language）

- PDL也称为伪码，这是一个统称，现在有多种不同的过程设计语言在使用。它是用正文形式表示数据和处理过程的设计工具。
- PDL具有严格的关键字外部语法，用于定义控制结构和数据结构；另一方面，PDL表示实际操作和条件的内部语法通常又是灵活自由的，以便可以适应各种工程项目的需要。

例：PDL描述查找拼错单词的算法。

```
PROCEDURE spellcheck IS
    BEGIN
        split document into signal words
        look up words in dictionary
        display words which are in dictionary
        create a new dictionary
    END spellcheck
```

过程设计表示法比较

表示法 / 准则	流程图	N-S图	PDL	PAD	判定表（树）
易用性	优	优	优	良	中
逻辑表达能力	中	中	良	良	优
机器可读性	中	中	中	中	差
易转换程序代码	良	良	优	良	良
结构化	差	优	良	优	差
易修改性	差	差	良	差	良
数据表示能力	差	差	中	中	中
易验证性	差	差	中	中	中
使用频率	高	低	高	低	低

软件开发

9 程序设计

主要内容

① 源程序质量
② 程序设计风格
③ 程序效率

9.1 对源程序的质量要求

- 好的编程风格
- 好的界面
- 易排错
- 易测试
- 高性能
- 移植性好

9.2 程序设计风格

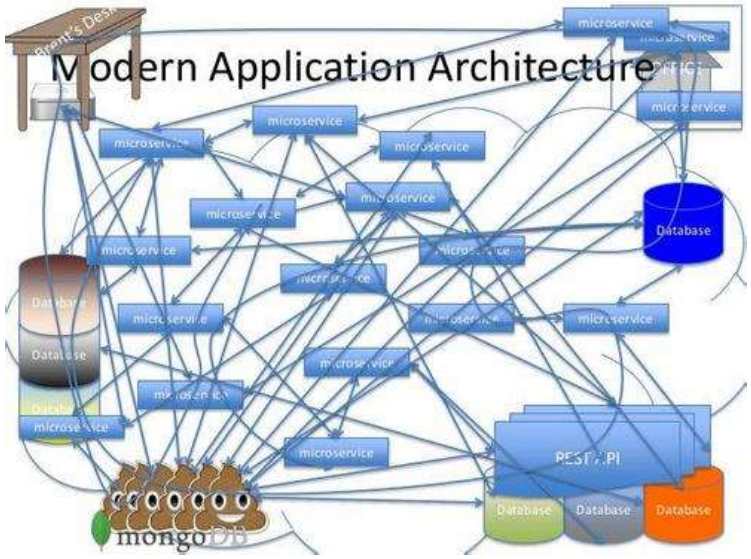
- 具有良好风格的代码更容易阅读和理解，几乎可以保证其中的错误更少。
- 程序不仅需要给计算机读，也要给人读。
- 好的风格使代码更容易读，无论是对程序员本人，还是其他读程序的人。
- 风格是一种习惯，通过规范形式给出。

9.3 程序效率

- 此处的性能主要是指软件的：
  - 运行速度
  - 存储空间
- 由于硬件的飞速发展，它对软件的速度和存储空间限制或要求越来越少。一个快速的程序如果给出的是错误结果，那么它一点也没有节省时间。

9.4 程序的复杂性

- 程序复杂度（区别于计算复杂度）是对软件内部诸多影响内部关联关系的属性的描述。
- 软件的高复杂度导致增加无意（与目的无关）的关联关系的风险，最终在做变更的时候更容易引入缺陷。在更极端的情况下，软件复杂到几乎无法修改。
- 有模块、类和程序三类复杂度：
  - 模块复杂度包含了关于模块的复杂度信息；
  - 类复杂度是针对那些使用McCabe面向对象特性的程序，它包含了关于类的复杂度信息；
  - 程序复杂度包含了关于程序的复杂度信息。



软件复杂性度量模型应遵循的基本原则：

(1) 软件复杂性与程序大小的关系不是线性的；

(2) 控制结构复杂的程序较复杂；

(3) 数据结构复杂的程序较复杂；

(4) 转向语句使用不当的程序较复杂；

(5) 循环结构比选择结构复杂，选择结构又比顺序结构复杂；

(6) 语句、数据、子程序和模块在程序中的次序对软件复杂性都有影响；

(7) 全程变量、非局部变量较多时程序较复杂；

(8) 参数按地址传递比按值传递更复杂；

(9) 函数副作用比显式参数传递更难以琢磨；

(10) 具有不同作用的变量共用一个名字时较难理解；

(11) 模块间或过程间联系密切的程序较复杂；

(12) 嵌套深度越深程序越复杂。

关于优化的讨论：

- 优化的第一要义是不做！
- 什么时候应该试图去加速一个程序？

改进性能的策略：

- 最好的策略是使用最简单、最清晰的算法和数据结构。
- 性能是其次要考虑的事情。
- 打开编译选择项，生成尽可能快的代码。
- 考虑对程序本身做什么改变才能有最大作用。
- 一次做一个改变并重新进行评价。
- 在改进性能过程中，始终保留最简单的版本，用它作为测试版本的对照物。

思考题

1. 对负责提取需求描述的工程师，如何搞清楚功能需求和非功能需求之间的关系？给出你的建议。
2. 当系统必须紧急变更的时候，系统中软件可能必须在需求变更被批准之前就修改。给出一个执行这些修改的过程模型，要确保需求文档和系统实现不会产生不一致。
3. 解释为什么在给出系统描述之前设计系统体系结构是必要的？
4. 讨论下面两个概念的区别：作为有效模块化特性的信息隐蔽和模块独立性。
5. 讨论在需求分析、系统设计和软件实现中的复用问题，如：什么可以复用、如何提高复用等？
6. 开发一个字处理软件，使用结构化分析为系统建立模型。